AALTO UNIVERSITY COURSE BOOK

Elements of Scalable Web Development

Architecture, measurement, scaling pressure, and agentic workflows

Juho Vepsäläinen

AALTO UNIVERSITY COURSE BOOK

Elements of Scalable Web Development

Architecture, measurement, scaling pressure, and agentic workflows

Juho Vepsäläinen

Travel edition - 5 October 2026

scalableweb.dev

READER'S MAP

Contents

This edition includes the core book, appendices, case studies, and the course project guide. Interactive activities link to the web edition.

Front matter

Introduction

Part I - Establish

Establish - Build and measure a baseline

1. What does scalability mean for web development?
2. How can coding agents support scalable development?
3. How can we measure web applications and development workflows?

Part II - Challenge

Challenge - Expose architectural assumptions

4. How does web architecture shape scalability?
5. How does development friction limit web projects?
6. Which web application assumptions fail under pressure?

Part III - Compare

Compare - Evaluate workflows and scaling responses

7. When do agent fleets improve web development?
8. How should web applications respond to scaling pressure?
9. What do three agent-assisted implementations teach us?

Appendices

Appendices

A. Evaluating agentic development workflows
B. Reproducible development environments
C. Advanced control boundaries in web applications
D. Agent-system security, control, and traceability
E. Formal scalability and reliability models
F. Durable workflows and retry-safe effects

Case studies

Slideotter - Learning faster than the product converged
Thesis Journey Tracker - Shipping by constraining the problem
Kirjolab - When code production scaled faster than architecture
Webernetes - Compatibility through source mapping and shared tests
Modern Web Guidance - Keeping web-platform advice current
Temper - Contracts, verification, and bounded autonomy

Course project guide

Course project introduction
Common project specification
Establish project
Challenge project
Compare project
Course development environments
Course assessment rubric

INTRODUCTION

Elements of Scalable Web Development

Created
January 16, 2026
Updated
October 5, 2026

Architecture, measurement, scaling pressure, and agentic workflows

Scalability describes the relationship between changing pressure and the behavior people expect a system to maintain. For a web application, that pressure may come from users, data, traffic, failures, geography, or new requirements. The response may involve architecture and infrastructure, but it can also change cost, complexity, ownership, and the speed at which the application can evolve.

This book therefore studies two connected systems. The first is the web application that receives requests, moves data, preserves authority, and produces browser-visible behavior. The second is the development system of people, agents, tools, instructions, tests, and review that changes the application. Scaling one while ignoring the limits of the other does not create a scalable project.

The central question is how application behavior and the development process can continue to meet their goals as pressure grows.

> **Sustainability angle**
>
> Scalability also has an environmental dimension. More traffic, more data, more retries, more logs, and more infrastructure all consume resources. Throughout the book, you can ask whether a design scales efficiently, not only whether it scales at all.

Concept connections
Explore definitions and connections in the web edition.
Explore this reading's concepts

0.1 Why this book now

If I had written this book only a few years ago, it would have focused strictly on web technologies rather than development itself. Coding agents changed that. Developers can now delegate substantial parts of implementation, moving more of their attention toward specification, architecture, validation, and ownership. Agents are also becoming consumers of web applications through browsers, APIs, and tools. This step up in abstraction brings its own opportunities and challenges, and this book examines how web development changes when agents participate on both sides of the application boundary.

During the past few months, I have developed projects whose complexity would previously have kept me from attempting them. The case studies examine several of them, including Slideotter, Kirjolab, and Thesis Journey Tracker. Each taught me something different: generation can outpace product judgment,

architecture can lag behind implementation, and deliberately narrow scope can turn the same capability into dependable software. Reading about other people's experience helps, but there is also value in making bounded mistakes of your own and forming your own judgment.

While the book was developed to support a course I run at Aalto University, I have tried to keep the chapters independent so that you can use it as reference material. The course project complements the book and lets you experience different ways of developing web applications with agents.

The book uses established vocabulary where possible. Terminology around coding agents and harnesses is still developing, and some labels describe this course's protocols or organizing framework. The concept reference gives working definitions, related sources, and “Usage in this book” notes where scope or terminology needs explanation.

0.2 How this book was made

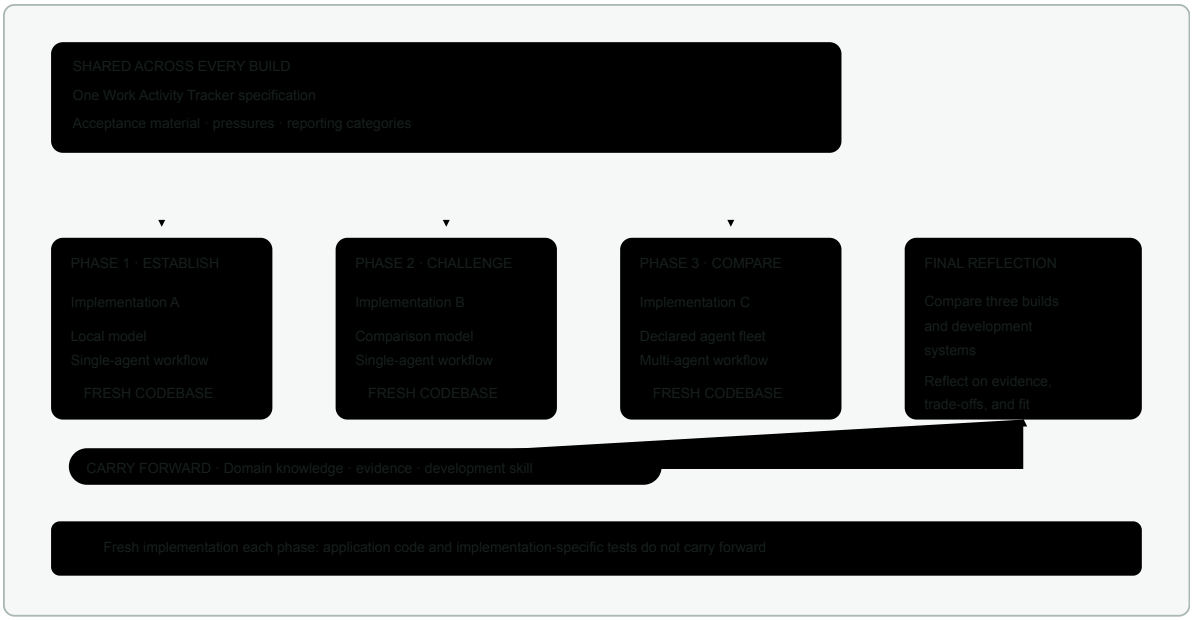
I developed this book with GPT-5.5 and GPT-5.6 through the iterative human–agent workflow it studies. The collaboration let me explore more material and build interactive demonstrations; validation and curation remained my central editorial work. Type and browser checks helped validate the publishing system; I reviewed the argument, source interpretation, and rendered teaching material. This account reflects my experience rather than a controlled comparison of productivity or learning outcomes. The online edition remains a living book whose feedback tools support continuing review.

0.3 One application, three development systems

The course project implements the same Work Activity Tracker specification three times. The required browser-visible behavior remains substantially stable while product interpretation, stack, rendering, storage, interfaces, and architecture remain open to justified choices.

Each phase begins from a fresh implementation and changes the development setup deliberately:

1. **Establish:** a locally hosted model in a single-agent workflow produces implementation A and its evidence baseline.
2. **Challenge:** a second, approved model in a single-agent workflow produces implementation B while the course exposes architectural assumptions and development friction.
3. **Compare:** a declared agent fleet produces implementation C before all three applications and development workflows are compared.



The task stays comparatively stable while the development system changes. Learning accumulates; application code does not.

The **comparison model** is the model assigned to Challenge, chosen to test a stated hypothesis about task quality, cost, or development effort. Local describes where inference runs; it does not name a capability class. Both phases may use foundation models: broadly pretrained models adaptable to many tasks, as defined in the Stanford report that introduced the term. Neither hosting location nor model size establishes which workflow will perform better on the course tasks.

The comparison is bounded rather than perfectly controlled. You enter each later phase with greater domain knowledge; implementations may make different design choices, and model or tool configurations can change. The shared specification, acceptance material, pressures, and reporting categories make those differences discussable without pretending that the course establishes a universal ranking.

0.4 How the argument develops

0.4.1 Establish — Build and measure a baseline

1. What does scalability mean for web development? defines scalability through maintained goals, pressures, constraints, and evidence.
2. How can coding agents support scalable development? examines how intent, authority, validation, correction, and ownership shape agent-assisted work.
3. How can we measure web applications and development workflows? creates comparable runtime and development evidence for implementation A.

0.4.2 Challenge — Expose architectural assumptions

4. How does web architecture shape scalability? traces how responsibility moves across servers, cloud, edge, rendering, data, and caches.
5. How does development friction limit web projects? follows the cost of changing one web behavior across those boundaries.
6. Which web application assumptions fail under pressure? uses documented incidents and the course project to connect failed assumptions, signals, and recovery.

0.4.3 Compare — Evaluate workflows and scaling responses

7. When do agent fleets improve web development? grows one agent into a bounded fleet and makes correction part of the workflow.
8. How should web applications respond to scaling pressure? selects architectural responses from measured limits rather than from a technique catalog.
9. What do three agent-assisted implementations teach us about web scalability? synthesizes a justified target architecture and development workflow from the three builds.

The main chapters provide the shortest path through the argument. Let your current task or question guide deeper exploration through the concept graph and curated starting points. Each starting point has a URL you can give directly to your agent, with a scenario to investigate one question at a time; you can also work through it yourself. The appendices preserve deeper evaluation methods, implementation details, security controls, and formal models for readers who need them.

PART I

Establish — Build and measure a baseline

Establish creates implementation A and a baseline for the later comparisons. Define what should remain true under pressure, build the common behavior through the approved local single-agent workflow, and record what the first implementation actually does.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

I.1 From scalability claims to comparable evidence

The three chapters form one progression:

1. What does scalability mean for web development? identifies the maintained goal, relevant pressures, constraints, and evidence a scalability claim would need.
2. How can coding agents support scalable development? treats the application and its development system as connected systems, with explicit constraints, validation, correction, and ownership.
3. How can we measure web applications and development workflows? turns runtime behavior and the development process into a repeatable evidence baseline.

Bind implementation A to a declared revision and evidence package so later comparisons can use recorded results.

I.2 Implementation A — Work Activity Tracker

The Establish project specification defines the implementation and evidence requirements. Choose a domain interpretation, stack, and architecture that fit the common contract and one question you want to investigate. Record the assumptions and trade-offs that later builds should test. Follow the shared transfer policy when carrying material into those fresh implementations.

I.2.1 Instructor perspective — Carry the learning, not the code

Building the application three times lets you compare development setups and see how growing domain knowledge changes your decisions. That learning is part of the course, even though it complicates causal attribution.

Agents can make a fresh implementation inexpensive enough to abandon a weak product interpretation or architecture. Carry forward the permitted specifications, acceptance material, evidence, and lessons instead of protecting code merely because it took effort to produce. By the third build, you should be better able to judge both the application and the workflow that produced it.

I.3 Implementation discovery lab

Use this lab to turn the shared specification into a direction to explore. It produces a first project-direction document that you can refine as implementation creates evidence.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/parts/establish/

CHAPTER 1

What does scalability mean for web development?

Created
January 16, 2026
Updated
October 1, 2026

A web application may cope with more requests while becoming harder to modify, accept more events while making old events too slow to query, or let more agents produce code while overwhelming review and integration. In this book, **scalability** means a system's ability to continue meeting a stated goal as a named pressure grows or changes. The goal, pressure, system boundary, and evidence are all part of the claim. Without them, scalability becomes a vague synonym for “large” or “well engineered.”

This framing is especially useful for the web because one user outcome can cross a browser, a network, an application boundary, persistent data, caches, external services, and operational tooling. The system also has to keep changing as browsers, consumers, requirements, dependencies, developers, and development agents evolve independently. Runtime scale and development scale are therefore related without being interchangeable.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

1.1 Begin with the goal that must continue to hold

Consider the event tracker at the center of the course project. At its smallest useful scale, a consumer records an event, the application validates and authorizes the request, accepted state becomes durable, and a later activity view exposes the result. A baseline should state which parts of that behavior matter rather than merely report that the application runs.

Different goals produce different scalability claims:

- A successful submission should still create one accepted event when many consumers submit concurrently.
- Recent activity should remain useful as retained history grows.
- A bounded feature change should remain understandable and reviewable as more people or agents contribute.
- An HTTP or data contract should remain evolvable as clients and requirements change independently.

Each claim can be expressed through the same frame:

> As **pressure P** changes within **range or scenario R**, the system within **boundary B** continues to meet **goal G**, as shown by **evidence E**, under **constraints C**.

The frame prevents one successful observation from becoming a universal promise. A browser measurement says little about database recovery; one load level does not prove behavior at another; and a fast generated change does not prove that another contributor can understand or reverse it. The Benchmarking chapter develops how to produce bounded evidence for these claims.

1.2 Four pressures organize the course

Many taxonomies of scalability are possible. This book uses four pressures that can be applied consistently to the event tracker and to its three independent implementations: demand, data, contributors, and change. These are the course's organizing dimensions, not an exhaustive or universally adopted taxonomy. Claims about contributors and change also concern maintainability, evolvability, and coordination; name the property being maintained rather than treating every dimension as one performance measure.

Table 1.1. Four pressures used to form bounded scalability claims in this book.

Pressure	What grows or changes	Goal that may need to hold	Early signals in a web project
Demand	Requests, concurrent actions, connected clients, traffic bursts, or geographic distance	Useful responses, accepted writes, latency, fairness, and correctness	Tail latency, error rate, saturation, queue depth, browser responsiveness
Data	Stored events, tenants, relationships, event variety, or retention time	Durable writes, useful queries, integrity, recovery, and bounded cost	Storage growth, write latency, query plans, replica lag, recovery time
Contributors	Developers, teams, agents, parallel tasks, or handoffs	Understandable ownership, reviewable changes, and reliable integration	Lead time, review queues, merge conflicts, rework, failed integrations
Change	Requirements, client versions, schemas, dependencies, policies, or deployment frequency	Compatible evolution, maintained behavior, and safe recovery	Change fan-out, regression rate, migration effort, failed rollbacks, recovery time

The dimensions interact without collapsing into one score. More demand usually creates more data. More contributors can increase the rate of change. Frequent schema changes can increase data migration work even when traffic stays constant. A design can therefore scale well along one axis while approaching a limit on another.

> **Sustainability angle**
>
> Meeting greater demand usually consumes more computation, storage, transfer, or human attention. A technically successful response may still be inefficient. Record resource use per useful outcome and the work displaced elsewhere instead of treating capacity as the only goal.

1.2.1 Demand changes the distribution of web behavior

Demand is not only a higher request count. More consumers create more concurrency, more varied devices and networks, more retries after ambiguous responses, and more opportunities for rare paths to occur. A browser may remain responsive while the origin saturates, or the server may respond quickly

while client rendering stalls on a growing activity view.

People are not the only consumers. Integrations, crawlers, and authorized agents can call an interface repeatedly and faster than a person operating a screen. The goal should therefore describe the authoritative outcome and acceptable user experience rather than assume one kind of client behavior.

1.2.2 Data changes paths as well as volume

Data pressure includes more than disk space. As event history grows, the useful write, read, analytical, deletion, archival, and recovery paths can diverge. A query that works for a few hundred events may scan too much history later; an index that accelerates one filter can increase write cost; and keeping every derived view immediately current can make ingestion harder.

The important questions are which state is authoritative, which views can be rebuilt, which results may be stale, and how long information should remain available. Those choices determine which data goals must be maintained as volume and variety change.

1.2.3 Contributors increase coordination pressure

A codebase that one person understands informally may become difficult to change when several people, teams, or agents work in parallel. More code output does not automatically increase accepted product output. Review capacity, integration, ownership, project memory, and the ability to diagnose failed work can become binding limits first.

Agents make this pressure especially visible because they can generate several plausible changes before a person could have implemented one. Stable contracts, focused tasks, executable checks, and explicit ownership can let contributors work more independently. They also require maintenance and can become harmful if they preserve rules that no longer serve the product.

1.2.4 Change makes old commitments visible

A web application changes even when its traffic does not. Browsers, APIs, integrations, event schemas, authentication rules, libraries, hosting platforms, and user expectations evolve on different schedules. Once a consumer relies on a response or production data adopts a representation, a change may require compatibility and migration rather than a simple code replacement.

Change scalability is the ability to keep making justified changes without turning every release into a coordinated rewrite. It depends on visible boundaries, useful tests, staged evolution, and recovery paths, but no design eliminates the need to understand what has already become a commitment. Planning for decision reversibility makes explicit which choices remain cheap to revise and which now require coordinated change.

Field note — When agent speed made me the bottleneck

Before AI-assisted development, I thought about scalability mainly through the conventional pressures of demand and data. Working with agents shifted my attention toward contributors and change.

Agents can implement changes far faster than I can. That makes my judgment and review capacity a binding limit, but removing that limit is not necessarily desirable. The pace creates a psychological risk I think of as **speed blindness**: becoming so accustomed to rapid output that I stop asking whether the work still serves the intended direction.

This is a hypothesis about my own workflow, not an explanation established by comparing other people's experiences. Task difficulty, repository familiarity, and tool configuration could also explain different outcomes. To challenge the hypothesis, I could alternate rapid generation and bounded changes on comparable tasks, recording review effort, defects found after acceptance, and whether I can explain the change unaided. If greater speed did not reduce understanding or increase missed defects under those conditions, speed alone would be a poor explanation for the risk I describe.

This book is an example. Agents and I are developing it together, but I still own its vision and remain responsible for whether it forms a coherent book for my students. The agents can help research, draft, restructure, and check it; they cannot take over that responsibility. For me, scaling agent-assisted work means preserving that ownership while taking advantage of the speed.

1.3 One pressure can cross the whole web system

Suppose a domain-specific event causes many consumers to submit events during the same short interval. That one demand pressure can affect several boundaries:

1. The browser must keep the interaction responsive and communicate pending, accepted, or failed state.
2. The HTTP boundary must handle concurrency, timeouts, retries, authentication, and request limits.
3. Application execution may exhaust workers, connections, memory, or dependency quotas.
4. The authoritative store must preserve event and workflow invariants under concurrent writes.
5. Queries and cached activity views must expose an acceptable level of freshness.
6. Operators need signals that distinguish user-visible failure from waiting, overload, duplication, or stale derived state.

Adding application servers helps only if application execution is the binding limit and the rest of the path can absorb the new work. Optimizing the browser helps only the work that occurs there. The response has to follow the pressure to the first limit that threatens the stated goal.

The same reasoning applies to development pressure. Adding agents can increase candidate changes while review or integration remains fixed. The result may be more waiting and rework rather than more accepted behavior. The scalable unit is therefore the complete path to a useful outcome, not the amount of activity at one stage.

1.4 Separate pressures, responses, and constraints

The earlier taxonomy deliberately names what varies. Several other concerns shape the response:

- **Architecture** assigns responsibilities, authority, contracts, and allowed dependencies across the system.
- **Infrastructure** supplies execution, storage, networking, deployment, and operational capacity.
- **Product and business priorities** determine which user outcomes matter, which risks are tolerable, and when additional capacity or coordination is worth its cost.
- **Constraints** such as budget, energy use, privacy, regulation, geography, hardware, provider quotas, time, and team expertise narrow the acceptable design space.

These concerns explain why applications facing similar demand may choose different responses.

A **binding limit** is the condition currently preventing the system from maintaining its goal as pressure increases. It may be a browser's main thread, network bandwidth, CPU, memory, a connection pool, database coordination, an external-service quota, a review queue, or missing expertise. Increasing capacity somewhere else can add cost and complexity without moving that limit.

1.5 Distinguish scalability from neighboring properties

Several useful properties are easy to conflate with scalability. The difference lies in the condition being studied.

Table 1.2. Scalability and related system properties answer different questions.

Property	Question	Example evidence from the event tracker
Performance	How does the system behave under one stated set of conditions?	Latency and resource use for one activity-view workload
Scalability	How does maintained behavior change as a named pressure changes?	Latency, correctness, and resource curves across increasing concurrent writes
Elasticity	Can supplied capacity expand and contract as demand changes over time?	Provisioning delay, backlog, and cost during and after a traffic burst
Resilience	Can the system preserve or recover useful service when something fails?	Accepted events and recovery time when storage or a dependency is unavailable
Efficiency	How much resource or coordination produces one useful outcome?	Compute, transfer, storage, or review time per accepted event or change

A system can perform well at its current load but degrade sharply as demand grows. It can add capacity automatically yet reproduce a correctness error on every instance. It can survive a dependency failure through expensive overprovisioning. No single metric establishes all five properties.

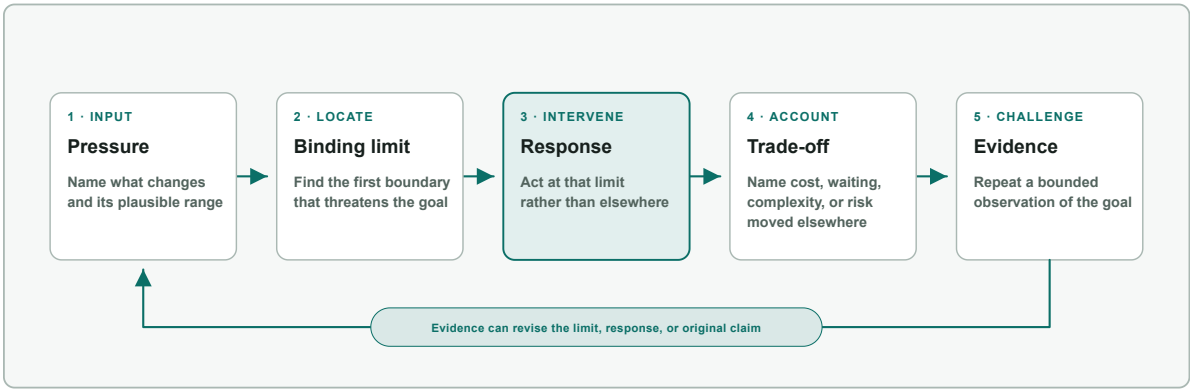
1.6 Reason from pressure to evidence

Use one causal chain before choosing a scaling technique:

```
pressure → binding limit → response → trade-off → evidence
```

- **Pressure:** name what changes and the plausible range or scenario.
- **Binding limit:** identify what first prevents the maintained goal from holding.
- **Response:** change the application, architecture, infrastructure, workflow, or scope at that limit.

- **Trade-off:** state which cost, latency, complexity, consistency, autonomy, or recovery obligation moved elsewhere.
- **Evidence:** repeat a representative observation that could challenge the claim.



A scaling response remains a hypothesis until bounded evidence challenges the maintained goal, limit, and trade-off.

For a runtime example, concurrent event submissions may exhaust the database connection pool before CPU becomes scarce. Admission control or buffering might protect accepted writes, but it can add waiting and require an explicit pending state. A useful evaluation would inspect success, duplication, tail latency, queue age, and recovery as the workload changes rather than only count server instances.

For a development example, parallel agent tasks may produce changes faster than people can review them. Smaller task boundaries, executable contracts, and automated checks might move repeatable findings earlier, but maintaining those controls also consumes time. Evidence should cover accepted outcomes, review delay, rework, failed integrations, and human intervention rather than generated lines of code.

This chain does not guarantee the correct response. It makes the hypothesis explicit enough to test and revise.

1.7 Constraints prevent premature scaling

Scalability is not a requirement to prepare every component for unlimited growth. An implementation that serves its likely demand, preserves its data, and remains understandable to its owners can be more appropriate than a distributed design built for a hypothetical future. Every additional cache, queue, replica, service, region, contract, and automation layer creates a new operating and change obligation.

Premature scaling begins when a solution is selected before the pressure, goal, or limit is known. Preparation is still valuable when it preserves a cheap path to learn: keep authority clear, avoid irreversible coupling without a reason, collect a baseline, and identify the signal that would justify the next move. Use that evidence to decide when the current architecture needs to change.

For the course project, time, available hardware, privacy, model access, team expertise, and implementation scope are legitimate design inputs. A bounded local implementation can be a strong baseline precisely because its constraints are visible. Later implementations can test different hypotheses without pretending that newer or more distributed means better.

1.8 Scalability dimensions lab

The demonstration below applies four pressures to the event tracker. Select the scenarios that matter and observe which dimensions receive the greatest relative strain. The weights are deterministic teaching inputs, not measurements of a real application, so use the result to form a claim rather than to select an architecture.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/what-is-scalability/

Pause & think

Check a scaling contract

A service must handle 200 requests/s while keeping p95 below 150 ms and errors at or below 1%. A run handles 220 requests/s with p95 = 180 ms and errors = 0.5%. Check the run against this contract.

▼ **Show worked answer**

Throughput and errors pass, but latency fails. Handling more requests is insufficient when the declared quality bound is violated.

Choose one leading dimension and write its pressure, maintained goal, likely first limit, and first useful signal. Then select a second pressure and inspect how the conclusion changes. If several dimensions rise together, preserve them as separate claims instead of averaging them into a single scalability score.

1.9 Project artifact

Produce two bounded scalability claims for implementation A: one about runtime demand or data, and one about contributors or change. For each claim, record:

- the goal that must continue to hold,
- the pressure and plausible range or scenario,
- the system boundary and relevant constraints,
- the likely first binding limit and signal,
- one candidate response and its expected trade-off,
- the evidence that would support or challenge the claim,
- and one dimension intentionally left outside the claim.

These claims become inputs to the shared measurement protocol in the Benchmarking chapter. They should be revised when evidence reveals a different limit, not protected as predictions that must appear correct.

1.10 Summary

A scalability claim states the goal, changing pressure, constraints, and evidence. Demand, data, contributors, and change help locate the first binding limit; architecture and infrastructure supply possible responses. Trace the response's trade-off and name an observation that could challenge it.

CHAPTER 2

How can coding agents support scalable development?

Created
April 30, 2026
Updated
October 5, 2026

Coding agents increase the rate at which requirements become candidate implementations. Development scales when people can understand, validate, integrate, and maintain those changes as the project evolves.

Concept connections
Explore definitions and connections in the web edition.
Explore this reading's concepts

2.1 What is an agent?

In classical AI, an agent observes an environment, chooses an action, and uses its effects to decide what to do next. The model applies both to physical robots and to programs acting in information environments (Poole & Mackworth, 2023).

In this book, a **coding agent** repeatedly observes a development environment and acts through tools to pursue a requested outcome. LLM practitioners vary in whether predefined workflows count as agents; this book uses the model-directed action loop as its working definition. Its Large Language Model (LLM) operates inside an **agent harness** that assembles context, exposes tools, carries state, invokes the model, and returns action results as feedback. Software-engineering agent research describes related perception, memory, and action modules around the LLM (Wang et al., 2025).

The table below shows the parts a developer can tune.

Table 2.1. Parts of a coding agent and the development choices that shape them.

Agent concept	Coding agent	What to tune
Goal	Requested outcome and acceptance criteria	Task examples, priorities, budgets, and stopping rules
Environment	Repository, terminal, browser, and services	Workspace setup, sandbox, and available services
Perception	Instructions, file reads, tool output, and images	Retrieval, provenance, and feedback quality
Controller	LLM and its observe–decide–act loop	Model configuration, context assembly, and orchestration
Memory	Session state and durable project knowledge	Selection, freshness, retention, and source authority
Actions	Edits, commands, API calls, and messages	Permissions, validation, rollback, and rate limits

Observations can be incomplete, tools can fail, and the stated goal can omit something the user needs. Improving an agent therefore involves its environment and feedback as well as its model and prompt.

2.2 What changes when agents implement code?

Specifications, executable tests, and iterative feedback predate coding agents. The change is that a probabilistic tool-using system can perform more of the implementation loop. The table below compares three composable patterns, not stages in a maturity ladder.

Table 2.2. Three development patterns that can coexist in one project.

Pattern	How implementation proceeds	Where control lives
Human implementation	People translate requirements into code through planned stages or short iterations	Design decisions, executable examples, and review
Deterministic generation	Generators transform declared models into implementation artifacts	Model definitions, transformation rules, and supported escape paths
Agent execution	A model interprets context and chooses tool actions through a harness	Maintained instructions, scoped tools, checks, and acceptance decisions

Iterative development, model-driven generation, and executable behavior examples can all contribute to an agent-assisted workflow. The event tracker could use BDD scenarios to define acceptance, generate an API client from a schema, and delegate implementation to an agent. Across these patterns, a requested outcome still has to become a checked change someone can maintain.

2.3 Constrain, generate, validate, revise

Agentic development follows a repeated loop: constrain the task, generate a candidate, validate it, and revise both the result and the project's durable knowledge. This is a corrective feedback loop: evidence of a deviation informs the next revision. Feedback can also reinforce a change, as when timeouts trigger retries that increase load and cause more timeouts; the Scaling chapter examines that case. Useful throughput is **accepted change**: work that someone can explain, validate against the intended behavior and architecture, integrate, and remain willing to own.

For the event tracker, consider adding CSV export. The task must settle which events are included, which fields may leave the application, and how the exported values are checked. A working download button alone leaves those decisions unresolved.

The table below identifies the evidence needed to advance each stage.

Table 2.3. The constrain–generate–validate–revise loop and the evidence that advances each stage.

Stage	Governing question	Main action	Exit evidence
Constrain	What outcome is wanted, and what may vary?	State the goal, context, boundaries, budget, and stopping or escalation rule	A bounded task and explicit acceptance criteria
Generate	Which candidate might satisfy those limits?	Let a chosen model and agent harness act with relevant context and tools	A candidate artifact plus its trace, cost, and attempts
Validate	Does the candidate satisfy the real need?	Apply independent checks and inspect the artifact a user will consume	Acceptance evidence and known unverified properties
Revise	What should change before the next attempt?	Correct the result and codify learning in durable project material	An improved artifact and better starting constraints

Constrain. State the outcome, relevant context, allowed variation, budget, forbidden effects, acceptance criteria, and escalation rule. Preserve recurring constraints in project instructions such as `AGENTS.md`. Treat initial requirements as hypotheses that interaction with a prototype may refine

(Brooks, 1987). Programming also builds the understanding needed to explain and change the system (Naur, 1985).

Generate. Choose the model and harness using evidence from representative work. Runtime configuration, context, tools, hardware, routing, and fallback can change the outcome, cost, and data exposure. Appendix A explains how to identify the configuration that actually served a task. Focused retrieval can reduce irrelevant context, but lossy views can hide decisive facts; keep important raw output retrievable and test the reduction policy.

Validate. Check the candidate against the intended behavior and inspect the artifact the user will consume. The Webernetes case compares behavior with an independent implementation; Slideotter checks geometry and rendered output. Agent-generated checks need scrutiny too. In one workshop report, screenshot tests did not exercise the application, supposed passkey authentication accepted a username, and stored data disappeared after restart (Talwar, 2026). Those failures suggest concrete checks: launch the real application, challenge authentication, and verify persistence across restart.

A reversible mutation can strengthen this relationship by checking that a test distinguishes the accepted implementation from one named fault. That contrast does not prove the mutant is realistic or the broader requirement is satisfied, but it is stronger than coverage alone. Tests must also tolerate correct changes. Bromley (2026) illustrates generated tests that copy implementation logic or accept either outcome; a complementary check is whether the same tests accept a behavior-preserving refactor. The concern-directed mutation lab lets you apply both checks to authorization, retry, or archived-state behavior: pass the baseline, fail the selected fault, and pass a variant that stores the events differently. Study its source or download the standalone lab.

Revise. Correct the candidate and preserve the lesson in requirements, tests, architectural decisions, or project instructions. The next attempt should begin with better information about the failure.

2.3.1 Constraint feedback loop

The demonstration starts with an underspecified request to add CSV export to an event list. Generate a candidate, validate it, revise the project memory from failed checks, and begin the next round. Compare a run with the default constraints with one that selects every constraint before generation.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scalable-development/

The candidate is deterministic teaching data, not model output. Use validation failures to revise the project's rules and context before the next generation attempt.

2.4 Assign authority and ownership

At each stage of the loop, make **action authorization** explicit: decide who may act, which evidence permits the next step, and who owns the result. For each decision, an agent may **propose** an option, **prepare** an action for approval, or **execute** within explicit limits. Ambiguous intent, weak evidence,

irreversible commitments, and material external effects should narrow that authority.

For the CSV export, an agent might propose fields, implement the agreed format, and run checks; the accepting person still decides whether the exported data and access rules meet the requirement. The Agent Fleets chapter extends this allocation across delegated tasks.

The table below separates the records needed to explain and stand behind that change.

Table 2.4. Questions that separate an artifact's origin, endorsement, accountability, and evidence.

Concept	Question it answers	Record to preserve	Does not prove
Provenance	Which exact revision or artifact is this, and which inputs, people, and tools produced or transformed it?	Artifact identity, revision, material inputs, actors, and tools	That the artifact is correct
Endorsement	Who stands behind which claim or decision on the final revision, including any reservations?	Named endorser, endorsed claim, scope, revision, and reservations	That every property was checked
Accountability	Who owns the consequences and must correct, maintain, or retire the result if it becomes wrong or outdated?	Accountable owner and correction or retirement obligation	That the owner produced the artifact
Evidence	Which properties were checked on that revision, under what conditions, and what remains unverified?	Check, environment, result, revision, limits, and omissions	Universal quality outside those checks

Bind acceptance decisions, checks, and known limits to the revision they concern. Perceived effort can influence quality judgments (Kruger et al., 2004), but time spent does not establish correctness.

2.4.1 Sustainable throughput requires ownership

The ownership standard should follow the work's consequences and expected lifetime. A disposable experiment may need little maintenance; shared code makes others inherit the cost of understanding, operating, and correcting it—what Cory Doctorow calls accretive work. Publishing an issue or patch also creates work for an external maintainer, so someone must own the claim and respond when it is wrong.

> **Sustainability angle**
>
> Count the cost of repeated generation, validation, and human correction alongside accepted changes. Smaller tasks and maintained project guidance are useful when they reduce rework without weakening the checks.

2.4.2 Optional exercise — Move a prototype into shared use

What would turn a working demo into a tool someone can operate and own? Give your agent the prototype starting point to investigate one decision at a time, or read the scenario yourself.

2.5 The project harness

The **project harness** is the versioned material that lets humans and agents change a repository: instructions, specifications, architecture notes, setup, examples, and validation commands. Project harness is the book's working label for that bundle of repository guidance and executable development controls; harness terminology varies between sources. It supplies project knowledge to the agent harness introduced earlier.

Table 2.5. Three related control layers in agentic development.

Term	Scope	What it includes	Governing question
Agent harness	One model-and-tool execution loop	Context assembly, tools, state, permissions, model invocation, and action feedback	How can the model observe, decide, and act?
Project harness	One repository or maintained codebase	Instructions, specifications, architecture notes, environments, commands, tests, rules, and examples	How can a contributor change this project without hidden context?
Development system	The process that changes an application	People, agents, agent harnesses, project harnesses, review, integration, ownership, and organizational policy	How does candidate work become accepted and maintainable change?

A useful project harness lets a new contributor clone the project, locate the relevant behavior, run the checks, and make a small change without oral rescue. The common project specification defines the required artifacts. vibe-template provides one example of keeping starter code, instructions, architecture rules, and validation together. The Friction chapter develops fresh-checkout validation and the choice between focused and broader checks.

An analysis of 3,864 public bug reports across Claude Code, Codex CLI, and Gemini CLI attributed 37.3% of root causes to API, integration, or configuration errors (Zhang et al., 2026). That sample cannot describe every agent failure, but it gives a reason to inspect tool contracts, commands, and configuration when improving the workflow.

2.5.1 Agents learn patterns from your codebase

Accepted code also becomes context for later work. Robinson (2026) describes allowing duplicated authorization checks on the assumption that an agent could maintain them later; the agent instead copied those shortcuts into subsequent changes. Review whether a change leaves a pattern worth repeating, as well as whether its present behavior passes.

2.5.2 Bound probabilistic generation with deterministic controls

Two agent runs may interpret a task differently while obeying the same state transitions, permissions, and acceptance rules. David Khourshid's Beyond the Prompt talk proposes letting an LLM choose among the events a deterministic state machine currently permits. The demonstration applies that pattern to delivering a code change.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scalable-development/

Pause & think

Follow the release gates

In this delivery statechart, release readiness requires both passing checks and an approved review. Checks have passed, but review has not started. Select the missing gate and the destination of REQUEST_CHANGES during review.

▼ **Show worked answer**

Passing checks alone does not complete the parallel verification state. Review must also approve. REQUEST_CHANGES returns the patch to implementation.

One cancellation rule covers the active state, tests and human review can proceed independently, and either failure can return the workflow to implementation. The statechart restricts the available routes; the checks and reviewer still determine whether the code meets the requirement. Deterministic controls can encode a wrong or incomplete rule. The Architecture chapter develops the related controls around data representation and durable state changes.

The Temper case study extends this pattern to agent-authored state machines and compares their verification boundary with Datadog's system testing and bounded live optimization.

2.5.3 Reuse practices without cloning the whole project harness

Copying a complete starter repository into an existing project is usually too coarse. Package a reusable capability with its fit criteria, files, dependencies, integration instructions, checks, provenance, and update path so a project can adopt the practice without inheriting every source-template decision.

Modern Web Guidance shows how a maintained external capability can package fast-changing domain knowledge with targeted retrieval, compatibility constraints, an update path, and task-level evaluation. Imported capabilities also create a trust boundary because instructions or bundled code may influence tools operating with the agent's authority. The Agent Fleets chapter develops the security implications, while the skill-adoption lab provides a bounded review exercise; study its source or download the standalone lab.

2.6 When generation outruns comprehension

An agent can increase the arrival rate of candidate changes while comprehension, review, and integration remain constrained. The resulting backlog appears as large diffs, delayed review, shallow acceptance, rework, or technical debt. The table below collects evidence about different parts of that mismatch.

Table 2.6. Evidence behind the distinction between candidate production and accepted change.

Evidence	What it supports	Important limit
Three field experiments covering 4,867 developers found about 26% more completed tasks with an AI coding assistant (Cui et al., 2026)	Candidate production can increase in some settings	Individual experiments were noisy, and the result does not apply equally to every person or project
A study of 78 developers across seven projects classified about 58% of observed time as program comprehension (Xia et al., 2018)	Understanding already consumed substantial development capacity before generative AI	Two companies using Java and C# do not establish a universal reading-to-writing ratio
A professional code-review study found that understanding supports knowledge transfer and alternative solutions as well as defect detection (Bacchelli & Bird, 2013)	Review is broader than checking whether generated code passes	The study predates current coding agents
A live LLM-review study at Mozilla and Ubisoft found that generated comments sometimes informed revisions, but reviewers accepted only a minority (Olewicki et al., 2026)	AI-assisted review can contribute evidence while still consuming human judgment	One deployment does not establish the effectiveness of every review agent

Types, tests, small modules, and architectural notes can reduce the work needed to understand a change. Their value is the uncertainty they resolve. Evolving software still needs deliberate work to maintain or simplify its structure (Lehman, 1980).

2.6.1 Test an explanation against behavior

An agent can help you approach unfamiliar code through concrete examples, data-flow diagrams, and questions. The explain-diff prompts combine surrounding-code context, intuition, a grouped walkthrough, and a comprehension quiz. These are proposed aids to inspection; a model that writes both the explanation and its quiz can repeat the same mistaken assumption in both.

For a consequential claim, predict an observable result, inspect the complete relevant code, and check that prediction against behavior. Separate what the implementation guarantees from conditions the explanation assumes, such as a worker staying alive or a request reaching the same instance. The optional explanation challenge uses a small event recorder to practice finding and correcting an overstatement.

2.6.2 Field note — Kirjolab

In Kirjolab, agents helped me reach a scope and level of integration I would not have attempted alone. Functional checks made later refactoring safer, but they had not prevented unclear ownership and difficult navigation. The case study records the development history and the structural feedback I carried into subsequent work.

2.6.3 Choose what to participate in

A workflow can produce acceptable software and leave its owner able to explain it, while removing an activity they wanted to experience. Choose assistance according to the result you need, the skills you need to retain, and the activities you want to take part in.

The table below separates three outcomes when choosing an assistance mode.

Table 2.7. Different outcomes of assistance require different evidence.

Outcome	Question	Evidence
Accepted output	Does the result meet the intended need?	Independent behavioral checks and acceptance review
Retained competence	Can the person explain and change it later?	Delayed unaided explanation or repair
Valued participation	Which parts of the work does the person want to experience?	Stated purpose, preferences, and reflection

Learning, investigating with colleagues, and enjoying implementation can be purposes of the work. Choose what to delegate in light of those purposes as well as the result's quality and cost. Manual work, agent critique, supervised implementation, and delegation can each fit a different task. Finishing the agreed work can be a reason to stop, even when the agent has capacity for another task.

Appendix A's optional Navigator exercise compares agent guidance with delegated implementation, then checks later unaided explanation and repair.

2.7 Project artifact

For implementation A, build the required project harness alongside the application. Use the common evidence bundle to show:

- how another group can find the core event workflow, run it, and validate a change;
- one consequential stack or storage decision and its owner;
- one representative AI-assisted task, its configuration and checks, human correction, acceptance decision, and known limits.

These are checkpoints within the existing submission bundle, not additional standalone deliverables.

2.8 Summary

The development loop turns a requested outcome into a checked, maintainable change. Its project harness carries constraints and learning into later work, while people retain responsibility for acceptance and correction. When generation exceeds review and integration capacity, improve those parts of the loop before increasing output.

2.9 References

1. Zhang, R., Dai, W., Pham, H. V., Uddin, G., Yang, J., & Wang, S. (2026). Engineering Pitfalls in AI Coding Tools: An Empirical Study of Bugs in Claude Code, Codex, and Gemini CLI. arXiv Preprint arXiv:2603.20847. https://doi.org/10.48550/arXiv.2603.20847
2. Cui, K. Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S., & Salz, T. (2026). The Effects of Generative AI on High-Skilled Work: Evidence from Three Field Experiments with Software Developers. Management Science. https://doi.org/10.1287/mnsc.2025.00535
3. Xia, X., Bao, L., Lo, D., Xing, Z., Hassan, A. E., & Li, S. (2018). Measuring Program Comprehension: A Large-Scale Field Study with Professionals. IEEE Transactions on Software Engineering, 44(10), 951–976. https://doi.org/10.1109/TSE.2017.2734091
4. Bacchelli, A., & Bird, C. (2013). Expectations, Outcomes, and Challenges of Modern Code Review. Proceedings of the 35th International Conference on Software Engineering, 712–721. https://doi.org/10.1109/ICSE.2013.6606617
5. Olewicki, D., Da Silva, L. M. P., Ben Sghaier, O., Mujahid, S., Amini, A., Mah, B., Castelluccio, M., Habchi, S., Khomh, F., & Adams, B. (2026). Impact of an LLM-Based Review Assistant in Practice: A Mixed Open-/Closed-Source Case Study. IEEE Transactions on Software Engineering, 1–12. https://doi.org/10.1109/TSE.2026.3663093
6. Talwar, S. (2026, July). LLMs Can't Program. functional.computer. https://functional.computer/blog/llms-cant-program
7. Brooks, Jr., Frederick P. (1987). No Silver Bullet—Essence and Accidents of Software Engineering. Computer, 20(4), 10–19. https://doi.org/10.1109/MC.1987.1663532

8. Lehman, M. M. (1980). Programs, Life Cycles, and Laws of Software Evolution. Proceedings of the IEEE, 68(9), 1060–1076. https://doi.org/10.1109/PROC.1980.11805
9. Naur, P. (1985). Programming as Theory Building. Microprocessing and Microprogramming, 15(5), 253–261. https://doi.org/10.1016/0165-6074(85)90032-8
10. Robinson, S. (2026, July). Write Code Like a Human Will Maintain It. Unstack. https://unstack.io/write-code-like-a-human-will-maintain-it
11. Kruger, J., Wirtz, D., Van Boven, L., & Altermatt, T. W. (2004). The Effort Heuristic. Journal of Experimental Social Psychology, 40(1), 91–98. https://doi.org/10.1016/S0022-1031(03)00065-9
12. Poole, D. L., & Mackworth, A. K. (2023). Artificial Intelligence: Foundations of Computational Agents (3rd ed.). Cambridge University Press. https://doi.org/10.1017/9781009258227
13. Bromley, M. (2026, September). The Problem With Your AI Tests. Michael Bromley's blog. https://www.michaelbromley.co.uk/blog/the-problem-with-your-ai-tests/
14. Wang, Y., Zhong, W., Huang, Y., Shi, E., Yang, M., Chen, J., Li, H., Ma, Y., Wang, Q., & Zheng, Z. (2025). Agents in Software Engineering: Survey, Landscape, and Vision. Automated Software Engineering, 32(2), 70. https://doi.org/10.1007/s10515-025-00544-2

CHAPTER 3

How can we measure web applications and development workflows?

Created
January 16, 2026
Updated
October 5, 2026

A page that feels fast on one developer's machine may behave differently under another workload, device, or network. Benchmarking makes that comparison explicit; observability helps investigate the result. Both need checks that the measured property still represents the outcome people want.

Concept connections
Explore definitions and connections in the web edition.
Explore this reading's concepts

3.1 Benchmark to test a claim

A **measurement** records a value for a property using a stated procedure. A **benchmark** uses declared conditions to compare that property across workloads, systems, or revisions. Begin with the decision the comparison should support: for example, whether the event tracker still records events within an agreed latency objective as concurrent demand increases.

Define the protocol before selecting a runner:

- **Decision and claim:** What comparison or threshold would change an engineering decision?
- **Workload:** Which event-recording, querying, filtering, or browser actions are performed, and in what mix?
- **Load:** Do arrivals follow previous completions or an independent schedule, at what concurrency or rate, and for how long?
- **Boundary and conditions:** Which application revision, environment, data volume, cache state, dependencies, and client conditions are included?
- **Metrics and distribution:** Which latency, throughput, error, resource, or queue measurements answer the question, and which percentiles or ranges preserve important variation?
- **Acceptance and limits:** What result would support or challenge the claim, and what does the setup leave untested?

Tools such as Artillery and k6 can generate load, schedule scenarios, and collect observations from that protocol. They cannot decide whether the workload represents real use, whether the boundary matches the question, or whether the comparison supports a statistically defensible conclusion. An agent can help draft a script, but the generated workload, metrics, thresholds, and interpretation still have to be reviewed against the intended decision.

3.1.1 Check whether the generator hides overload

A **closed-loop** worker waits for a result before issuing its next operation; an **open-loop** generator schedules arrivals independently of completions. Suppose four closed-loop workers immediately repeat requests with no think time or overhead. At 0.1 seconds per request they offer about 40 requests per second; at one second they offer only four. The same concurrency setting has reduced demand just when the application became slower. By contrast, an independent stream of 20 requests per second facing a service capacity of four creates about 16 additional waiting or rejected requests per second, under this simplified constant-rate model. Neither experiment substitutes for the other: the open-versus-closed workload study by Schroeder, Wierman, and Harchol-Balter demonstrates why the arrival model changes the interpretation.

The shared project baseline uses a small closed-loop workload and fixes the timing boundary, deadlines, outcome counts, and percentile rule. Its higher-concurrency profile is not evidence of sustaining an independent production arrival rate. Use the bounded arrival-controlled pressure run when testing queues, overload, or admission control. Count offered, admitted, and completed work separately: a lower latency among successful requests can coexist with more rejected or unfinished work. Timeouts supply deadline lower bounds, not observed completion times; report them beside the completed-request latency distribution.

3.1.2 Simulated benchmark lab

The demonstration below models 60 requests to an event-recording endpoint. Change the number of concurrent clients, the share of requests that take a slow storage path, and the p95 latency objective. The values are deterministic and run only in the browser, so this is a teaching model rather than a capacity test.

Pause & think

Calculate the average and the tail

A fixed sample contains 20 requests: 18 take 40 ms each and two take 240 ms each. Use the nearest-rank percentile: sort the values, then take rank ceil(0.95 × 20). The objective requires latency strictly below 120 ms. Calculate the mean and p95, then choose which meet the objective. This sample is separate from the adjustable simulation in Explore.

▼ **Show worked answer**

The total is 18 × 40 + 2 × 240 = 1200 ms, so the mean is 60 ms. Rank 19 is 240 ms. The average meets the objective while p95 does not; an average can conceal a slow minority.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/benchmarking/

Predict the default result before running the simulated workload, then compare the average with p95. Treat the first result as a baseline, change one condition, and predict again before recording the comparison. Record the workload, load, metric, and acceptance threshold together.

Synthetic benchmarks deliberately simplify reality. They can omit workload mixes, client behavior, cache histories, data shapes, provider quotas, dependency contention, and failure combinations that appear in production. A result therefore supports a claim only for its declared conditions. Compare synthetic evidence with field telemetry where available, and revise the protocol when real behavior exposes a missing condition.

3.1.3 Benchmark browser journeys with Playwright and Lighthouse

Server latency does not describe the whole user experience. A browser still has to download resources, parse and execute JavaScript, calculate styles, lay out the page, paint it, and respond to input. Three related tools expose different parts of that work:

- The browser's Performance APIs can measure marks, durations, resources, and selected rendering events in the page itself.
- Playwright drives a real browser through a repeatable user journey and checks whether the expected behavior occurred.
- Lighthouse audits a page in a controlled lab setup and reports performance metrics and diagnostics.

These tools are complementary rather than interchangeable. A Playwright test can prove that filtering the event list produces the expected rows, but its elapsed test time is not a Web Vital. A Lighthouse navigation can diagnose initial page load, but it does not prove that the application's business flow works. Lighthouse user flows can audit interactions, although they require browser automation outside the page; a normal web page cannot grant itself the required browser-control privileges.

Browser performance workbench

The workbench below runs completely in the current page. It builds and filters an event list either as one uninterrupted task or in batches that yield to the browser. It measures from the start of the work through two animation-frame callbacks after the DOM work, repeats the same flow five times, and reports total elapsed time and the longest instrumented JavaScript slice. The callbacks provide rendering opportunities, not a timestamp proving that pixels reached the display (HTML rendering algorithm).

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/benchmarking/

Before each measurement, predict which strategy will finish sooner and which will leave the main thread available sooner. Run both strategies with the same number of events. Then increase the workload and check whether the conclusion remains the same.

This is a real measurement, but it is deliberately narrow. The timer includes work performed by this demo on this browser and device; it is not a Lighthouse score, INP measurement, or claim about other users. The chunked strategy may take longer overall while shortening the measured JavaScript segments

and leaving opportunities for other browser work. The longest-slice metric excludes the forced layout after those segments and other rendering work; the demo does not measure input delay, so it cannot by itself establish improved responsiveness. This distinction between throughput and responsiveness is often more useful than asking which version is simply "faster."

For client-side collection in a real application, the small web-vitals library implements LCP, INP, CLS, FCP, and TTFB using browser APIs and follows the same measurement rules used by Google's web-performance tools. Those measurements describe actual page visits. Lighthouse instead provides synthetic lab evidence under declared conditions.

From page metrics to product metrics

A page-level metric may describe a different element from the one the user came to see. For the event tracker, define a milestone such as the event list becoming visible and usable. Element Timing can report rendering of a supported annotated element; application instrumentation and interaction checks can establish the remaining behavior. Choose the signal from the product question, name its collection rule, and record the browsers and users represented. Paint evidence alone cannot establish that a region is interactive or correct.

Use Appendix B's browser measurement procedures for repeatable page audits and a standalone Playwright journey. Choose the procedure that answers the product question and report the conditions and limits of its evidence.

3.1.4 Check measurements against a model

Measurements become more useful when they can be checked against a model. Little's Law states that, for a stable system over a suitable observation period, average work in progress (`L`) equals the average arrival rate (`λ`) multiplied by average time in the system (`W`) (Little, 1961):

```
L = λW
```

For an event pipeline, `L` can be the average number of events waiting or being processed, `λ` the average arrival rate, and `W` the average time from arrival to completion. If 20 events arrive per second and each spends half a second in the system, the expected average work in progress is 10 events.

Queue balance lab

Use the lab below to check whether three observed averages are mutually consistent. Change one value at a time and explain whether the mismatch suggests overload, incompatible measurement windows, or a missing boundary.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/benchmarking/

Pause & think

Check whether the averages agree

At one declared boundary you observe L = 25 operations, λ = 40 operations/s, and W = 0.25 s. Calculate λW, then select the conclusion supported by these three values alone.

▼ **Show worked answer**

λW is 10, not 25. Check compatible boundaries, observation windows, units, omitted waiting, and whether arrivals and completions balance. The discrepancy alone does not locate a bottleneck or show that adding workers will help.

The law does not identify the cause of latency, prove that a queue is healthy, or promise that the system can sustain any chosen arrival rate. Before applying it, state the system boundary and check that arrivals and completions were approximately balanced during the observation window. A large mismatch is a reason to inspect the measurement setup or system stability, not permission to force the numbers to fit.

3.1.5 Follow the queue into development

A queue can contain events waiting for processing or changes waiting for review. Before calculating, name the work and measurement boundary in each case: generating a change and accepting it are different completion events. Try the question without opening the references, then use their assumptions to explain what carries across the two examples.

Pause & think

Follow work from events to review

A stable event pipeline completes 20 events/s with average end-to-end time 0.5 s. A stable development workflow accepts four changes/hour with average time from submission to acceptance of three hours. Both times include waiting inside their respective boundaries. Calculate the average work in progress implied by Little's Law for each. Then suppose submissions rise to six changes/hour while acceptance stays at four, with no discarded changes. Can the earlier steady-state calculation describe this growing backlog indefinitely?

▼ **Show worked answer**

The event calculation is 20 × 0.5 = 10 events. The development calculation is 4 × 3 = 12 changes. The same relationship applies to different work when boundaries, windows, and units agree. After the increase, unfinished changes accumulate at two/hour, so the earlier stable calculation cannot describe the backlog indefinitely. Check arrival/completion balance and waiting time again. Neither calculation identifies the cause of delay or proves the waiting time is acceptable.

Explain one difference between the event pipeline and human review that could make a capacity prediction unreliable. The numerical check establishes the stated relationship; your explanation should also identify its limits.

3.2 Measurements can lead you astray

Goodhart's law describes how a metric can stop tracking the desired outcome when it becomes a target (Goodhart, 1975). Software dashboards and quality gates make such proxies easy to optimize.

Consider a few examples:

- Test coverage can reveal unexercised code, but a coverage target can reward assertions that execute lines without checking meaningful behavior.
- A Lighthouse score can direct attention to performance problems, but optimizing only the score can remove valuable functionality or tune a page for the audit rather than its users.
- Mean latency can improve while tail latency or error rate becomes worse.
- Agent task pass rate can rise because tasks, judges, or outputs become tailored to the evaluation set rather than because the workflow handles new work better.
- Commit count, generated lines, and closed tickets can all increase while review and maintenance costs grow elsewhere.

Goodhart's law does not imply that targets or metrics are useless. It means that the measurement system changes once people or agents can adapt to it. A quality gate should therefore be treated as a partial model of the desired outcome, not as the outcome itself.

For an important target, record four things:

1. **Outcome:** What user or operational result is the metric intended to represent?
2. **Proxy:** Why should the chosen measure move with that outcome under the current conditions?
3. **Gaming path:** How could the number improve without the outcome improving?
4. **Counterevidence:** Which second signal, failure review, or qualitative check would expose that divergence?

Keep raw distributions and failed examples available instead of retaining only an aggregate score. Review the proxy when the workload, system, incentives, or development process changes. An effective metric is not permanently valid; it is a hypothesis that must survive contact with the behavior it influences.

3.2.1 Field note — Slideotter

Slideotter let me test whether a local language model could support presentation authoring. Using Qwen3.5-9B through the project's generation harness, I found that it could usually produce semantically sensible drafts: the slides contained information related to the requested topic and formed a recognizable presentation. That established adequacy, not excellence. Some slides remained weak or misunderstood my intent.

This conclusion came from my editorial review rather than a separate evaluation. The observed result belonged to the complete configuration—model, prompts, schemas, validators, and review workflow—so I could not confidently attribute poor outputs to the model rather than the harness. Structural and rendering checks could establish that a deck was valid; they could not establish that it was a good response to what I meant.

3.2.2 Goodhart pressure test

The challenge below turns that four-part review into two decisions. First choose a proxy and predict how an optimizer will respond when the number becomes a target. Then inspect the displaced cost and revise the gate so that improving the proxy cannot compensate for losing the intended outcome. The scenarios use deterministic illustrative values to make the feedback loop visible; they are not empirical comparisons between tools or teams.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/benchmarking/

Try at least two targets. Notice that monitoring the proxy is still useful after the repair, but it no longer decides success alone.

3.2.3 Resource proxies are not the outcome

Energy use is harder to measure than latency or transferred bytes, and neither of those proxies is an automatic substitute for it (Kalliola & Vepsäläinen, 2025).

EcoAssist illustrates the need to check the product alongside resource use (Barrocas et al., 2026). Its before-and-after benchmark reported reduced local CPU and GPU energy on selected standalone webpages using one M4 MacBook and Chrome testbed. In a separate lab study, some participants observed removed content or changed layouts after optimization. The measurements concern those pages and that device, exclude the optimizing model's full footprint, and are not a whole-system carbon estimate. An optimization must preserve the intended behavior as well as improve the selected resource measure.

Run one bounded optimization experiment:

1. Choose one representative Playwright journey and preserve its baseline.
2. Make one targeted AI-assisted frontend optimization.
3. Run the same journey under the same declared conditions before and after the change. If the chosen signal varies, repeat each condition at least five times.
4. Record direct energy only if the platform exposes a defensible measure. Otherwise label transferred bytes, JavaScript or asset size, CPU time, or long-task time as **resource proxies**, not energy or carbon.
5. For a deterministic measure such as built asset size, report one value per condition. For a run-varying measure, report the median and range.
6. Re-run the functional checks, inspect for visual or accessibility regressions, note rejected suggestions or regressions, and state what the comparison cannot establish.

3.3 Measurement alone cannot explain behavior

A benchmark can reveal that the request distribution changed, but it does not necessarily explain why one request was slow. Telemetry provides evidence about a running system through signals such as traces, metrics, and logs. Observability is the broader capability to use that evidence to investigate system behavior, including failures you did not predict in advance.

OpenTelemetry is a vendor-neutral framework for generating, processing, and exporting telemetry. It supplies APIs, SDKs, semantic conventions, and a Collector, but it is not itself a storage or visualization backend. That distinction matters: instrumenting an application creates evidence, while operating the pipeline and asking useful questions turns that evidence into an observability practice.

3.3.1 Simulated OpenTelemetry trace lab

The trace explorer below reuses the event-recording request from the benchmark demonstration. Switch between a healthy write, a slow write, and a successful request that required a retry. Select individual spans to inspect their duration, status, source, and attributes. A span represents one timed unit of work and carries its own span ID, while every span in the same trace shares a trace ID. The spans are not necessarily sent as separate network messages: an SDK can batch completed span records into one OTLP export request before a Collector processes and routes them.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/benchmarking/

The slow trace localizes most of the request time to connection waiting and event storage. That is a stronger lead than the end-to-end latency alone, but it still does not establish root cause. The next investigation might inspect database saturation, pool configuration, query behavior, or competing workloads. Also notice that useful domain spans and attributes require conscious instrumentation, while generic HTTP spans can often be produced automatically.

3.4 Observe the agentic workflow too

Observe the development workflow as well as the application. In both, outcome checks establish what worked and traces help explain the path to that result.

Table 3.1. Parallel evidence roles in a running web application and its agentic development workflow.

Evidence role	Web application	Agentic development workflow
Outcome to protect	Useful user and operational behavior	Accepted implementation and maintained project goal
Controlled comparison	Benchmark run under a declared workload	Bounded task or evaluation trial
Diagnostic evidence	Logs, metrics, request traces, and profiles	Prompts, tool calls, diffs, checks, and retries
Configuration	Revision, deployment, dependencies, and resource limits	Model, agent harness, tools, starting revision, and limits
Intervention	Operator diagnosis, mitigation, and recovery	Human review, correction, approval, and escalation

Keep the **task**, one attempted **trial**, its actual **outcome**, and the **trace** that led there distinct. Grade the outcome; use the trace to diagnose failures, account for cost and intervention, and check requirements such as permission compliance. Bind the result to the model and runtime where exposed, harness, tools, starting revision, resource limits, and routing or fallback.

Use the same reporting categories across A, B, and C: accepted outcome, checks, elapsed time, retries, cost or resource use, human correction, and known limits. Generated lines and transcript length can hide work shifted into review or maintenance; they do not replace these outcome measures. Appendix A develops grader design, repeated trials, uncertainty, and configuration qualification.

> **From a coding task to software delivery**
>
> A faster patch is only one part of getting a useful change into production. DORA's five delivery metrics cover change lead time, deployment frequency, failed deployment recovery time, change fail rate, and deployment rework rate (Harvey, 2026). Interpret them together for one application or service over a declared period. They describe delivery throughput and instability, while the task record above describes one attempt at producing a change.
>
> Consider an invented timeline: work starts at 09, a patch is committed at 09, and it reaches production at 15. The twenty-minute interval ends at a commit; DORA change lead time runs from that commit to deployment, five hours and forty minutes later. If the deployment fails and service recovers at 15, record the thirty-minute recovery separately. Failed and incident-driven deployments also belong in the period's failure and rework counts, with their denominators stated.
>
> A short project may have too few deployments or failures for stable rates. Use this timeline to interpret the measurement boundaries; individual rankings and an uncontrolled before/after chart cannot establish an AI effect.

3.4.1 Field note — Preserve accepted context, not every interaction

I have not established a practice of preserving complete agent transcripts or detailed workflow telemetry. When returning to a project, I rely mainly on code, documentation, and commit history to recover the implemented state and accepted intent.

Architecture Decision Records are especially useful. Vibe Template's ADR workflow keeps proposed, accepted, implemented, and superseded decisions distinct. I let agents draft records, but negotiate important choices carefully; implementation can then expose what an unresolved decision still needs to settle.

Those records preserve the rationale I accepted, not every prompt, disagreement, retry, or abandoned path. They cannot reconstruct the complete agent workflow. For the course comparison, preserve the selected task's trace and correction record while the work happens, rather than trying to infer them later from the accepted repository.

Pause & think

Choose a valid comparison

You want to test whether batching improves useful write throughput while keeping p95 end-to-end latency below 100 ms and errors at or below 0.1%. Select the comparison that isolates batching. Then evaluate a run with higher throughput, p95 = 90 ms, and errors = 0.8% against those declared criteria.

▼ **Show worked answer**

Keep the workload, dataset, environment, and acceptance criteria comparable while changing batching. A run with p95 = 90 ms passes the latency bound, but 0.8% errors violates the 0.1% error bound. Higher throughput does not compensate for failing a declared acceptance criterion.

3.5 Project artifact

Add implementation A's measurement baseline to the Establish evidence bundle:

- the representative runtime workload, load, conditions, prediction, thresholds, and results, including distributions for varying measurements;
- one explanatory telemetry signal and inspected failure, with the first warning that behavior is leaving the measured conditions;
- one representative AI-assisted task and its configuration, trace, checks, retries, correction, cost or resource use, and acceptance decision.

Tie the records to the application revision and retain the reporting frame for B and C. State what each comparison leaves untested.

3.6 Summary

A benchmark tests a claim under declared conditions; telemetry helps investigate the behavior it reveals. Preserve variation and check that an improved proxy still represents the desired outcome. For agent-assisted work, evaluate the accepted artifact and use configuration, attempts, and interventions to explain how it arose.

3.7 References

1. Kalliola, J., & Vepsäläinen, J. (2025). Challenges Related to Approximating the Energy Consumption of a Website. IEEE Access, 13, 139001–139017. https://doi.org/10.1109/ACCESS.2025.3596459
2. Barrocas, A., Jardim Nunes, N., Nisi, V., & Martelaro, N. (2026). EcoAssist: Embedding Sustainability into AI-Assisted Frontend Development. Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems, 1–16. https://doi.org/10.1145/3772318.3791330
3. Little, J. D. C. (1961). A Proof for the Queuing Formula: L = λW. Operations Research, 9(3), 383–387. https://doi.org/10.1287/opre.9.3.383

4. Goodhart, C. A. E. (1975). Problems of Monetary Management: The U.K. Experience. In Papers in Monetary Economics (Vol. 1). Reserve Bank of Australia. https://www.econbiz.de/Record/problems-of-monetary-management-the-u-k-experience-goodhart-charles/10002525062
5. Harvey, N. (2026). DORA's Software Delivery Performance Metrics. DORA Guides. https://dora.dev/guides/dora-metrics/

PART II

Challenge — Expose architectural assumptions

Challenge examines the architecture behind implementation A, the cost of changing it, and the assumptions that fail under production pressure. Build implementation B afresh with a comparison-model single agent and compare its behavior and development effort with the baseline.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

II.1 From hidden assumptions to comparable evidence

The three chapters form one progression:

1. How does web architecture shape scalability? separates responsibilities from locations, traces how web placement choices accumulated, and turns one implementation A decision into an architecture hypothesis.
2. How does development friction limit web projects? follows one browser-visible change across setup, code, contracts, validation, review, and deployment to expose where the project resists change.
3. Which web application assumptions fail under pressure? connects documented production failures, application signals, degradation, and recovery to assumptions that local correctness left untested.

II.2 Implementation B — Work Activity Tracker

Use the Challenge specification and shared transfer policy. Keep the common behavior, pressures, and reporting categories stable while testing a revised architecture hypothesis. Record what you learned from A's public discussion and distinguish that practice effect from model or workflow differences where the evidence permits.

II.3 Technical debt reduces optionality

Technical debt names earlier choices that make later change harder. A shortcut can be rational while a team is learning or time is limited; the problem begins when its future cost stays invisible, unpriced, or allowed to compound. A system with high debt may still run, pass tests, and satisfy users today while giving the team fewer safe choices tomorrow.

Architecture makes those commitments visible, friction reveals their daily cost, and production pressure shows which ones restrict recovery or growth. Challenge therefore treats debt as one supporting lens for comparing the two implementations rather than as a synonym for defects or for every simple design.

II.3.1 Field note — Codify the development system

Earlier in my agentic-development work, agents produced changes faster than I could evaluate them. I have since codified recurring lessons in Vibe Template: initialization, architecture rules, specifications, decision records, instructions, and layered validation. The Kirjolab experience added earlier structural feedback when functional progress began hiding architectural confusion.

Build your own equivalent by turning repeated mistakes and useful practices into reviewable guidance and commands. Revise those controls as the projects, models, and your judgment change.

> **Sustainability angle**
>
> Inefficient design can become sustainability debt. Unbounded logs, poor retention policies, N+1 queries, heavy dependencies, and wasteful background jobs may not break the system immediately, but they compound infrastructure cost and resource use over time.

CHAPTER 4

How does web architecture shape scalability?

Created
January 16, 2026
Updated
October 1, 2026

Web architecture connects consumers, representations, computation, and authoritative data across networks. Servers, cloud services, programmable edges, and browsers offer different places to put that work, with different latency, trust, cost, and failure consequences. Ask which pressure justifies moving a responsibility and which contract must remain stable when it moves.

Concept connections
Explore definitions and connections in the web edition.
Explore this reading's concepts

4.1 A conceptual model: separate roles from locations

A framework-shaped label such as “serverless application” or “single-page application” combines several decisions that can be examined separately. Begin with four architectural roles:

Table 4.1. Four roles that remain useful as web deployment locations change.

Role	Question it answers	Typical examples
Consumer	Who or what expresses intent and observes the result?	A person using a browser, an integration, a crawler, or an agent
Representation	What crosses a boundary to make state or action visible?	HTML, JSON, an image, a stream, a form, or a hypermedia control
Execution	Where is behavior evaluated?	A build process, browser, physical server, cloud service, or edge node
Authoritative state	Which component decides and preserves the current truth?	An application service, database, event log, or external system

Roles are not machines. One machine may perform several roles, and one role may be distributed across many machines. An **origin server** serves the resource in an HTTP exchange, rather than forwarding the request as an intermediary; the role need not be one physical machine. In web security and HTTP URI terminology, an **origin** is the normalized scheme, host, and port tuple, such as `https`, `example.org`, and `443` (RFC 9110, section 4.3.1). CDN discussions also use “origin” as shorthand for the upstream origin server or backend; this placement alone does not identify the authoritative data store. Likewise, an edge function may execute application logic without becoming the authority for the data it reads.

Use the event tracker to keep the model concrete. A `GET /events` request asks for a representation and may be answered from a browser cache, an edge cache, or the origin. A `POST /events` request expresses an action whose validation may begin near the consumer, but whose accepted result must eventually reach the component that owns the event history. The two paths can share an interface while placing execution, copies, and authority differently.

4.1.1 Request boundary lab

The demonstration below traces one `POST /events` request through the browser, network, API, storage, and response path. Each boundary has an owner, a failure mode, and a contract that future clients, humans, and agents may depend on.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/architecture/

Pause & think

Separate response from outcome

An API commits an event to storage, then the network loses its response. The browser times out. Use only these stated facts to classify the outcome.

<details>
<summary>▼ Show worked answer</summary>

The storage commit happened even though the browser received no confirmation. A timeout describes what the caller observed; it does not undo or disprove the commit.
</details>

Use this trace before drawing the architecture diagram. If the request path is unclear, the diagram is likely to hide the most important architectural decisions.

4.1.2 Automated consumers change demand and exposure

A crawler, integration, monitoring probe, agent, and abusive bot may send similar requests while creating different value, cost, and risk. Keep the operation's authoritative meaning stable across consumers, then decide which reads are public, which actions require authorization, and which expensive paths need quotas or admission control. For the event tracker, a public summary might be cached while event writes require an authorized actor and a bounded request rate. Cooperative crawling rules such as `robots.txt` do not enforce access control. Appendix C develops crawler policy, rate-limit identity, and an endpoint review.

> **Sustainability angle**
>
> Placement determines resource flow. Chatty APIs, unnecessary round trips, duplicated computation, oversized payloads, heavy client bundles, aggressive polling, and poorly targeted edge execution all turn architectural choices into recurring resource use. Moving work is valuable when it removes a

> justified constraint rather than merely multiplying infrastructure.

4.2 Choose where work runs

An origin-centered application can begin with one server handling requests and storing data. Separating presentation, application logic, and data into three tiers separates responsibilities; it does not require three machines. Cloud provisioning and programmable edges expand the placement options without removing the origin's authority or the need to recover from failure.

Table 4.2. Placement choices, the pressures they address, and the work they retain.

Placement choice	Pressure it can address	Obligation to examine
Owned or hosted server	Predictable workload, hardware control, or data locality	Capacity planning, replacement, and recovery
Replaceable cloud instances	Variable demand and automated provisioning	Startup delay, quotas, shared state, and downstream saturation
Managed service	Limited team capacity for operating a database, queue, or storage layer	Provider limits, consistency, cost, export, and recovery contracts
CDN and programmable edge	Repeated delivery, distant consumers, or early request rejection	Cache correctness, execution limits, and distance to authoritative writes

These choices can coexist in one application. The NIST cloud definition emphasizes pooled resources, rapid provisioning, elasticity, and measured service; these change how capacity is acquired, not how quickly a database can accept writes. A session, upload, or job kept only on one replaceable instance can disappear or become unreachable from the next request. Show where that state survives and which dependency saturates first.

A CDN moves reusable representations close to consumers; a programmable edge also runs bounded request handling there. The event tracker can deliver public assets and a cached summary near users while keeping event writes beside the authoritative database. If an edge function must contact that database on every request, the long trip remains. Choose placement from reuse, locality, trust, and operational capacity, and show managed dependencies and provider boundaries in the diagram.

4.2.1 HTTP provides a stable seam

HTTP semantics define a stateless request-response protocol in which clients act on resources through messages and receive status information and representations in return. The protocol intentionally hides how a service is implemented behind a uniform interface. It also allows intermediaries—proxies, gateways, and tunnels—to participate between the user agent and origin. Those properties let deployment topology change without requiring every consumer to understand the new arrangement.

A request method communicates intent, the target identifies what the request concerns, headers carry metadata and control information, and the response status describes the result. Correct use of safe and idempotent methods, validators, cache metadata, authentication, and content negotiation affects what browsers and intermediaries are allowed to do. HTTP is therefore architectural material rather than plumbing supplied by a framework.

HTTP gives each request understandable semantics independently of its connection. Application identity, continuity, and durable state still need explicit owners; the data section develops those obligations.

4.2.2 Application contracts outlive deployment choices

HTTP can carry several application interaction styles, including resource-oriented messages, hypermedia controls, GraphQL operations, and RPC calls. Each style distributes knowledge and change pressure differently between consumers and providers, but none removes the need to define representations, errors, retries, authorization, and compatibility.

For the project, choose one minimal event-creation and event-query contract, then ask how it changes when requests are retried, devices go offline, workflow states evolve, or consumers need additional data. Appendix C compares the interaction styles and contains the API contract lens and executable HTTP example.

4.2.3 Organizational boundaries shape application contracts

An application contract is also a communication contract between its owners. In his original 1968 paper, Melvin Conway observed that organizations tend to produce system structures shaped by their communication structures. For web architecture, this means that a shared API or data model can become the place where several teams must coordinate, while a nominally independent service can remain tightly coupled when every change still requires the same people to agree.

Do not create a service boundary merely to imitate an organizational chart. Instead, identify a capability with coherent authority and change reasons, give its owners enough control to evolve its internals, and keep the consumer contract small enough that other teams can depend on it without joining every implementation decision. When one workflow crosses several owners, record who may change the contract, how compatibility is checked, how consumers learn about deprecation or migration, and which decision requires cross-team agreement. The ownership card later in this chapter should therefore name both the code boundary and the communication boundary that maintains it.

4.3 Rendering moves work across time and place

Cloud and edge platforms expanded where code can execute, while build tooling and capable browsers expanded when a representation can be created. Start with **who constructs the next view and how it reaches the browser**:

- A conventional multi-page application (MPA) follows links and submits forms to receive complete HTML documents.
- A hypermedia-driven application (HDA) keeps views and available actions in server-provided HTML; browser code follows those controls and can apply HTML fragments to parts of the page (Gross, 2022).
- A typical single-page application (SPA) fetches data and uses client-side rendering (CSR) to construct subsequent views without replacing the document.

These are representative flows, not mutually exclusive categories: a conventional MPA also uses hypermedia, and partial page updates alone do not distinguish HDA from SPA. The distinction in the latter two examples is whether server HTML or a client application model defines the views and actions. An HDA can still use scripts for local interface behavior (Gross, 2022).

Then ask **how the initial view is produced**. Server-side rendering (SSR) produces HTML for a request; static site generation (SSG) produces it ahead of requests; incremental static regeneration (ISR) refreshes previously generated HTML; CSR builds the view in the browser. CSR is the natural starting point for the SPA example, but an SPA can also initialize from server-rendered HTML and take over later rendering, as React's hydration API illustrates (React, n.d.). Caching server-generated HTML can serve an MPA or HDA without moving control of its views to the browser.

Compare the consequences for freshness, server work, navigation, client responsibility, caching, accessibility, and failure.

4.3.1 Web architecture lens

Use the same event-recording interaction across the three flows. First choose how the application presents the state after `POST /events`, then vary how its first view arrives. The MPA and HDA examples start with server HTML; the SPA example starts with CSR and also supports server-rendered or cached initial HTML. Run the trace and watch which artifact crosses the network.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/architecture/

The ISR behavior in the lab is a simplified stale-while-revalidate model. Exact regeneration, invalidation, and consistency behavior depend on the framework and deployment platform. That platform-specific contract should be documented as part of the architecture.

4.3.2 Activation is a separate architectural decision

Rendering and navigation do not explain how delivered HTML becomes interactive. Hydration, progressive hydration, islands, and resumability move different amounts of code, state, and startup work across the server-browser boundary. Record which regions need behavior, when that behavior becomes available, and which state or communication constraints the approach introduces. Appendix C compares the activation approaches and provides a focused lab.

4.3.3 Runtime generation adds a control boundary

Runtime-generative UI lets model output choose data, composition, or executable behavior while the application runs. A descriptor needs a validated vocabulary of components and actions; executable output additionally needs isolation and tightly scoped access to host data and tools. Both need authorization at action time and a safe fallback. Appendix C develops these controls and the descriptor audit.

4.4 Data limits how freely responsibility can move

Compute and representations can often be duplicated or replaced more easily than authoritative data. Identify the **authoritative state** for each domain outcome and the boundary that owns it. The book also uses state authority for this ownership question; it is separate from a caller's authorization to request a change. A read may be served from a replica or cache, but concurrent writes still need a rule for which result becomes true. Moving execution closer to users without deciding where authority lives can exchange network latency for inconsistent state, complex conflict resolution, or a longer path to the real owner.

For the event tracker, raw events, current work-item state, and derived summaries have different roles. Raw events may support replay or audit, current state must preserve workflow invariants, and summaries can often be rebuilt or temporarily stale. The architecture should state which data is authoritative, which is derived, where each is placed, and what happens when a location cannot reach its authority.

A derived summary may be rebuilt from retained events only when that history contains the necessary facts. Recomputing it must not repeat external effects such as notifications, and new events must not be lost during the rebuild. Appendix C works through a summary rebuild and cutover.

4.4.1 Data placement follows authority and access

An origin-centered application can keep its primary database close to the application server and pay one predictable network cost for most operations. Cloud services make storage independently provisioned and managed, and they make replicas, object stores, queues, and analytical systems easier to combine. Edge execution makes the remaining distance more visible: code may run near a consumer while the authoritative write still has to cross a region or continent.

Choose a data shape from the behavior that must remain correct and the access path that must remain efficient. A relational model is useful when constraints and relationships must be enforced together; a document can keep an aggregate together; a key-value store can serve known-key access; object storage can hold large immutable representations; and an append-oriented log can preserve history for replay or derivation. These are composable roles rather than stages of progress. One application may use a relational source of truth, object storage for attachments, and a derived search index without giving all three equal authority.

Distribution makes four distinctions especially important: authoritative versus derived data, reads versus writes, transactional versus analytical work, and current state versus retained history. Replicating a read path may reduce latency without changing write authority. Partitioning can increase capacity or isolation, but then the partition key determines which queries and transactions remain local.

4.4.2 Stateless requests still depend on state

An interchangeable server may receive a workspace handle without sharing an earlier request's in-process session. It must still authenticate the caller, authorize access, and resolve authoritative state. A handle does not grant permission, and state available only on its originating instance still requires

affinity. Retries after a lost response need a stable application command identity, with the mutation and reusable result stored atomically. Appendix C provides the two-instance state and retry audit.

4.4.3 Transactions preserve invariants across failure and concurrency

A transaction groups operations that must commit together and defines how concurrent work may interact. For the event tracker, a successful lifecycle transition must update current state and append its activity event without allowing either write to escape alone. The deployed database's constraints, isolation level, acknowledgment point, and failure model determine the actual guarantee.

Appendix C develops ACID, concurrent transitions, and the transaction lab.

4.4.4 Model valid data combinations

Constructive data modeling uses tagged unions, sealed hierarchies, or guarded constructors to require the data belonging to each outcome. It reduces contradictory in-memory values; runtime parsing, authorization, and atomic persistence still enforce their own rules. Appendix C develops a transition-result example and selected-invariant exercise.

4.5 Caching coordinates controlled copies

Caching began as a central content-delivery technique, but its architectural reach is wider than the CDN. A browser can keep a private response, a shared intermediary can reuse an HTTP representation, an application can retain a computed result, and a data system can expose a replica or materialized view. Each case creates a controlled copy whose identity, lifetime, and relationship to authority must be defined.

Table 4.3. Caching layers as controlled copies of authoritative or derived state.

Cache location	Copy being reused	Main benefit	Main correctness question
Browser	A response for one user agent	Avoids transfer and repeated work	May this user reuse it, and when must it validate?
CDN or reverse proxy	A response shared across suitable requests	Reduces latency and origin work	Which request properties belong in the cache key?
Application	A query, computation, or assembled view	Avoids repeated service or data work	What invalidates it, and can callers accept bounded staleness?
Data layer	A read replica, index, or materialized result	Moves or accelerates read work	How far may it lag behind the authoritative write path?

4.5.1 HTTP cache control is application policy

An HTTP cache can avoid an origin request, reduce transferred bytes, and hide network latency only when reuse is correct for the response and its consumers. The response must communicate who may store it, how long it remains fresh, how it can be validated, and whether stale reuse is acceptable.

The HTTP caching specification defines the core directives and freshness model:

- `public` permits storage by shared caches, while `private` limits a response to private caches.

- `max-age` defines freshness relative to response generation; `s-maxage` can define a different freshness lifetime for shared caches.
- `no-cache` still permits storage, but the stored response has to be validated before reuse.
- `no-store` asks caches not to store the response at all.
- `must-revalidate` prevents reuse of a stale response without successful validation.
- `Vary` makes selected request headers part of response selection. This can protect correctness, but every varying dimension can fragment the cache.

The cache key must distinguish requests whose responses or required processing differ. Query parameters can fragment reuse even when they only describe a campaign, but ignoring an authorization or content-selection parameter can make reuse incorrect. Appendix C develops the optional `No-Vary-Search` mechanism and its limits.

Validators avoid retransmitting an unchanged body. An origin can attach an `ETag`; once the stored response needs validation, a cache sends that value through `If-None-Match`. A matching validator allows the origin to return `304 Not Modified` without the representation body, while a changed validator produces a new `200` response.

The `stale-while-revalidate` extension moves latency again: a cache may immediately serve a stale response for a bounded period while validating it in the background. That can be appropriate for a public activity summary, but not for every response. Personalization, authorization, invalidation, and the cache key are correctness and security decisions rather than deployment details.

4.5.2 Cache decision lab

The lab below follows a second request through browser cache, shared cache, and origin. Change the response profile, move time forward, and decide whether the origin representation changed. Then inspect whether the request is served fresh, served stale while validation happens, conditionally revalidated, or fetched in full.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/architecture/

Cache behavior should be tested at the deployed boundary because application code is only one participant. A framework, reverse proxy, CDN, browser, or hosting platform may add, remove, or reinterpret cache metadata. Record the final response headers, cache status, `Age`, and origin traffic rather than assuming the source-code declaration is the complete policy.

4.6 Architecture is maintained as responsibility moves

Throwaway code, piecemeal growth, and pressure to keep a system working can gradually blur its structure (Foote & Yoder, 1997). Adding services, edge functions, browser state, or caches creates more responsibilities to keep coherent. Record their owners, revisit temporary shortcuts, and reorganize code

when several unrelated change reasons accumulate together. Architecture notes and a decision log preserve those choices for later contributors.

4.6.1 Field note — When syntax familiarity stopped deciding the stack

Agents let me explore technologies before I know every syntax detail or API. I am using that capability in an Apache Jena proof of concept for validating a knowledge base. I am not a strong Java developer, and I do not yet know whether this is the right architecture; the immediate question is whether the approach produces useful results.

I can delegate unfamiliar syntax while examining the data model and observed behavior. If the prototype proves useful, its architecture, maintainability, and my ability to own it still need review.

4.6.2 Make the implementation architecture reviewable

By this point, you have already made architecture choices in implementation A, even if you did not yet have a complete vocabulary for them. The public project discussion should make those choices visible: different groups received the same behavioral specification but chose different stacks, boundaries, storage models, and request paths.

Keep the architecture diagram consistent with the project harness: global rules, feature contracts, decision records, and agent instructions should describe the same implementation.

A component diagram can show that two boxes exist without explaining which one owns a decision. For each independently evolvable capability, record its source root, composition root, authoritative state and its owner, public contracts, allowed dependency direction, and intentionally excluded responsibilities. The resulting ownership card is a review aid rather than a demand for one directory per feature.

File size, fan-in, churn concentration, or a crowded directory can act as structural smoke alarms, but none proves that a split is correct. When such a signal fires, inspect whether several authorities or change reasons have accumulated together. Consolidate ownership, split responsibilities, or document a justified exception; do not divide a file merely to satisfy a number.

4.7 Architectural dimensions for a web application

Placement, rendering, data, and ownership choices interact along several dimensions. Use the table to compare architectures without treating any option as a universal destination.

Table 4.4. Architectural dimensions to examine when designing or comparing web applications.

Dimension	Available choices	Pressure the choices can address	Obligation introduced by the decision
Infrastructure ownership	Owned server, hosted machine, VM, container, function, managed service	Provisioning speed, utilization, operational capacity	Provider contracts, quotas, cost visibility, and recovery
Execution placement	Build system, browser, origin, cloud region, edge location	Latency, elasticity, device capability, or data locality	More boundaries, deployment targets, and partial-failure paths
Representation rendering	Build time, request time, incremental regeneration, client time	Freshness, origin work, delivery speed, personalization	Invalidation, duplicated logic, activation, and fallback behavior
Browser interaction	Document navigation, hypermedia updates, client-managed navigation	Interaction richness, continuity, client responsiveness	Client state, compatibility, accessibility, and recovery

Dimension	Available choices	Pressure the choices can address	Obligation introduced by the decision
Client activation	Hydration, progressive hydration, islands, resumability	Initial payload and time to useful interaction	Serialization, loading policy, tooling, and cross-boundary state
Authoritative state owner	Client-local state, application service, database, event log, external owner	Correctness, offline work, audit, and integration	Invariants, authorization, retries, and lifecycle
Data distribution	Central store, replicas, partitions, derived views	Volume, locality, isolation, and read or write pressure	Consistency rules, routing, migration, and reconciliation
Caching and freshness	Browser, intermediary, application, and data-layer copies	Repeated work, transfer, latency, and origin capacity	Cache identity, freshness, invalidation, privacy, and observability
Consumer contract	HTML and forms, resource HTTP, hypermedia, GraphQL, RPC, events	Independent evolution and different consumer needs	Compatibility, idempotency, rate limits, and bounded authority
Operational ownership	One team and machine through multiple teams, providers, and regions	Organizational scale and specialized capability	Explicit ownership, signals, incident paths, and decision records

The rows interact. Moving rendering to the edge may require a cache-freshness policy; moving interaction to the browser may require a new API contract; moving compute into replaceable cloud instances may require state to leave process memory; and moving reads near users may lengthen the path to authoritative writes. Check that the consequences of those choices agree.

4.8 Project artifact

Review the architecture and data-flow diagrams submitted with implementation A, identify one consequential architectural choice, and write an architecture hypothesis for implementation B. The hypothesis should use the conceptual model and dimensions table to state:

- which consumer, representation, execution location, and authority the selected workflow uses,
- which pressure justifies preserving or changing that placement,
- what new contract, consistency rule, or operational obligation follows,
- which automated consumers may reach the selected path and where cooperation, access control, quotas, or rate limits belong,
- which team or organizational boundary owns the contract and which changes require coordination across it,
- and what evidence would show whether the decision works.

Also complete an ownership card for the event workflow and identify one structural signal that would prompt another ownership review. Implementation B should still include its own diagrams and decision record.

4.9 Summary

Architecture assigns execution, representations, and data authority to locations that fit the pressure. HTTP and application contracts let consumers survive many placement changes, while state and caching introduce coordination obligations. Keep those obligations and their owners visible so the next change can be evaluated against the same model.

4.10 References

1. Foote, B., & Yoder, J. W. (1997). Big Ball of Mud. Proceedings of the Fourth Conference on Pattern Languages of Programs. https://www.laputan.org/mud/
2. Gross, C. (2022, February). Hypermedia-Driven Applications. htmx. https://htmx.org/essays/hypermedia-driven-applications/
3. Gross, C. (2022, November). Hypermedia-Friendly Scripting. htmx. https://htmx.org/essays/hypermedia-friendly-scripting/
4. React. (n.d.). hydrateRoot. React documentation. Retrieved https://react.dev/reference/react-dom/client/hydrateRoot

CHAPTER 5

How does development friction limit web projects?

Created
January 16, 2026
Updated
October 1, 2026

A web application does not have to serve more users before it encounters a scaling problem. It can first become difficult to change. A feature that sounds local may cross a browser form, an HTTP contract, server-side validation, persistent data, a query, a cached representation, and a deployment path. Each boundary adds knowledge, evidence, and coordination that the change must preserve.

Development friction is the resistance a team feels when trying to set up, understand, change, test, review, deploy, or operate that system. It is the everyday experience of technical debt, hidden coupling, and weak feedback. A system with high friction may still be useful, but each change requires more effort than its behavior alone would suggest.

Friction accumulates as a change crosses interfaces, teams, and commitments that are costly to reverse. Some of that friction protects users, data, and collaborators; some merely repeats work. The goal is not to remove every constraint, but to move useful evidence earlier and remove resistance that provides no corresponding control.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

5.1 Trace one web change from the browser to production

Consider one change to the event tracker: add an optional event category and let users filter the activity view by category. The visible request is small, but its implementation can follow this path:

1. **Browser form:** add a control, preserve accessible labels and error states, and decide what an omitted category means.
2. **HTTP contract:** extend the event representation without breaking an older client or independently implemented consumer.
3. **Validation:** apply the same allowed values and defaulting rules at the untrusted-input boundary.
4. **Storage:** persist the category, account for existing events, and decide whether a migration or backfill is required.
5. **Query and view:** filter without changing authorization, pagination, ordering, or the empty state.
6. **Cache:** include the filter in the cache identity and invalidate or tolerate previously derived representations.

7. **Deployment:** run compatible versions together, observe the new path, and preserve a safe way to stop or reverse the change.

This path is not a prescription for seven separate components. It is a way to find where one behavior is represented and who owns it. In a small application, several stages may live in one module and deploy together. They remain distinct responsibilities because a mistake at each boundary produces different evidence and a different user-visible failure.

Friction becomes visible when the path is hard to trace, the same rule has several unsynchronized definitions, a local check cannot exercise the changed boundary, or a deployment makes earlier assumptions costly to recover. The path also gives the chapter a stable unit of analysis: ask what the category change costs in implementation A, then repeat the same audit for implementation B.

> **Sustainability angle**
>
> Friction creates repeated work. Slow setup, flaky tests, confusing documentation, and unclear code make humans and agents rerun commands, regenerate changes, and spend more compute to reach the same result. Developer experience therefore has an indirect resource cost.

5.2 Recognize where friction appears

The category change exposes several related forms of resistance:

- **Setup friction** appears when a fresh checkout cannot reach a running application and baseline checks without undocumented local state.
- **Comprehension friction** appears when a developer cannot find where the category contract, query, or cache policy is owned.
- **Change friction** appears when one rule must be edited in unrelated places or when hidden consumers fail after the contract changes.
- **Testing friction** appears when checks are slow, flaky, or unable to distinguish a browser problem from an API, storage, or cache problem.
- **Review friction** appears when generated or human-written diffs mix the category change with unrelated rewrites and configuration churn.
- **Deployment and operational friction** appears when compatibility, rollout, signals, or recovery depend on knowledge that is not represented in the repository or platform.
- **Coordination friction** appears when ownership is unclear and teams or agents duplicate work, wait on one another, or make incompatible assumptions.

These categories interact. For example, unclear ownership increases comprehension time, produces wider changes, and sends more uncertainty into review. A slow or untrusted test suite encourages larger batches, making a failure harder to localize. An undocumented environment turns every collaborator into a separate integration experiment.

Not all of this resistance is waste. Authentication, validation, compatibility checks, migration stages, review, and release gates can be **protective friction** when they control a named risk and return useful evidence. A flaky test, repeated manual setup, or approval that merely duplicates an automated check is **accidental friction** when it delays work without increasing confidence or control. Before removing a step, ask which failure it is meant to prevent and what evidence would be lost.

5.3 What documented web-development cases reveal

The accounts below were published by the organizations that performed the work. They are useful evidence about particular systems, not controlled comparisons or transferable performance promises. Follow the links for the architecture and migration details, then use the transfer question to examine the event tracker.

Table 5.1. Documented cases interpreted as forms of web-development friction.

Documented case	Friction exposed	Intervention and reported result	Transfer question for the event tracker
Spotify Golden Paths, Backstage, and Tingle	Creating a web project required manual coordination across a repository, cloud project, CI/CD, documentation, and other engineers	Spotify automated a common path for websites and backend services and reports reducing setup to a first build and deployment from about 14 days to under five minutes	Which repeated setup and deployment decisions can become a maintained project path?
GitHub.com development in Codespaces	Local environments and branch changes became brittle as dependencies, services, and schemas changed	GitHub used declared environments, cached dependencies, and prebuilds, reporting that GitHub.com environment creation fell from 45 minutes to 10 seconds	Which environment assumptions should be versioned, rebuilt, or replaced instead of repaired by hand?
Shopify's modular Rails monolith	Hundreds of developers needed understandable ownership and dependency boundaries without stopping feature work	Shopify replaced call-graph analysis taking over an hour with Packwerk static analysis taking a few minutes, allowing dependency violations to become pull-request feedback	Which dependency or ownership rule would make the category change easier to locate and review?
Stripe's online subscription migration	A data-model change had to preserve availability and accuracy across hundreds of millions of live objects	Stripe staged the change through dual writes, moving read paths, moving write paths, and removing the old model while the service remained operational	At which step does the data change become a coordinated migration rather than an ordinary code revert?
Slack's CI/CD circuit breakers	Test traffic, queues, retries, and dependency overload caused misleading failures and cascading disruption	Slack deferred and shed test work under pressure, reporting no cascading internal-tooling incidents during the following two years while its time-to-test-results metric held steady	When would delaying or selecting a check return more trustworthy feedback than running everything at once?

The migration case exposes a question that setup speed alone cannot answer: when does a change stop being safely reversible through code alone?

5.4 Find the commitment point

Fast generation can make a change cheap to introduce without making it cheap to reverse. In a repository study of ten open-source database applications, schema revisions often co-evolved with dependent application code, illustrating how a local data-model choice can acquire a wider change footprint (Qiu et al., 2013).

For the category change, the **commitment point** is the first moment when an ordinary code revert no longer restores the prior state. This may occur when persistent production data adopts the new representation, an independently deployed consumer relies on the extended contract, a queued job carries the new value, or a cached artifact can outlive the deployment. Beyond that point, reversal becomes a coordinated change involving compatibility, migration, invalidation, reconciliation, or communication.

A commitment is not automatically technical debt. The audit asks whether its future coordination cost was visible and justified. Before the point, deleting a candidate implementation may be enough. Near the point, expand-and-contract changes, mixed-version tests, staged rollout, and observation add protective friction because they preserve options. After the point, "roll back" must name what happens to accepted data and dependent consumers rather than only which code artifact is deployed.

The Reality chapter returns to compatibility, migration, rollback, and fix-forward under production pressure. Here, make the asymmetry between introduction and safe reversal visible.

5.4.1 Plan soft and hard decisions

Use **decision reversibility** when planning a change. Soft and hard are the book's shorthand for low and high reversal cost, related to the one-way/two-way-door distinction. A **soft decision** is still cheap to revise or undo because few durable effects or external dependencies rely on it. A **hard decision** requires more coordination or recovery to change safely because accepted data, consumers, or operating commitments depend on it. Soft and hard describe a spectrum, and the same decision can become harder as those dependencies accumulate.

Trying a category representation in an isolated branch can remain a soft decision. Once production events store that representation and independently deployed clients rely on it, changing the same choice may require compatibility and migration. The amount of code changed does not tell you how much freedom remains. A hard decision can be necessary and justified; its reversal cost alone does not make it technical debt.

Keep uncertain choices soft while a bounded experiment can still answer the question. Before crossing the commitment point, record what makes the choice hard, the evidence that justifies committing, and how you would revise it safely. Ask: what can we change locally today, who or what will depend on it tomorrow, and what evidence would show that reversal succeeded?

5.4.2 Keep a decision log

Preserve that reasoning in a **decision log** so a future contributor can understand why a choice was made and when to reconsider it. An **Architecture Decision Record (ADR)** is one format for a consequential architecture choice. Michael Nygard's ADR approach records context, decision, status, and consequences, retaining superseded records with a link to their replacement.

For this course, also record the alternatives considered, supporting evidence, owner, current reversibility, and a condition for revisiting the choice. Acceptance and implementation status answer different questions from reversibility: an accepted decision can remain soft, and implementing it may create dependencies that make it hard.

Record field	Illustrative category decision
Context and choice	Trial an optional category field to support filtering while preserving existing events.
Alternatives and rationale	Defer the field or introduce it now; use the bounded trial to test the change path before committing.
Status and owner	Accepted for an isolated trial; the team must review the evidence before production adoption.
Evidence and consequences	Link tests for uncategorized events and older clients; note the compatibility obligation introduced by storing the field.
Reversibility	Soft in the isolated branch; harder once production data and independent clients depend on the representation.
Revisit condition	Review before production adoption, or when a failed compatibility check challenges the choice; name the migration needed for reversal.

Use the project's existing decision record or log rather than creating a separate reporting bundle. When evidence changes the choice, record the new rationale and link the earlier decision so the history of the commitment remains visible.

5.5 Reduce friction without hiding the boundary

5.5.1 Treat the environment as a versioned input

Declare a path from a fresh checkout to a running application and its baseline checks, including toolchain versions, services, configuration names, supported platforms, and resource limits. Inject secrets at runtime and name external dependencies that the declaration cannot reconstruct.

Use the cross-team audit to record setup time, failures, retries, and undocumented intervention. Replay repairs from fresh state. Appendix B develops the audit and distinguishes declared setup, isolation, and bit-for-bit reproducibility.

5.5.2 Make ownership and contracts traceable

The category path should have one understandable owner for each rule even if several layers enforce it. The browser may give immediate feedback, but the server-side input boundary owns acceptance. The database may constrain stored values, but the application contract owns what consumers are promised. The cache may copy a representation, but it does not own the authoritative event.

Record those responsibilities in the architecture, specification, schema, types, and tests that future developers and agents are likely to inspect. Keep duplicate enforcement mechanically connected where possible—for example, generate types from a schema or run the same contract examples against independent implementations. The aim is not to place every rule in one file; it is to prevent several files from silently becoming competing authorities.

5.5.3 Agents can move friction rather than remove it

Agent assistance can change which technology choices appear feasible. In a practitioner essay, Ronacher (2026) argues that agents reduce language-learning and rewrite friction enough to make performance-oriented languages and low-level systems work accessible to more developers, while also noting that

prior knowledge improves the result. Treat this as a hypothesis about the capability of a human-agent pair, not evidence that the human has retained the same skill or that an unfamiliar stack is cheaper to own.

Lower entry friction can move cost into comprehension, debugging, security review, operations, and succession. Before adopting an unfamiliar stack, use a bounded spike to compare it with a familiar baseline under the same behavioral tests and explicit performance and resource targets; require a team member to explain and modify the result; and record the toolchain and rollback path. Choose the stack only when the measured benefit survives those ownership costs.

Field note — A boundary discovered across projects

Agents helped me expand Slideotter before its product and backend responsibilities had settled. Developing Kirjolab exposed a shared need: source material and provenance should have one owner, while each product owns its writing or presentation workflow. The Kirjolab case study develops that proposed split. Faster generation moved my work from producing features to deciding what each project should own.

This cost can be visible even when final behavior is unchanged. Trivedi & Schmitt (2026) compared matched clean and deliberately degraded repositories across 660 agent trials. Cleanliness did not change the task pass rate, but agents working in cleaner repositories used 7–8% fewer tokens and revisited files 34% less often. The result does not show that every clean-code rule improves every task, but it supports a narrower claim: maintainability can affect the operational footprint of agentic development, not only human preference.

5.6 Match feedback to the changed boundary

Validation can itself become a source of friction. A ten-minute clean-install or browser suite may provide valuable release evidence while being a poor response to every saved file. Conversely, a fast test that never crosses the changed boundary can create confidence without useful coverage.

For the category change, a focused validation-rule test can return the first failure quickly. An HTTP contract check can then cover serialization and compatibility; a database migration test can cover existing and new rows; a browser check can cover the filter interaction; and a deployed-boundary check can cover the actual cache and rollout behavior. Running the most expensive stage first wastes feedback time, while stopping at the focused stage leaves the web change unverified.

Agent review can keep producing candidate findings after the useful ones are exhausted. Count independently confirmed findings and the human time needed to adjudicate them, and stop against an explicit risk threshold; Appendix A develops the full review-evidence exercise.

For one representative change, map the project's feedback ladder. The table below separates the evidence needed during implementation from the more expensive evidence needed at a changed boundary or release decision.

Table 5.2. The project feedback ladder from focused checks to boundary-specific evidence.

Stage	Evidence returned	Typical cost	When it is required
Focused	One behavior or local invariant	Seconds	During implementation

Stage	Evidence returned	Typical cost	When it is required
Affected	Likely consequences of changed files	Seconds to minutes	Before sharing the change
Baseline	Ordinary project readiness	Minutes	Before accepting non-trivial work
Boundary-specific	Browser, clean install, workflow, performance, security, or mutation evidence	Potentially expensive	When that boundary changed or release confidence is required

Name the purpose of each stage and choose the commands accordingly. Measure both time-to-first-useful-failure and total readiness time. If a slow check repeatedly catches problems that no earlier stage sees, consider whether a cheaper focused signal can move that feedback forward without weakening the final gate.

5.7 Audit the event-tracker workflow

5.7.1 Workflow loop lab

The demonstration below puts the work into a visible loop: human framing, implementation, tool feedback, review, and correction. It separates agent speed from workflow quality, because an agent can produce more output while still pushing uncertainty into human review.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/friction/

Pause & think

Calculate the whole change cycle

A change passes through four sequential stages with no overlap: clarification 10 minutes, editing 10, checks 15, and review 25. A tool halves editing time; every other stage stays unchanged. Calculate the new total and the time saved.

▼ **Show worked answer**

The original cycle takes 60 minutes. Editing falls from 10 to 5 minutes, so the new cycle takes 55 minutes and saves 5 minutes. A local speedup affects only its share of the full cycle.

Use this first to make the human-in-the-loop process explicit. Then use the friction audit to ask which part of that process is creating repeated work.

5.7.2 Friction audit lab

The audit below records observed effort in a bounded change path and makes an intervention's hypothesis explicit. Enter active person-minutes from your records, keeping passive waiting, acceptance evidence, and changed conditions in the notes. The arithmetic can show increased effort as well as a reduction; it does not predict productivity or assign risk from an assistance mode.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/friction/

Apply the same audit to implementation A using the category change or another equivalently bounded request. Observe setup, comprehension, change, testing, review, deployment, operations, and coordination; use timing records and concrete blockages to identify the largest supported source of resistance and choose one intervention to test. Leave unmeasured stages unknown rather than treating them as zero, and record the intervention's introduction and ongoing maintenance costs separately. For one implemented choice, compare introduction with safe reversal: mark its commitment point or record that none was reached, name what now depends on it, and identify the evidence that would show reversal succeeded.

Repeat the unchanged audit for implementation B. The comparison is useful only if the change request and categories remain stable. A difference may come from architecture, tooling, documentation, generated defaults, or team practice; the audit should expose competing explanations rather than attribute every difference to the implementation approach.

5.8 Project artifact

Produce a friction audit for implementations A and B. It should:

- trace one bounded web change from the browser-facing interaction through its contract, validation, data, derived view, cache, and deployment boundaries,
- separate protective friction from repeated work that provides no useful evidence or control,
- identify the first commitment point and explain what safe reversal requires after it,
- explain which decisions remain soft, which have become hard, and what would change their reversibility, linking one consequential choice to its decision-log entry,
- name the largest source of setup, comprehension, change, testing, review, deployment, operational, or coordination friction,
- map the project's validation ladder and identify one check that should move earlier, later, or become conditional,
- and improve one feedback loop that can be measured again without changing the audit request.

5.9 Summary

Follow one change from the browser through contracts, data, validation, and deployment to locate its largest source of friction. Preserve checks that protect accepted behavior, expose the point where reversal becomes a coordinated operation, and move useful feedback earlier where possible. Repeat the same audit to see whether the next implementation actually makes the change easier.

5.10 References

1. Trivedi, P., & Schmitt, O. (2026). Does Code Cleanliness Affect Coding Agents? A Controlled Minimal-Pair Study. arXiv Preprint arXiv:2605.20049. https://doi.org/10.48550/arXiv.2605.20049
2. Ronacher, A. (2026, August). Fast and Hard Code. Personal technical essay. https://lucumr.pocoo.org/2026/8/22/fast-hard-code/
3. Qiu, D., Li, B., & Su, Z. (2013). An Empirical Analysis of the Co-Evolution of Schema and Code in Database Applications. Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering, 125–135. https://doi.org/10.1145/2491411.2491431

CHAPTER 6

Which web application assumptions fail under pressure?

Created
January 16, 2026
Updated
October 2, 2026

A web application that works locally has shown that one implementation can satisfy one workload in one controlled environment. Production changes the environment: browser applications and integrations retry, networks delay or lose responses, live data accumulates, shared dependencies slow down, deployments change running systems, and operators have to recover without discarding authoritative state.

Production failures expose architectural assumptions that do not hold under actual operating conditions. Reconstruct the failure chain, locate the earliest useful signal, and identify a path back to useful service.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

6.1 Local correctness leaves production assumptions untested

Production is not simply a larger version of a developer's laptop. It contains independently changing consumers, intermediaries, services, data stores, providers, and operational tools. A failure can cross several of these boundaries before a user sees an error.

Consider five assumptions that local development can easily hide:

- **Consumer behavior:** a browser, integration, or agent may retry after an ambiguous timeout or reconnect in a synchronized burst.
- **Network behavior:** a request can reach the authority even when its response never reaches the consumer.
- **Dependency behavior:** a slow identity, storage, payment, or messaging service can hold resources and spread waiting through otherwise healthy components.
- **Data behavior:** replicas can lag, writes can conflict, queues can age, and a restored endpoint can still return stale or incomplete state.
- **Operational behavior:** the dashboard, deployment system, credentials, status page, or rollback mechanism may depend on the service being repaired.

These are architectural assumptions because they determine where authority, waiting, retries, and recovery live. Testing the happy path proves none of them.

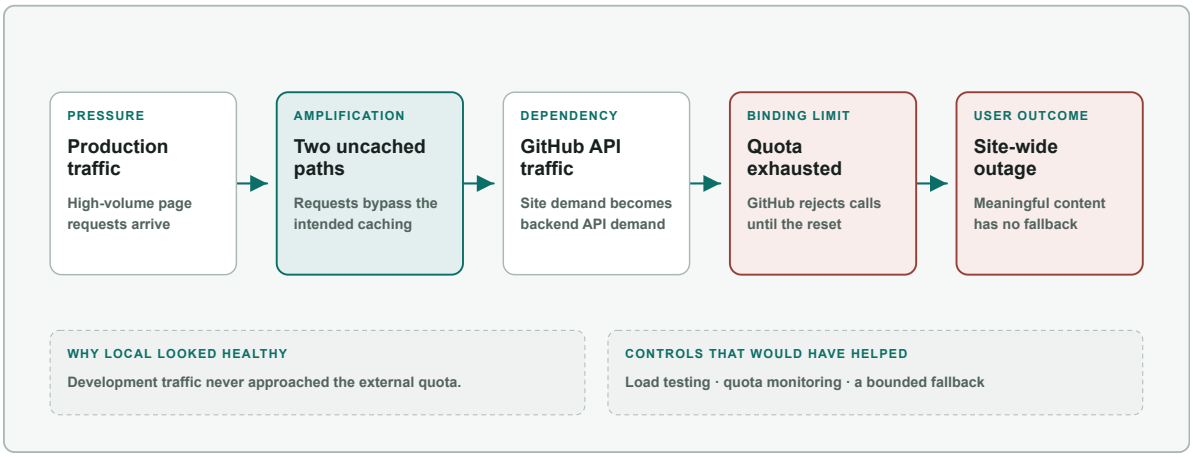
> **Sustainability angle**
>
> Production pressure often exposes waste that local development hides: retry storms, polling, failed jobs, duplicated events, abandoned data, and oversized logs. Backoff, deduplication, retention, and load shedding can reduce both failure amplification and unnecessary resource use.

6.1.1 Field note — When caching complexity became an availability risk

In a commercial Next.js project, GitHub effectively served as the backend for most of the site's meaningful data. Two sections used Next.js caching incorrectly, causing production page requests to repeatedly reach the GitHub API. This remained invisible during local development because our traffic never approached the API quota.

Under high production load, the repeated requests exhausted the quota. Because the site could not present useful content without GitHub, a rate limit on one external dependency became a site-wide outage. During the incident, I debugged the behavior with LLM assistance and found one faulty caching path. After the quota reset, traffic exposed a second path with the same problem, causing another period of downtime.

We eventually removed the GitHub dependency because it had also become unstable for our needs. In retrospect, load testing should have revealed that site traffic was being amplified into API traffic, while monitoring should have warned us before the quota became an availability boundary.



Production traffic was amplified into GitHub API traffic until one external quota became the availability boundary for the whole site.

The incident also reinforced my preference for simpler web primitives. For this kind of site, I consider Next.js's interacting rendering and caching mechanisms more complex than the product warranted. If rebuilding it today, I would use a multi-page application with hypermedia-driven fragments and View Transitions for the interactive portions.

That alternative would still need caching, monitoring, fallbacks, and bounded external dependencies. My hypothesis is that explicit page, fragment, and data-access paths would make this site's behavior easier to explain and operate. The caching incident does not test that counterfactual. I would compare a small implementation of the same journey under the same cache and dependency-failure checks, asking an unfamiliar maintainer to trace a response and diagnose a stale result. If diagnosis remained equally difficult or required more coordination, that would challenge my preference.

6.2 What documented web incidents reveal

The four accounts below were published by the organizations that experienced the incidents. They provide concrete timelines, observations, and recovery choices, but they are not controlled experiments or independent audits. Treat each reported number as evidence about that system and use the transfer question to decide what, if anything, applies to the event tracker.

Table 6.1. Documented web incidents interpreted as failed assumptions and recovery questions.

Incident	Failure chain	Assumption exposed	First useful signal and recovery	Transfer question for the tracker
Discord authentication outage	Zonal maintenance, a degraded disk, cold row caches, limited capacity, timeouts, and retries compounded	Zonal redundancy and a successful short test provided enough margin to continue	HTTP success collapsed; downstream-service time and saturation led to load shedding while capacity returned	Which optional actions can stop before retries overwhelm a shared dependency?
GitHub database partition	A network partition triggered failover, leaving writes in two data-center histories	Automated database failover would produce a topology the application could safely use	Replication topology and faults exposed divergence; GitHub degraded features, failed forward, rebuilt, and drained queues	Which state must remain authoritative even if availability has to be reduced?
Fastly global outage	A valid customer configuration activated a latent software bug across the edge network	A valid configuration change could not trigger a platform-wide failure	Monitoring detected global disruption within one minute; disabling the triggering configuration restored most service	How small is the blast radius of one valid configuration or deployment change?
Cloudflare Workers KV incident	Deployment automation selected the wrong build; KV and tools depending on KV failed together	Tested rollback, authentication, and deployment tooling would remain usable during loss	Immediate alerting identified failure, but recovery required a break-glass route to the previous known-good build	Can the application be recovered when its ordinary control path is unavailable?

Use one incident to identify the earliest point where a prepared control could have interrupted propagation. Then test that reasoning against a smaller event-tracker failure.

6.3 Trace one failure across the web boundary

The Architecture chapter separated consumer, representation, execution, and authority. A production failure becomes understandable when the same roles are traced over time rather than inspected as a static diagram.

Consider a deliberately small event-tracker scenario:

1. A browser sends `POST /events` without a stable idempotency key.
2. The origin validates the request and commits the event to the authoritative store.

3. The response is lost after the commit, so the browser sees a timeout.
4. Client application logic retries because it cannot tell whether the action completed.
5. Another application instance accepts the retry and records a second event.
6. A cached activity view briefly shows neither event and later shows both.
7. Request-success metrics look healthy even though one user intent produced duplicate durable state.

No individual observation explains the whole failure. The timeout is a consumer symptom, the lost response is a network event, the retry is a policy, the duplicate is an authority failure, and the stale view is a derived-state delay. Useful evidence has to connect them through a stable command identifier, request trace, commit result, cache status, and user-visible outcome.

Idempotency is a property of the intended effect: repeating one logical operation must have the same effect at the declared boundary as performing it once. First name every outcome the application contract promises, then check whether repeating the operation preserves them together. An unchanged work-item status is insufficient when the same command also promises one activity event for that transition. The retry exercise asks you to inspect both; Appendix C develops the specific HTTP method semantics.

The corrective design is similarly cross-boundary. The consumer reuses one idempotency key, the authority stores the mutation and reusable result atomically, and the interface makes a pending or confirmed outcome visible. Retries may then recover from transport ambiguity without inventing a second domain action.

6.4 From failed assumptions to useful signals

A system has a **competence envelope**: the range of conditions it can handle with its current resources, architecture, procedures, tools, and expertise. Ordinary disturbances move the system around inside that envelope. Saturation occurs when pressure reaches a binding limit and pushes the system to or beyond the boundary of what it can currently handle (Woods, 2018).

The limit is not necessarily a server at 100 percent CPU. It may be physical, such as memory or network bandwidth; deliberately imposed, such as a bounded queue, connection pool, quota, or rate limit; or organizational, such as responders unable to diagnose several interacting failures quickly enough. Software, infrastructure, operators, deployment paths, and operational tools all affect where the boundary lies (Hochstein, 2026).

A useful incident view needs several kinds of evidence:

- **User outcome:** success rate, error state, latency, duplication, stale data, or unavailable capability.
- **Binding limit:** CPU, disk, connection pool, database quorum, dependency quota, queue capacity, or responder attention.
- **Amplification:** retries, reconnects, fan-out, cache misses, synchronized jobs, or backlog replay.
- **Integrity:** authoritative write result, replication position, invariant violation, or reconciliation status.
- **Recovery:** rollback state, restored capacity, backlog age, replica lag, and whether user-visible behavior has settled.

This framing turns “Will it scale?” into four more useful questions:

1. Which limit is the pressure approaching?
2. Which signal reveals that the remaining margin is shrinking?
3. What failure or feedback loop begins at saturation?
4. Which prepared recovery actions can return the system to a useful state?

6.5 Connect failed assumptions to response patterns

For the lost-response example, the response is an idempotent command: preserve one operation identity and return its recorded result on retry. The new obligation is atomic storage of the state change and reusable result. Other pressures call for different responses; Chapter 8's response guide connects them to the operating contracts they introduce. First check whether the measurements describe a coherent failure.

The Little's Law relationship introduced in the Benchmarking chapter can check one queue or in-flight boundary. If a stable event-processing stage contains an average of 120 waiting or active events and completes an average of 8 events per second, the implied average time in the stage is `W = L / λ = 120 / 8 = 15 seconds`. That calculation does not explain the delay, but it can expose incompatible measurements or show that a claimed user-visible completion time cannot fit the observed backlog and throughput. If arrivals and completions are not approximately balanced during the observation window, record the system as changing or overloaded instead of forcing the averages into the law.

For implementations A and B, select one queue, pool, request path, or other conserved-work boundary. State its boundary, estimate or measure average work in progress, completion rate, and average time in the system over one consistent window, then explain whether `L = λW` is approximately coherent and what the relationship leaves unexplained.

6.6 Recovery is part of the architecture

Recovery is not a generic command executed after diagnosis. The available response depends on what has already become externally visible or durably authoritative.

Table 6.2. Recovery modes and the preparation each requires before an incident.

Recovery mode	What it protects	Required preparation	Main risk
Reject or shed load	Remaining capacity and core operations	Admission rules, retry guidance, and an explicit non-critical boundary	Rejected work may be retried immediately or lost silently
Degrade functionality	Data integrity or one essential user path	Features that can be paused independently and clear user states	Hidden dependency means the supposedly optional work is still required
Roll back	A known-good code or configuration path	Compatible state, retained artifact, health gate, and reachable control	Durable writes or migrations may make reversal unsafe
Fix forward	Accepted state that cannot safely be discarded	Reconciliation plan, bounded change, and strong observation	Recovery takes longer while the system remains degraded
Rebuild and replay	Authoritative history and deferred effects	Backups, logs, replayable work, ordering, and deduplication	Backlog load can trigger another saturation cycle

Recovery mode	What it protects	Required preparation	Main risk
Break glass	Recovery when the ordinary control plane is unavailable	Independent access, narrow authority, audit, and regular testing	Emergency power can bypass normal safeguards

Two boundaries deserve particular attention. First, recovery tooling should not depend entirely on the component it is meant to repair. Second, **service restored** and **recovery complete** are different states. Traffic may succeed while replicas lag, caches remain stale, queues grow, notifications wait, and user-visible inconsistencies still require reconciliation.

6.6.1 Follow a fix through affected deployments

An accepted patch and a successful release report progress at different boundaries from verified remediation of affected deployments. The practitioner critique Everybody's Lost Their Minds argues that getting affected systems updated can remain the limiting step even when vulnerability discovery and patch preparation become faster. Use that argument to ask where a fix waits in your own delivery path and who owns the remaining work.

Where would a successful release leave repair work unfinished? Give your agent the remediation starting point, or inspect the runtime inventory yourself.

6.6.2 Pressure map lab

The incident challenge below turns changed assumptions into a pressure map over three rounds. Investigate one signal and commit one response before time advances. The consequences persist: a response may protect one boundary while moving waiting, cost, correctness risk, or operational work somewhere else.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/reality/

Pause & think

The status is unchanged. Is the operation idempotent?

Before choosing, predict whether an unchanged final status is enough to make an operation idempotent. Write a brief reason naming the effects you would inspect.

A work item starts in progress with no completion activity event. Its completion contract promises to set the status to Done and record exactly one activity event for that transition. The first attempt commits both, but its response is lost. A retry on another instance leaves the status at Done and appends a second completion activity event. Both commits are durable, and there is no deduplication.

Count the stored completion activity events, assess the whole operation against its contract, and select the mechanism that would prevent the duplicate event for retries of this logical command.

▼ **Show worked answer**

There are two completion activity events. The status assignment is idempotent in isolation, but the whole operation is not: its promised effect includes exactly one activity event for the transition. Idempotency is this property of the intended effect, rather than the name of a retry mechanism. One implementation reuses a stable command key and atomically enforces that key while recording the state change, activity event, and reusable result in authoritative storage. A retry then returns the recorded outcome without appending another event. A per-instance cache cannot coordinate retries routed elsewhere, and a longer timeout cannot guarantee that a response arrives. The caller's uncertainty does not change the stored facts given in this scenario. Explain why a transaction that commits both writes together still needs a rule for repeated commands.

Before revealing the worked answer, explain which promised effect your verdict depends on. Then change one assumption: suppose the second record is only a diagnostic request log, while the contract's activity event is recorded once. Would your verdict change, and what makes those records different?

Try one path, inspect the debrief, and replay with a different response. Use the exported brief to connect the operating pressure to the stressed component, binding limit, likely failure, first signal, and prepared recovery. The values are deterministic teaching inputs rather than measured capacity, so the useful evidence is the causal explanation and not the final number.

6.7 Challenge assumptions without changing the common core

Implementation B should not be framed as “make the app bigger” or as an extension of implementation A. Apply the same production pressures to both independent implementations while keeping their required behavior stable:

- many concurrent tenants, projects, groups, locations, or other domain instances,
- tens or hundreds of thousands of users,
- high and bursty event volume,
- client retries and dependency failure,
- eventual correctness where immediate accuracy is unnecessary,
- and a deployment or configuration change that must be stopped or reversed.

Choose one documented case and reconstruct it as a chain of trigger, latent condition, propagation, user effect, first useful signal, recovery choice, and residual work. Then transfer only the structure of that chain to the event tracker. Do not claim that the project has reproduced the source company's scale, infrastructure, or result.

Explain which assumptions differ between implementations A and B and which remain shared. A good reconstruction makes failure and recovery obligations visible without trying to solve every possible future problem. Use that diagnosis to choose and test a response in the Scaling chapter.

6.8 Project artifact

Produce a documented-case transfer and a pressure map for implementations A and B. The artifact should:

- link the selected source and separate its reported facts from your interpretation,
- trace one failure across consumer, network, execution, authority, and operational boundaries,
- name the assumption, binding limit, first useful signal, amplification path, and user-visible effect,
- apply Little's Law to one declared queue, pool, request path, or other conserved-work boundary and interpret its limits,
- connect the failed assumption to one justified response pattern and the new obligation that response would introduce,
- identify a prepared recovery mode and the evidence that would show recovery is complete,
- and explain what differs between the two implementations without changing the common acceptance core.

6.9 Summary

Trace a failure from trigger and latent condition through propagation, user effect, and recovery. Use the first useful signal to locate the broken assumption and evaluate a response with its new obligations. Recovery must protect authoritative state and remain possible when ordinary tools fail; successful requests can resume before reconciliation and deferred work are complete.

6.10 References

1. Woods, D. D. (2018). The Theory of Graceful Extensibility: Basic Rules that Govern Adaptive Systems. Environment Systems and Decisions, 38(4), 433–457. https://doi.org/10.1007/s10669-018-9708-3
2. Hochstein, L. (2026, July). Saturation: How Your Software Will Fail at Scale. Software Should Work conference presentation. https://www.youtube.com/watch?v=PHYCRubnmSM

PART III

Compare — Evaluate workflows and scaling responses

Implementations A and B expose what changed across two single-agent development systems, but they do not yet show what orchestration adds or costs. Compare produces implementation C through a declared agent fleet, applies common scaling pressures to all three applications, and turns their evidence into a justified target architecture and development workflow.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

III.1 From orchestration to an evidence-based decision

The three chapters form one progression:

1. When do agent fleets improve web development? grows one agent into a bounded fleet, makes authority and integration explicit, and treats failure correction as part of implementation C.
2. How should web applications respond to scaling pressure? identifies approaching limits, applies formal models, and compares possible responses without treating a technique catalog as a design.
3. What do three agent-assisted implementations teach us about web scalability? connects the application, development, and agentic loops and turns the three evidence packages into a final decision.

Complete Compare with implementation C and its fleet trace, a three-way application and workflow comparison, and a target design whose claims distinguish observation, inference, and proposal.

III.2 Implementation C — Work Activity Tracker

The Compare project specification defines the third fresh implementation and its bounded fleet workflow. The common behavior, permitted acceptance material, pressures, and reporting categories remain stable while delegation, orchestration, authority allocation, integration, and fleet controls vary deliberately. When practical, use the same comparison model as implementation B so orchestration is the primary changed variable; otherwise record the configuration difference and narrow the comparison claim.

Implementation C also turns its development process into application data. A transformed fleet trace is loaded into the Work Activity Tracker so its history, filters, and relationships can expose attempts, failures, handoffs, validation, correction, and human intervention. Use that view to locate where coordination failed and which correction worked.

III.3 Compare applications and development systems

Explain what each setup made easier or riskier, which application architecture best fits the selected pressure, and what the team can sustain.

Record architectural, configuration, and practice differences as alternative explanations for the result. The final decision should name the conditions under which you would choose the application and development workflow.

CHAPTER 7

When do agent fleets improve web development?

Created
May 14, 2026
Updated
October 2, 2026

An **agent fleet** coordinates independently operating agents on parts of a larger task. It is a cooperative multi-agent system used for development; fleet is the book's working label for the coordination, integration, and acceptance arrangement described here. The broader term multi-agent system also covers agents with different goals and agents that do not use LLMs. Each has its own execution context and tool actions; an orchestration layer assigns work, routes results, and controls shared state. Writing several roles into one prompt does not create those separate executions.

A web feature can expose parallel work across interface, API, and persistence code, but those parts must agree on the same behavior. This chapter follows a fleet from task decomposition through integration, correction, and measurement. “Fleet” here belongs to the development system; agents running inside the delivered application have separate users, data, and operational obligations.

> **Sustainability angle**
>
> Count the fleet's total compute and human review cost. Parallel work earns that cost when it improves accepted outcomes, reduces rework, or saves scarce human time.

Concept connections
Explore definitions and connections in the web edition.
Explore this reading's concepts

7.1 From one agent to a fleet

A fleet should be an answer to the shape of the work, not the starting assumption. Begin with the smallest coordination structure that can complete the task.

Table 7.1. Progression from a single coding agent to an integrated development fleet.

Stage	Structure	Useful when	New cost or risk
One agent	One context and action stream	The change is small, coupled, or depends on frequent interpretation	Limited concurrency; one interpretation can dominate
Sequential delegation	One agent hands a bounded result to the next	Later work genuinely depends on an earlier finding or decision	Handoff latency and accumulated misunderstanding
Parallel specialists	Independent agents work against a settled interface	Tasks touch different modules and have independently testable outputs	Duplicate exploration, conflicts, and a growing review queue

Stage	Structure	Useful when	New cost or risk
Integrated fleet	Orchestration, isolated workspaces, shared contracts, and gates	The workflow repeats often enough to justify explicit infrastructure	Control, observability, recovery, and integration become system work

The first useful expansion is often sequential rather than parallel. A read-only scout maps the relevant routes, components, data flow, and tests; an implementer then receives the compressed findings and a bounded objective. This keeps noisy reconnaissance out of the implementation context without pretending that dependent work can happen simultaneously. An implementer followed by a reviewer is another sequential pattern: the reviewer receives a concrete diff and validation evidence rather than guessing what might change.

Parallelism becomes reasonable only after a shared decision has settled the seam between tasks. For implementation C, one possible dependency graph is:

`reconnaissance → shared domain and API contract → interface work ∥ data work → integration → browser acceptance`

The interface and data branches may run together because both depend on the same contract. The contract itself, integration, and end-to-end acceptance remain sequential. If both branches need to reinterpret the lifecycle, edit the same schema, or wait for the same person, their apparent parallelism merely moves waiting and conflict downstream.

7.1.1 Separate agent concurrency from human work in progress

Agent concurrency counts agents executing at the same time. For one integration owner, count **human work in progress** as started changes still awaiting that person's acceptance, including changes waiting for review or correction. Define what counts as one change before comparing these quantities: interface and data tasks can contribute to the same accepted feature.

Two agents implementing the interface and data path for a CSV export can serve one unfinished change. If the same two agents move on to a feed filter and a lifecycle change while the export waits for review, the owner now supervises three unfinished changes. The worker count stayed at two, while human work in progress increased. Neither count alone establishes accepted throughput, attention cost, or quality.

A worker limit bounds simultaneous execution; a work-in-progress limit bounds how many changes may start before earlier ones are accepted or explicitly abandoned. An owner can keep several workers on one change, or admit several changes while only some workers execute. Record both quantities alongside the review queue and review capacity.

In a practitioner essay, Bleecher Snyder (2026) proposes improving the human feedback loop before hiding model latency with more concurrent tasks. He explicitly reports that one or two agents have not yet matched his many-agent net throughput. This motivates a local comparison, rather than establishing that fewer agents or a lower work-in-progress limit always improve results.

7.1.2 Amdahl's Law and the sequential bottleneck

Amdahl's Law gives a simple upper bound on speedup when only part of the work can run in parallel (Amdahl, 1967). If `p` is the parallelizable share and `N` agents execute that share, the idealized speedup is:

```
S(N) = 1 / ((1 − p) + p / N)
```

If half of a workflow is inherently sequential, unlimited agents cannot make the whole workflow more than twice as fast. In web development, framing the feature, settling a shared interface, integrating overlapping edits, reviewing risk, and accepting the browser-visible result often remain sequential. Real fleets perform below the ideal bound because communication, duplicated exploration, conflicts, and waiting add overhead that the formula does not model.

The lab below keeps the workflow at 100 units of work and changes only the parallelizable share and number of agents. Predict the effect of the next agent before moving the control, then compare the ideal speedup with the infinite-agent ceiling.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/agent-fleets/

Pause & think

Does a faster feature mean more accepted work?

Before calculating, predict whether doubling a fleet from four to eight agents will double accepted output, and explain your prediction.

Each feature needs 80 minutes of implementation with one agent and 20 minutes of review by one person. The agents work together on one feature, divide implementation time perfectly, and start the next feature as soon as implementation ends. Review of earlier features can overlap that implementation. Assume no coordination cost, continuous work, every feature passes review, and no withdrawal.

Calculate completion time for an isolated feature with no queue ahead, then production and review rates for the eight-agent fleet. Use ongoing rates, not counts in the first clock hour. Finally, calculate unfinished-work growth and assess the effect of doubling the fleet.

▼ **Show worked answer**

With four agents, an isolated feature takes 80/4 + 20 = 40 minutes; with eight, it takes 80/8 + 20 = 30 minutes. These correspond to ideal Amdahl speedups of 100/40 = 2.5 and 100/30 ≈ 3.33 for the fixed task. During continuous work, four agents finish implementation every 20 minutes, producing three candidates/hour; eight finish every 10 minutes, producing six. One reviewer can accept at most 60/20 = three features/hour. With eight agents, unfinished work therefore grows by 6 − 3 = three features/hour. Doubling the fleet finishes an isolated feature sooner but leaves accepted throughput unchanged. Later features wait longer as the review backlog grows, so 30 minutes does not describe every feature in that stream. Explain which assumption you would investigate before recommending more agents, and what evidence would justify changing review capacity or admission.

Apply the calculation to one observed fleet task. Name the sequential work and keep coordination time visible rather than hiding it inside the parallel share. The difference between the ideal bound and the observed result is evidence about coordination cost, not proof that the law failed.

Before revealing the worked answer, explain why completion time for one feature and accepted throughput over many features can change differently. Trace the result through review capacity, the queue, and saturation. Then change one assumption: suppose some candidates are rejected, or review times vary. Which rates and waiting times would you measure again, and what evidence would justify adding agents, changing review capacity, or bounding admission?

7.2 What documented web-development cases show

The company-authored accounts below show different ways to organize agentic web development. Read each account as a snapshot of its stated tasks and configuration, not a current estimate of agent productivity. The transfer questions identify what to inspect in the event tracker.

Table 7.2. Documented agentic-development cases and their transfer questions.

Documented case	Practice in the web-development workflow	Reported evidence and important limit	Transfer question
GitHub assigns Copilot work in github.com	Issues become proposed changes to UI, APIs, migrations, and production code	One month of task examples; no counts, review time, or defect rate	Which decisions may an agent execute, and who accepts the result?
Vercel teaches agents product design	Repository skills, linters, and review carry UI and interaction decisions forward	June 2026 workflow account; no general product-quality effect established	How does a web project make product judgment available and maintainable?
Stripe runs parallel Minions	Unattended agents produce pull requests in isolated development environments	February 2026 report of over 1,000 merged PRs per week; no attempt or quality denominator	What infrastructure makes parallel work bounded and reviewable?
Stripe evaluates web integrations	Agents modify frontends, backends, databases, and live payment flows	March 2026 suite of eleven environments with UI and API graders; limited task coverage	What evidence establishes that the resulting web behavior is correct?

Vercel's separate January 2026 Next.js evaluation reported that an available skill went uninvoked in 56% of cases under its default setup. That is evidence about guidance retrieval in that evaluation, not the success rate of the later product-design workflow or a universal ranking of skills against instruction files. The transferable check is to observe whether guidance was loaded before judging whether it was followed.

For implementation C, connect each delegated task to its acceptance check and integration owner. The comparison with a single agent must include failed attempts and human review, which the published activity counts do not always expose.

7.2.1 Field note — Different reviewers expose different blind spots

I have found GPT useful for reviewing implementations, and a colleague's Claude review of one of my pull requests surfaced different issues. Claude also sometimes overstated an issue or preferred a different design. The disagreement made assumptions visible; resolving it still required a judgment about the intended behavior and the simpler workable design.

I usually ask the agent to implement a fix once I agree with its reasoning. Different reviewers can broaden the search, but I remain responsible for accepting the direction.

7.3 Build a small web-development fleet

Use the course-assigned environment and record its version, model routes, delegation settings, and tool permissions. The lab requires bounded child tasks, inspectable results, and an explicit integration owner. Appendix D's environment guide helps you inspect these boundaries and offers optional implementation examples.

7.3.1 Prepare the project before adding agents

Follow the setup instructions published for the assigned harness and start in a version-controlled project. Confirm that the single-agent baseline has delegation disabled before enabling the declared fleet configuration. Before adding subagents, ask one agent to inspect the repository:

> Inspect this project and propose an `AGENTS.md`; do not edit yet. Include the project purpose, setup and validation commands, coding conventions, architectural boundaries, forbidden operations, and review checklist. Cite the files that support each proposal and mark unknowns.

Review the proposal before saving it. An instruction file is maintained project policy, not an automatically authoritative summary. For a web project it should expose at least the development command, unit and browser-test commands, route and persistence locations, styling and accessibility expectations, responsive states, secrets policy, and the definition of done.

Project memory needs a lifecycle. Keep unresolved questions and temporary findings in a working artifact; record consequential choices in a decision log; promote settled behavior into the relevant specification or architecture document; and connect it to executable checks. A discovery transcript should not silently become the specification, and parallel agents should not work from different generations of the same contract. Repeatable review or debugging workflows can become small project skills, while clear mechanical rules belong in linters or tests.

7.3.2 Define the web task graph

The fleet produces the complete common core of implementation C rather than four unrelated features. Settle the domain vocabulary, lifecycle transitions, event schema, API shapes, error semantics, and browser acceptance journey before starting parallel implementation. Then assign bounded work such as:

Table 7.3. A minimal fleet bound to cross-layer concerns in implementation C.

Task	Web-specific responsibility	Boundary and required output
Contract scout	Map routes, domain transitions, events, UI states, storage, and existing checks	Read-only; return file references, unknowns, and proposed contract decisions
Interface implementer	Build forms, loading, empty, success, and error states; preserve accessibility and responsive behavior	Own named UI files; return focused checks and screenshots or browser observations
Data implementer	Implement persistence, valid transitions, event atomicity, API responses, and idempotency	Own named server and data files; return API, migration, and invariant checks

Task	Web-specific responsibility	Boundary and required output
Verifier and integrator	Integrate both branches and exercise the complete user journey	May repair only assigned integration defects; return browser evidence and limits

Role names are illustrative. Grade whether every task has an objective, allowed files and tools, expected output, validation commands, non-goals, owner, and stopping condition. If two roles must edit the same shared contract, resolve that decision before they branch or reduce the fleet. Use separate workspaces or worktrees for concurrent writers, and pin the same declared base model for the single-agent and fleet runs where possible. Record any substitution as a confounding factor.

7.3.3 Research note — A clean merge is not a correct integration

CodeCRDT's experiments on six small TypeScript and React tasks achieved textual convergence while preliminary inspection still found semantic conflicts (Pugachev, 2025). A shared document can merge successfully even when interface and data code disagree about the same event. Keep integrated behavior checks after concurrent work. Appendix A examines the study and related coordination evidence, including their limits.

7.3.4 Map authority and acceptance

Reuse the constrain–generate–validate–revise loop. Delegation changes who performs the work and where results must come together; it does not remove any stage. For each consequential decision, name the performer, accountable owner, and whether the agent may **propose**, **prepare**, or **execute**. Authority can differ between decisions within one task (Parasuraman et al., 2000).

Table 7.4. The same development loop with explicit fleet responsibilities and authority.

Stage	Fleet work	Authority and acceptance boundary
Constrain	Settle the requirement, shared contract, task graph, and stopping rules	Agents propose alternatives; the owner resolves ambiguity and authorizes the scope
Generate	Execute bounded tasks in isolated workspaces and integrate their outputs	Agents act within declared files, tools, and budgets; the integrator owns the combined candidate
Validate	Check focused results and the complete browser journey	Agents provide evidence and a proposed verdict; the accepting owner reviews failures and limits
Revise	Diagnose a mismatch, assign one correction, and update the contract or checks	The owner resolves any scope change; acceptance requires validation of the corrected revision

For example, an interface agent may discover that a requested field requires changing a shared event schema. It should present the affected consumers, compatibility checks, and migration choices to the system owner before either branch adopts a new contract. State which independent tasks may continue while that decision waits and which must stop.

An effective escalation gives the recipient enough information to understand the state, anticipate the effect, and redirect the work—the observability, predictability, and directability emphasized by Coactive Design (Johnson et al., 2014). For each approval or stop rule, name the component that enforces it; a marker the producing agent can bypass is insufficient.

Trace one delegated task from originating actor through session, tool attempts, validation, and final outcome. Grant only the required tools, data, and external access, and preserve the identity and policy evidence needed to reconcile those attempts. Ownership continues through release, observation, and

later correction (Nissilä, 2026). Publishing a patch, deployment, or message requires an accountable human who can explain, correct, or withdraw it. Appendix D develops enforcement, imported-capability review, and trace reconciliation.

7.4 Practice failure and correction

The labs expose where a delegated task diverges from its contract and whether the workflow can recover. Start with one small mismatch whose effect can be checked through the application.

7.4.1 Field note — Correct the direction before the details

Sometimes I stop an agent session, discard its unaccepted changes by returning to the last known-good Git state, and begin again. Rebuilding context can be cheaper than repairing work based on the wrong assumptions.

For user-interface work, I inspect the composition and interaction model before asking the agent to polish details. A discovery prototype can tolerate more inconsistency; a production interface needs earlier review before a weak direction spreads.

Screenshots preserve the rendered state I am discussing. I am also experimenting with shared browser sessions and feedback captured inside the application: attach my description to the UI state and location where the problem appears. That remains a direction I am exploring, but it could make correction less dependent on reconstructing what I saw.

7.4.2 Delegation lab

Use the planner below before running real agents. Choose a task and project-readiness level, then change the roles and model allocation. The generated packet is a rehearsal: if the task is vague, boundaries overlap, or the review burden exceeds available capacity, reduce the fleet or harden the project first.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/agent-fleets/

Translate the packet into a task graph rather than launching every named role automatically. Run three small patterns before the complete implementation: scout then plan, implement then review, and parallel scouts over independent subsystems. For each pattern, state why the tasks are sequential or independent and who integrates the result.

7.4.3 Correction lab

After the fleet integrates implementation C, select one seeded defect from the table below. The instructor can supply a faulty branch, or you can introduce the fault after saving a known-good baseline. Do not reveal the chosen fault to the diagnosing agent.

Table 7.5. Small faults that expose fleet integration and correction problems.

Seeded defect	Why local work may look correct	Web-visible or state-visible failure
UI and API use different event or field names	Each side's focused tests use its own vocabulary	A user action appears successful but activity or summary does not update
An event is written before transition validation	The valid-transition test remains green	A rejected action still changes event history or derived counts
A retried mutation has no idempotency boundary	One request produces the expected record	A network retry duplicates an event or transition
Interface and data agents changed the same contract	Both branches pass before integration	The merged browser journey fails or displays a stale state
The verifier runs unit checks but no browser probe	Components, handlers, and queries pass independently	The complete user journey cannot finish

Apply the same development loop to the seeded fault:

1. **Constrain:** preserve the known-good baseline, settled contract, and expected user journey; name the integration owner and correction scope.
2. **Generate:** integrate the candidate containing the seeded defect and retain the contributing agents' focused results.
3. **Validate:** rerun those checks, then create a workspace and work item, perform valid lifecycle transitions, confirm activity and summary, filter the feed, and refresh to verify persistence. Attempt an invalid transition and verify that it changes neither current state nor event history.
4. **Revise:** use the diff and fleet trace to locate the first divergence from the contract, then assign one bounded correction with an explicit hypothesis. Avoid several speculative fixers against the same files. Preserve the failed attempt and repair in the trace, update the relevant check or contract, and return to validation: both the focused regression check and complete browser journey must pass before acceptance.

A focused check can pass while the complete user journey fails. Repeated agent agreement can also preserve a shared blind spot; resolve findings through behavioral evidence. Appendix A develops repeated review, and Appendix E models detection and repair across a reliability chain.

7.4.4 Measure accepted throughput

Measure time and cost through acceptance, including failed runs, duplicated exploration, handoffs, merge conflicts, retries, model use, and active human integration and review. Record reviewer backlog alongside completed tasks to see whether the fleet is exceeding the team's acceptance capacity.

Treat one end-to-end fleet attempt as one trial. The internal agents, retries, and handoffs are parts of that trial rather than independent samples. For a narrower workflow contrast, use the Compare project's bounded paired task: fresh copies of the same frozen revision, the same brief, model, checks, environment, intervention policy, and aggregate resource limits. The full B and C builds remain case studies because their design choices and the team's experience differ. One matched pair supports an exploratory observation about this task; reliability or general effect claims require a justified sampling design and repeated trials.

Finally, transform the execution trace into implementation C. The fleet run maps to a workspace, delegated tasks to work items, agents and humans to actors, and attempts, validations, failures, reviews, and interventions to activity events. Use the application's history, filters, summaries, and relationships

to answer two questions: where did coordination first fail, and which correction produced accepted behavior? A portable fleet-trace example provides a small transformation target, mapping guide, redaction record, and repeat-import contract. Adapt the actual tool export to that level of evidence rather than treating the example as a universal agent-tool schema. Use the application to inspect the workflow that built it.

7.5 Optional extensions

To investigate a specific boundary further, use Appendix D's exercises on retrieving changed guidance, checking contract completeness, or testing a human review gate. The walkthrough above provides the required path from decomposition through correction and measured coordination cost.

7.5.1 Optional exercise — Vary human work in progress

If the fleet trace shows competing review requests, explore admitting fewer unfinished changes with the same worker limit. Give your agent the work-in-progress starting point to plan the comparison, or read the trial protocol yourself.

7.6 Project artifact

In implementation C's existing Compare evidence package, connect the fleet plan to what actually happened:

- declared model and harness configuration, task dependencies, authority, integration ownership, and stopping rules;
- the correction probe and comparable measurements through acceptance;
- the transformed trace loaded into C, with findings that explain coordination failures and successful corrections.

Preserve consequential changes to the plan and link acceptance to the checked revision. The specification defines the complete submission requirements.

7.7 Summary

A fleet needs independent tasks, settled shared contracts, isolated work, and a path to one checked application. Its trace should explain failures and corrections as well as successful delegation. Compare accepted outcomes, latency, total cost, and human review effort with the single-agent workflow before increasing concurrency.

7.8 References

1. Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. Proceedings of the April 18–20, 1967, Spring Joint Computer Conference, 483–485. https://doi.org/10.1145/1465482.1465560
2. Pugachev, S. (2025). CodeCRDT: Observation-Driven Coordination for Multi-Agent LLM Code Generation. arXiv Preprint arXiv:2510.18893. https://doi.org/10.48550/arXiv.2510.18893
3. Parasuraman, R., Sheridan, T. B., & Wickens, C. D. (2000). A Model for Types and Levels of Human Interaction with Automation. IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans, 30(3), 286–297. https://doi.org/10.1109/3468.844354
4. Johnson, M., Bradshaw, J. M., Feltovich, P. J., Jonker, C. M., van Riemsdijk, M. B., & Sierhuis, M. (2014). Coactive Design: Designing Support for Interdependence in Joint Activity. Journal of Human-Robot Interaction, 3(1), 43–69. https://doi.org/10.5898/JHRI.3.1.Johnson
5. Nissilä, T. (2026, August). Päätöspolku on ohjelmistokehityksen uusi organisaatiokaavio. LinkedIn article in the HAR: Human Agent Relationship newsletter. https://www.linkedin.com/pulse/p%C3%A4%C3%A4t%C3%B6spolku-ohjelmistokehityksen-uusi-tapio-nissil%C3%A4-0ke2f/
6. Bleecher Snyder, J. (2026, October). ETOOMANYTHINGS? Run Fewer Agents. exe.dev blog. https://blog.exe.dev/etoomanythings

CHAPTER 8

How should web applications respond to scaling pressure?

Created
January 16, 2026
Updated
October 5, 2026

Scaling changes a system in response to a specific pressure. Each response should relieve an identified limit while making its new costs and failure modes visible.

Concept connections
Explore definitions and connections in the web edition.
Explore this reading's concepts

8.1 Start with the pressure

Use the pressure map from the Reality chapter to select a response hypothesis. The table below connects common pressures to the contracts each response adds.

Table 8.1. Scaling response hypotheses and the obligations they introduce.

Pressure or failed assumption	Response pattern worth considering	New obligation and evidence
Writes and reads compete for one path	Separate the authoritative write path from derived read views	Define freshness, rebuild, and reconciliation; measure write completion and read usefulness
A burst contains work that need not finish synchronously	Queue or buffer bounded background work	Expose pending and terminal states, queue age, retry behavior, and worker saturation
A request may be repeated after an ambiguous result	Use a stable idempotency key and authoritative deduplication	Store the mutation and reusable result atomically; test a retry after a lost response
One tenant or domain instance consumes shared capacity	Partition, quota, or isolate work by a meaningful domain boundary	Define routing, fairness, cross-partition operations, and migration
Incoming work exceeds the rate at which it can be completed	Apply backpressure, admission control, rate limits, or load shedding	Make rejection explicit, bound waiting, guide retries, and preserve the essential path
Optional work threatens a core operation	Degrade a named capability while preserving the core	Define the degraded user state, dependency boundary, recovery trigger, and completion signal
History grows beyond the useful query or retention path	Add a measured index, derived summary, archival, or deletion path	Account for write cost, staleness, rebuild, retention policy, and restored-state checks

Combine techniques only where the pressure requires them. A queue can duplicate effects without idempotency, and controlled rejection can amplify load if clients immediately retry.

8.1.1 Scaling pressure lab

The demonstration below starts from a concrete event-tracking pressure: event writes spike, dashboard reads get slower, and the latest-events query has to stay useful without pretending the system needs every scaling technique at once.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scaling/

The important part is the exported trade-off table. Use it to explain why a chosen technique fits the pressure, what it costs, what failure mode it introduces, and which signal would show whether the design is working.

8.2 Name the saturation boundary

Scaling pressure becomes operationally important when it approaches a limit. The competence-envelope model introduced in the Challenge part calls the boundary condition **saturation**: the system can no longer absorb the disturbance with its current capabilities (Woods, 2018). Naming the saturated resource prevents "add more servers" from becoming the default response. The table below groups common limits by the evidence that reveals their approach.

Table 8.2. Categories of saturation limits and their observable signals.

Limit category	Examples	What approaching saturation may look like
Physical resources	CPU, memory, disk space, storage I/O, network bandwidth	Growing wait time, swapping, failed allocations, full disks, or reduced throughput
Virtual resources	Connection and thread pools, file descriptors, bounded queues, quotas, rate limits	Pool wait, rejected work, `429` responses, queue growth, or quota errors
Dependency capacity	Databases, identity providers, third-party APIs, provisioning systems	Timeouts, blocked callers, correlated downstream errors, or delayed scaling
Operational capacity	On-call attention, deployment speed, diagnostic access, coordination	Slow decisions, unsafe workarounds, competing responders, or recovery blocked by normal process

Saturation can create reinforcing feedback loops: overload causes timeouts, timeouts cause retries, and retries add load. Synchronous fan-out can overload an internal service, and blocked threads can make a CPU-based autoscaler misread demand. A resource leak may separate the causal change from the eventual failure by hours or months. These dynamics explain why a locally sensible mechanism can worsen a system-wide incident and why the pressure map must identify both the first limit and what happens after it is reached (Hochstein, 2026).

8.2.1 Saturation boundary lab

The model below compares one effective event rate with four declared stage limits. Increase demand until a boundary becomes binding, expand that limit, and observe which constraint becomes visible next. Then enable backpressure and compare hidden backlog growth with explicit rejection.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scaling/

Pause & think

Locate the next capacity limit

Each request visits every stage once. Capacities are API 180, database 110, workers 150, and downstream 130 requests/s. Offered load is 150 requests/s, with no retries. Use the smallest stage capacity as the throughput ceiling. Then raise database capacity to 150 requests/s, leaving all other capacities unchanged.

▼ **Show worked answer**

Initially the database limits throughput to 110 requests/s. Increasing its capacity moves the limit to the downstream stage at 130 requests/s. Offered load still exceeds capacity by 20 requests/s.

Treat the rates as a reasoning aid rather than measured capacity. The lesson is structural: removing one bottleneck changes the system's limiting factor, while retries can consume headroom before useful demand changes.

8.3 Use a formal model before choosing a technique

Formal models help challenge an intuition about the limit. Choose the question before the equation:

Table 8.3. Questions that guide the choice of a formal model.

Question	Model and worked treatment
Do backlog, throughput, and average time agree?	Little's Law
How much speedup can the parallel part provide?	Amdahl's Law
How do failures across critical steps combine?	Series reliability
Can coordination make added concurrency slower?	Universal Scalability Law

Appendix E collects the equations, assumptions, and limits.

Before opening the concept references, choose between Little's Law and Amdahl's Law for each question below. Explain your choice before checking the arithmetic.

Pause & think

Choose the question before the equation

You have two questions. First, a stable event pipeline completes 40 events/s with average time 0.25 s inside one declared boundary: what average work in progress is consistent with those observations? Second, a fixed development task has an 80% parallelizable share and a 20% sequential share: what is the ideal speedup with four workers and no added coordination cost? Choose a law for each question, then calculate its result.

▼ **Show worked answer**

Little's Law answers the flow-consistency question: 40 × 0.25 = 10 events. Amdahl's Law answers the fixed-workload speedup question: 1 / (0.2 + 0.8/4) = 2.5. Matching averages do not identify a bottleneck or prove acceptable latency. Ideal speedup does not guarantee accepted-change throughput: integration overhead, variable tasks, and correctness can change the result. Explain which measurement or assumption you would check before recommending more workers.

What would you still need to measure before recommending more workers? Connect the answer to throughput, latency, and the assumptions of the chosen model.

The project does not require every model. Apply at least one suitable law or formal model to an application-architecture decision and one to the development workflow. Each application should identify the variables, state the assumptions, show the calculation or reasoning, connect the result to a measured or proposed limit, and explain what the model leaves out. A formula added after the design decision is not evidence that the formula guided the decision.

8.3.1 Contention beyond Amdahl's bound

Amdahl's fixed sequential share omits coordination costs that grow with concurrency. The contention curve in Appendix E makes that difference visible and examines a production case of limiting admitted work. Fit its parameters to representative throughput measurements before using it for a capacity claim.

8.4 Three documented cases of web scaling pressure

Scaling techniques become useful in combinations rather than as isolated vocabulary. The cases below begin with published production accounts from GitHub, Spotify, and Shopify. They provide observed pressures, interventions, and outcomes that a fictional scenario cannot. They are still company-authored retrospectives rather than controlled experiments: they report what each organization measured in its own system, and several changes may have occurred together. Treat their reported outcomes as evidence about those systems, not as performance promises for another application.

Read each case through the same questions:

1. What did users experience?
2. Which resource or boundary approached saturation?
3. What evidence exposed that boundary?
4. What did the organization change?
5. What improved, and what new trade-off appeared?
6. Which part transfers to the event-tracking project, and which part does not?

The table below previews the evidence and the transfer question in each case.

Table 8.4. Documented production pressures and the questions they raise for the project.

Production case	Observed pressure	Intervention described by the source	Reported evidence	Transfer question for the tracker

Production case	Observed pressure	Intervention described by the source	Reported evidence	Transfer question for the tracker
GitHub Issues navigation	Repeated page visits still paid network and rendering costs	Persistent client cache, stale-while-revalidate, controlled preheating	More React navigations became instant, with measured cache divergence	Which repeated read can tolerate controlled staleness?
Spotify podcast-video ingestion	Transcoding capacity saturated and queued publication work aged	More capacity, work prioritization, earlier alerts, planned backpressure	Publication moved from minutes to hours during the incident	What does acceptance mean, and how will users see pending work?
Shopify flash-sale request processing	Slow requests and browser refreshes consumed application-worker time	Detect and discard requests whose clients had already disconnected	Order-of-magnitude spikes, full request queues, and `499` responses exposed waste	Which work should be refused, cancelled, or shed before adding hosts?

> **Sustainability angle**
>
> Scaling techniques have resource profiles. Caches, queues, replicas, CDNs, shards, and multi-region deployments can improve user experience, but each adds infrastructure and operational cost. Ask which pressure justifies the extra resource use.

8.5 Orchestration maintains declared capacity

Horizontal scaling is not merely the instruction to "run more servers". An orchestrator repeatedly compares a declared desired state with the state it observes and takes action to close the gap. A Deployment can declare a replica count and pod template, while a Service gives clients a stable route as individual pods change.

8.5.1 Orchestration control-loop lab

The lab below runs Webernetes, a partial browser-based port of Kubernetes intended for interactive teaching. Change the desired replica count, send requests through a service, terminate a pod, and perform a rolling update. Watch for the distinction between the operator's instruction, the temporary mismatch, and the controller action that restores the declared state.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scaling/

This is a mechanism demonstration, not a capacity benchmark or a production Kubernetes environment. The browser runs compatible control logic and a simulated network, but it does not execute real container images or model persistent volumes, resource limits, autoscaling, and the full production networking stack. Ask which component noticed the mismatch, what action it took, and which scaling problem remains unsolved.

Replica orchestration cannot make every component horizontally scalable. If each application instance depends on one saturated database, holds incompatible local state, or performs the same expensive downstream work, adding pods may move or amplify the bottleneck. Treat the replica count as a response hypothesis and measure whether useful capacity or availability improves.

8.6 Latency and geography

Geography becomes a scaling pressure when physical distance, regional failure, data residency, or a geographically distributed user base approaches an application requirement. The fastest handler still waits for network propagation and every sequential round trip consumes more of the latency budget.

Static assets and public cacheable responses can often move closer to users through a CDN or edge cache without moving the authoritative write path. Personalized data and mutations are harder: executing code at the edge does not remove the distance to a single authoritative database. Moving or replicating that authority introduces routing, failover, consistency, conflict, and residency decisions. Measure which bytes and round trips dominate before treating multi-region deployment as the inevitable final stage of scaling.

8.6.1 Partitions force an operational choice

For a distributed read-write object in an asynchronous network, a network partition can prevent a design from guaranteeing both linearizability and an available response from every non-failing node (Gilbert & Lynch, 2002). **Linearizability** requires one operation order that respects real time: a read starting after a completed write must see that value or a later write. **Availability** requires every request to a non-failing node eventually to return a response allowed by the object's contract; it sets no latency bound.

For the tracker, suppose Alice and Bob send different start commands for the same initial work item to regions that cannot communicate. If both regions accept the transition locally, both clients receive success even though the lifecycle permits only one initial-to-active transition. Reconciliation can repair stored state, but it cannot retract the success already observed by either client. A majority-quorum design can keep one accepted history by making the isolated minority delay or reject the request.

Choose which operations may be stale, rejected, delayed, merged, compensated, or repaired without violating the domain. Appendix C's CAP proof and partition lab develop the reasoning and compare the two policies.

In some project themes, geography may matter very little. In others, a platform serving users or devices in different regions may need to consider latency, data residency, or regional failure. Justify multi-region architecture against the project's latency, availability, and operating requirements.

8.6.2 Latency budget lab

The demonstration below treats latency as a budget. Some of the budget is bounded by physics: a signal cannot cross distance instantly, and real networks add routing and protocol overhead. Some of the budget is architectural: every sequential database round trip adds another wait.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/scaling/

The point is not to memorize exact numbers. Real latency depends on providers, routing, congestion, TLS, database load, and query plans. The useful lesson is the shape of the problem: distance creates a lower bound, and repeated sequential database access multiplies that bound.

8.7 Trade-offs

For each mechanism, explain what becomes simpler and what becomes harder:

- Caching reduces repeated computation but adds freshness and invalidation decisions.
- Queues shorten synchronous work but add backlog, retries, idempotency, and delayed completion.
- Partitioning reduces shared-resource pressure but adds routing, migration, and cross-partition queries.

The "waterbed" metaphor describes this movement of complexity (Randal et al., 2003); it is a review question, not a law that complexity must remain constant. Ask what pressure justifies the mechanism, why a simpler design is insufficient, and which new failure would make you reconsider.

> **Sustainability angle**
>
> Retaining raw data, precomputing summaries, and reserving peak capacity consume resources even when they protect a useful goal. State that goal and measure the recurring cost.

8.8 Implement the smallest justified response

A target diagram shows that a group can reason about a possible architecture. It does not yet show that the response works in the selected application, preserves required behavior, or moves the expected limit. Complete one bounded implementation experiment before finalizing the target design.

8.8.1 Field note — Thesis Journey Tracker

Thesis Journey Tracker serves only two people, so runtime scale was not the problem. The goal was to deliver the project quickly while keeping it easy to maintain and adjust as the supervision workflow changed. I deliberately chose a lightweight, server-rendered interface instead of a conventional single-page application. The first agent-generated version rendered complete pages, but less than two hours later the agent introduced a narrow HTML-fragment endpoint for the student panel. A small browser script replaced that panel without a full refresh and retained ordinary navigation as a fallback. This provided responsiveness where it mattered without requiring a client-side application architecture for the whole system.

Select implementation A, B, or C and one pressure for which the group has runtime evidence or a credibly projected limit that the available environment can exercise. Preserve the accepted revision and its workload result, then implement the smallest response that addresses that limit. Suitable experiments include a measured query index, a bounded cache, an idempotency boundary, admission control, a queue with an explicit pending state, tenant isolation, or another technique justified by the evidence. Distributed infrastructure is not required.

Keep the workload, fixture, correctness checks, and measurement procedure unchanged where the response is intended to affect them. After the change:

1. rerun the common acceptance scenarios and inspect for behavior displaced by the response;
2. rerun the selected pressure and report every repetition rather than only the best result;
3. compare latency, throughput, errors, correctness, and the chosen resource-efficiency proxy before and after;
4. exercise one failure or recovery obligation introduced by the response, such as stale data, a duplicate retry, a full queue, a rejected request, or an unavailable worker;
5. state whether the result supported, challenged, or left the original hypothesis unresolved.

An unsuccessful intervention remains useful evidence when the baseline, change, and failure are preserved. Do not replace a negative result silently with a more elaborate design. The target architecture can retain the response, revise it, or reject it, but it should distinguish this implemented evidence from its remaining proposals.

8.9 Prepare to be surprised

Design can expand the competence envelope, but finite systems always retain another boundary. Graceful degradation keeps selected functions useful when capacity is constrained. **Graceful extensibility** is a complementary idea: when an unanticipated condition exceeds the designed envelope, the sociotechnical system can adapt its capabilities and recover (Woods, 2018).

That ability requires optionality prepared before the incident:

- dynamic configuration and feature flags that expose understood controls;
- fast, practiced paths to restart, scale, shed work, roll back, or fix forward;
- diagnostic access and telemetry that remain useful under pressure;
- people who understand the system and have authority to act;
- exercises that keep rarely used recovery paths discoverable and safe.

More controls are not automatically better. An undocumented switch or unsafe manual path creates another failure mode. The goal is to balance work that prevents known failures with work that improves the ability to respond when the design encounters a surprise.

8.10 Project artifact

Produce three connected deliverables:

1. **Three-way comparison.** Apply the same pressure set to implementations A, B, and C. Mark each result as observed in running code or proposed.
2. **Implemented response.** Preserve the before-and-after revisions, unchanged workload and acceptance checks, observed result, new failure or recovery obligation, and conclusion.
3. **Target design.** Justify the selected responses and explain why simpler alternatives are insufficient.

Across the package:

- connect at least one pressure and proposed response to a documented production account;
- separate what the source reports, your interpretation, and the transfer you propose for the project; do not assume that the source's scale, result, or trade-off carries over;
- apply one formal model to an application decision and one to the development workflow.

Use one trade-off table row for each response in the target design. Record:

- the pressure and approaching limit;
- the technique, intended benefit, cost, and new failure mode;
- the operational signal and at least one prepared recovery action;
- where complexity moved, such as into clients, data freshness, background operations, routing, migration, or incident response;
- why a simpler alternative was not enough.

If the target design contains asynchronous multi-step work that can perform an external side effect, include one crash-after-side-effect audit. Record:

- the last durable coordination fact and the external effect that may already have occurred;
- the timeout and retry rule;
- the retry-safety or deduplication mechanism, its owner, and any stable key it uses;
- the terminal-state behavior and evidence available to an operator.

If no component at the effect boundary can deduplicate, state that duplicate delivery remains possible.

8.11 Summary

Choose a scaling response from the pressure and first binding limit. Use a suitable model to bound expectations, implement the smallest justified change, and repeat the workload through the real deployment. Judge both the improvement and the consistency, recovery, resource, and ownership costs it introduces.

8.12 References

1. Woods, D. D. (2018). The Theory of Graceful Extensibility: Basic Rules that Govern Adaptive Systems. Environment Systems and Decisions, 38(4), 433–457. https://doi.org/10.1007/s10669-018-9708-3
2. Hochstein, L. (2026, July). Saturation: How Your Software Will Fail at Scale. Software Should Work conference presentation. https://www.youtube.com/watch?v=PHYCRubnmSM
3. Randal, A., Sugalski, D., & Tötsch, L. (2003). Perl 6 Essentials. O'Reilly Media. https://www.oreilly.com/library/view/perl-6-essentials/0596004990/

4. Gilbert, S., & Lynch, N. A. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. SIGACT News, 33(2), 51–59. https://doi.org/10.1145/564585.564601

CHAPTER 9

What do three agent-assisted implementations teach us about web scalability?

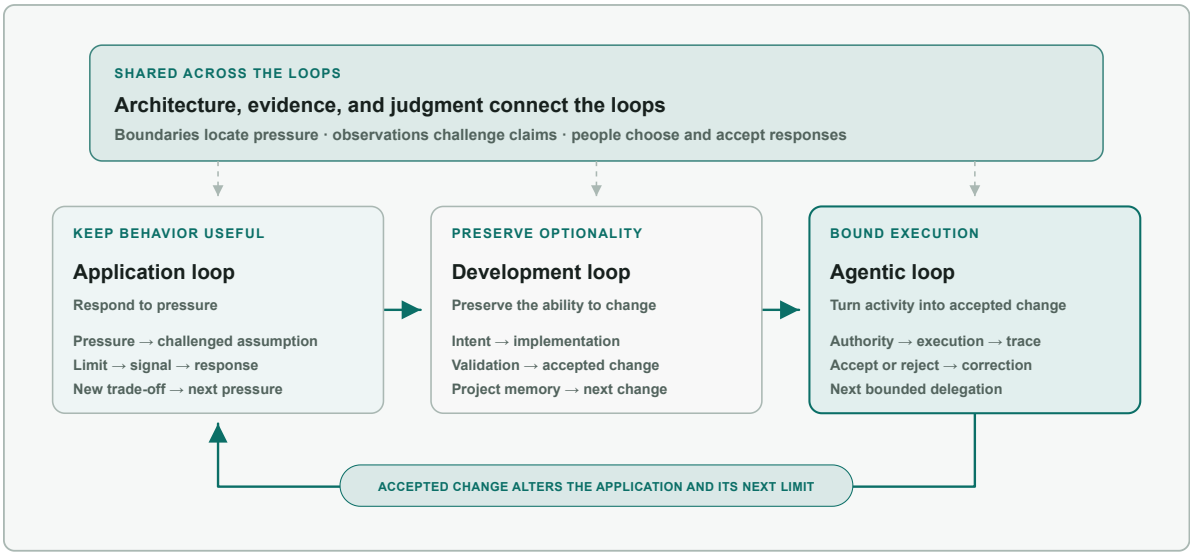
Created
May 14, 2026
Updated
October 1, 2026

Scalability is the ability to keep a system useful while its conditions change. The three implementations show how application behavior, the cost of change, and the development workflow constrain one another. Use their evidence to choose an architecture and workflow for a stated future scenario.

Concept connections
Explore definitions and connections in the web edition.
Explore this reading's concepts

9.1 One system, three connected loops

A scaling response changes the application and becomes a development task. The development workflow determines whether people and agents can implement it safely; the resulting system creates the next set of observations.



Pressure becomes development work; accepted agent-assisted change alters the application and its next likely limit.

9.1.1 The application loop: respond to pressure

Changing demand challenges an architectural assumption. Measurements locate the first limit, and a response changes capacity, correctness, or recovery behavior. Trace which signal identified the limit and what new failure mode or operating cost the response introduced.

9.1.2 The development loop: preserve the ability to change

Each response must be implemented, checked, and maintained. Contracts, tests, setup, and project knowledge determine how expensive that work becomes. Identify what made the change difficult and which lesson became an executable check or maintained decision record.

9.1.3 The agentic loop: turn execution into accepted change

The constrain–generate–validate–revise loop allocates authority, produces a candidate, checks it, and corrects failures. Its trace exposes attempts, handoffs, and human intervention that the final repository can hide. Determine whether delegation reduced completion time through acceptance or moved effort into integration and review.

9.2 How the loops affect one another

Consider adding a queue to protect event ingestion during a traffic spike. The response affects all three loops:

Table 9.1. One scaling response interpreted through all three loops.

Perspective	Consequence of the queue
Application loop	Burst absorption improves, but freshness lag, backlog growth, duplicates, and worker saturation become possible
Development loop	The project gains worker code, retry and idempotency rules, new local setup, deployment, tests, and ownership
Agentic loop	Tasks may decompose around producer, consumer, and verification boundaries, but their shared event contract must settle first
Evidence connecting all three	Queue age, end-to-end event visibility, duplicate rate, failure recovery, review effort, and trace history show whether the response worked

A faster request path may leave events waiting longer for processing and make the system harder to operate. Evaluate both the pressure relieved and the new obligations.

> **Sustainability angle**
>
> Which additional computation, storage, bandwidth, and human attention produced useful capacity or learning? Count waste displaced into backlog, retries, review, and maintenance as part of the response's cost.

9.3 Apply the map to the three implementations

A establishes the local single-agent baseline, B rebuilds the same required behavior with a comparison-model single agent, and C introduces fleet orchestration. The common brief and protocol make their differences inspectable. They do not isolate every cause: the team learns between builds, and architectures, tools, hardware, optional scope, or reporting may differ. Record those differences when interpreting a result.

9.3.1 Build the evidence matrix

The matrix belongs in the **final decision** bundle of the four-part Compare submission. For each dimension, link evidence from implementations A, B, and C—or explicitly state `not measured`—before answering the comparison question. Use the earlier phase records and the implementation, fleet, and scaling/deployment bundles as sources; the matrix is an index and interpretation of that evidence.

Table 9.2. Evidence matrix for synthesizing the three implementations through the three feedback loops.

Evidence dimension	Collect from each implementation	Compare across implementations
Accepted browser behavior	Acceptance results, escaped defects, and correction evidence	Which differences affected users rather than only repository structure?
Architecture and contracts	Request trace and key boundary decisions	Which boundaries made independent change safer or harder?
Pressure and saturation	Pressure map and first predicted or observed limit	How did each architecture respond to the same growth scenario?
Measurement and recovery	Benchmark, operational signals, and recovery result	What was directly observed, and what remained invisible?
Development friction	Cross-team audit and change effort	Which setup, test, or coupling cost would dominate the next change?
Human–agent workflow	Execution trace, interventions, and review effort; include handoffs and backlog for the fleet	Which allocation produced accepted work at a sustainable correction cost?
Resource use and externalities	Runtime and model-use evidence	Did added resources create useful capacity or quality, or merely more activity?

Do not turn absent evidence into a favorable score. If a build was not load-tested, state that its saturation behavior is proposed rather than observed. If model usage was not captured, do not infer efficiency from elapsed time. If implementation C produced more branches but required more review and correction, count both the activity and the acceptance cost.

9.3.2 Separate observation, inference, and proposal

A strong synthesis labels the status of each important claim:

- **Observed:** directly inspected or measured in a repository, running application, audit, test, trace, or recorded workflow.
- **Inferred:** a plausible explanation for an observation, with alternatives and confounding factors kept visible.
- **Proposed:** a future response or target design that was not implemented and validated in the compared build.

For example, “implementation B had the lowest measured p95 under the declared workload” is an observation. “Its data model caused the difference” is an inference unless the comparison isolates that factor. “A queue would protect its write path” is a proposal until the new behavior and trade-offs are tested. This vocabulary lets you reason beyond the available experiment without presenting plans as results. The three builds are engineering case studies, with design, experience, and tool conditions changing together. Keep their observations separate from the bounded paired workflow task, which holds a frozen revision and declared conditions stable for a narrower exploratory contrast.

> **Case — Faster libraries and uncertain attribution**
>
> Lemire (2026) reports rebuilding revisions of six libraries and measuring workload-specific runtime gains on one Intel Xeon Gold 6548N machine. For the ada URL parser, he reports throughput rising from 0.54 to 1.28 GB/s on a corpus of 100,000 URLs. He also states that he cannot determine how much AI contributed to every change and discloses an advisory relationship with one contributing tool provider.
>
> The reported measurements support a claim about those revisions, workloads, and hardware. Attributing the gains to AI requires a comparison the account does not provide, and the parser's gain does not establish an application's end-to-end speedup. What additional evidence would you need to compare development effort or isolate AI's contribution? Keep the useful runtime result and the uncertainty about its cause visible together.

Pause & think

Classify evidence and next steps

Use observation for a reported measurement, inference for an explanation drawn from evidence, and proposal for an action to try. Classify each statement by its role; you are not being asked whether the explanation is true.

▼ **Show worked answer**

Measurements describe what was recorded. A causal explanation goes beyond that observation. A proposed experiment states what to do next and can test the explanation.

9.4 Turn comparison into a decision

Use the final comparison structure to turn the matrix into one argument: establish the comparison, explain consequential results, bound the explanation, and choose with a revisit condition. A useful decision connects a constraint to evidence and an owner:

> For **[goal and constraints]**, choose **[architecture and workflow]** because **[observed result and its limits]**. Reject **[alternative]** because **[cost or unsupported assumption]**. **[Owner]** accepts the operational and correction obligations; revisit the choice when **[signal or pressure]**

> changes.

In the queue example, faster HTTP acknowledgment alone would not justify the decision. The argument must also account for event-visibility delay, recovery behavior, and the effort to maintain and review the new worker path. Use the same evidence to explain both the benefit and its price.

9.4.1 Field note — Familiarity is part of effective capability

GPT-family models' communication style is easier for me to follow than Claude's default prose. That personal fit affects how quickly I can review and correct their work; it is not a general model ranking. A model's nominal capability can be less useful if I must first translate its explanations into terms I understand. Changing models therefore changes both the tool and the reviewer's fluency with it, which belongs among the comparison's alternative explanations.

9.4.2 Final reflection package builder

The builder uses the same four bundles as the specification. Mark the bundles you have assembled and replace the outline prompts with links to your evidence. The selections help organize the submission; they do not verify its completeness or the strength of its claims.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/chapters/recap/

9.5 Project artifact

Complete the evidence matrix and final decision narrative within the Compare submission. Link the other three bundles and earlier phase records, mark missing evidence and remaining risks, and defend the next decision.

9.6 Author perspective — Learn where to look

I do not expect you to become an expert in every topic covered here. I want the book to help you recognize which questions deserve attention and where to learn more. An agent can produce a plausible implementation quickly, but you still need to notice the product, architecture, accessibility, security, operational, and maintenance concerns it leaves unresolved.

Like in art, part of the skill is knowing where to look. Sustainability is easy to ignore if it is absent from your mental map. Recognizing it as a concern tells you when to ask a better question, gather evidence, or seek expertise.

Developing this book has also exposed gaps in my understanding. Learning how to question claims, compare evidence, and revise a judgment is a skill I expect to keep practicing as tools and assumptions change.

9.7 Summary

An engineering decision connects the pressure a system can handle with the team's ability to change and operate it. Defend that decision with the three implementations' evidence, explain its limits, and leave the next team a signal and correction path they can use.

9.8 References

1. Lemire, D. (2026, September). A Summer of AI Optimization. Daniel Lemire’s blog. https://lemire.me/blog/2026/09/22/a-summer-of-ai-optimization/

APPENDICES

Appendices

The main chapters provide the shortest path through the book and the course project. The appendices preserve deeper methods, implementation details, and research cases that are useful when a particular problem calls for them but are not prerequisites for following the main argument.

Use them selectively:

- **Evaluating agentic workflows** develops the evaluation vocabulary, experimental discipline, graders, uncertainty, and routing questions introduced briefly in the Benchmarking chapter. It also provides an optional comparison of assistance modes for retained competence.
- **Reproducible environments** distinguishes repeatable setup, declared environments, hermetic execution, and reproducible builds, and provides optional browser measurement procedures.
- **Advanced web boundaries** examines state continuity, transactions, replicated state and the CAP trade-off, client activation, constructive invariants, runtime-generated interfaces, interaction styles, executable HTTP contracts, event-view rebuilding, and endpoint/cache policy.
- **Agent-system security** develops control planes, enforcement, traceability, imported capabilities, cross-run state, contribution-policy examples, and optional exercises on memory, contract completeness, and human review gates.
- **Scalability models** gives a deeper treatment of workflow reliability and contention models.
- **Durable workflows** examines retry-safe external effects and recovery across process failure.

Each appendix states the question it helps answer and links its method back to the relevant chapter or project decision. The case studies provide further optional readings, including Temper's approach to executable behavioral contracts.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

APPENDIX A

Evaluating agentic development workflows

Created
August 27, 2026
Updated
October 5, 2026

Use this appendix to design, run, or critique an agentic-development evaluation. The Benchmarking chapter defines the course baseline.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

A.1 Apply the method selectively

Begin with representative tasks, checked outcomes, preserved failures, and a declared workflow configuration. Read evaluation anatomy, task selection, and grader design for that minimal path. Add other sections when the decision requires them:

- Retained competence for comparing assistance modes when learning is part of the task's purpose.
- Uncertainty for repeated, multi-task comparisons.
- Served configuration for model, environment, and context-reduction controls.
- Cost denominators, routing, and confidence checks for efficiency decisions.
- Compressed evidence and repeated review for review workflows.
- Research examples on judges and coordination for interpreting published comparisons.

A.2 Anatomy of an agent evaluation

A software test usually invokes known code with controlled inputs and checks the result. An agent evaluation must also define the situation in which a model acts, because tool access, instructions, the starting repository, resource limits, and intermediate state can all change the result. The following vocabulary helps keep those layers separate (Grace et al., 2026): The table below locates each term within one evaluation.

Table A.1. Vocabulary for the nested objects and responsibilities in an agent evaluation.

Term	Unit or scope	What it records or does	Common confusion
Task	One declared problem	Defines inputs and success criteria	A repeated attempt is not a new independent task
Trial	One attempt at one task	Runs the task once so repeated trials can expose behavioral variation	Trial count is not task diversity
Outcome	End state of one trial	Preserves the resulting patch, database state, generated artifact, or other effect	The final claim is not necessarily the actual outcome

Term	Unit or scope	What it records or does	Common confusion
Trace or transcript	Path through one trial	Records outputs, tool calls, intermediate results, and relevant interventions	A plausible trace does not prove a correct outcome
Grader	One evaluated property	Checks an aspect of the outcome or trace through code, a model-based rubric, or human judgment	One grader is not the whole evaluation
Agent harness	Model-and-tool execution loop	Supplies context, tools, state, and control for one agent run	It need not provision or score the evaluation
Evaluation harness	Evaluation infrastructure	Provisions tasks and environments, invokes the agent harness, preserves evidence, applies graders, and aggregates results	It is distinct from the agent being evaluated
Evaluation suite	A declared set of tasks	Groups tasks intended to measure a capability or behavior	A shared benchmark is one kind of suite

These terms describe different measurement objects. A repository test is often one deterministic grader inside an agent evaluation rather than the entire evaluation. A public benchmark is a shared evaluation suite designed for comparison across systems, while a local product or project evaluation should represent the work and failure modes that matter in its own setting. An RL environment additionally exposes a reward signal for optimization or training. Treating these as synonyms hides differences in task selection, incentives, and supported claims.

Prefer grading the outcome when several valid paths can solve the task. Use the trace to diagnose failures, account for cost and intervention, and check process requirements that genuinely matter, such as permission compliance or a mandatory approval. Rigidly requiring one anticipated sequence of tool calls can reject a valid solution, while accepting only the final statement can miss that the claimed change never occurred.

A.3 Capability and regression evals

A **capability eval** asks whether the workflow can perform work that is currently difficult. Its initial pass rate may be low because it is intended to reveal a boundary and provide room for improvement. A **regression eval** asks whether the workflow still handles previously accepted work and should normally pass almost all trials (Grace et al., 2026). When a capability task becomes reliably solvable, preserve it as a regression task rather than discarding the evidence and starting over.

Start with tasks already checked manually, consequential failures, and representative project changes. For each task, keep a reference solution or known passing run to show that the stated problem is solvable and the graders can accept at least one correct outcome. Everything required by a grader should be justified by the task brief or a declared invariant; an agent should not fail because the evaluator silently assumed a filepath, output format, or environmental resource. Balance positive and negative cases so optimizing one behavior does not produce its opposite failure elsewhere.

A.3.1 Evaluate the cost of the next change

An initial pass does not show how easily an implementation can be extended. SlopCodeBench addresses this by asking agents to extend their own solutions through successive specification checkpoints, leaving internal structure open and measuring structural erosion and redundant code alongside functional

correctness (Orlanski et al., 2026). Preserving the implementation lets earlier decisions affect later work. Its structural measures are diagnostic proxies; human comprehension and maintenance effort still need their own evidence.

Consider two implementations, X and Y, of a small parcel-quotation tool. Each receives the same three changes in the same order:

1. Quote one domestic parcel using a fixed rate and reject invalid weights.
2. Accept a batch, preserving input order and reporting invalid parcels individually.
3. Accept configurable rates while preserving the original rate when configuration is omitted.

The following numbers are **invented teaching data**, not SlopCodeBench results or measurements from a course project. Suppose the fictional trace shows X duplicating rate logic in its batch path, while Y reuses the single-parcel calculation. At checkpoint 3, X accidentally removes the default from the batch path, breaking a previously passing check. Both implementations eventually pass all current and retained acceptance checks after any repairs.

The table below records what that final pass alone would hide. Each paired cell gives **X / Y**; times are active human minutes for that checkpoint, with review and repair recorded separately. Agent spend includes every attempt and retry through acceptance, including calls made during repairs, but excludes human time.

Table A.2. Invented checkpoint evidence for two implementations that eventually pass the same acceptance checks.

Checkpoint	Regressions found before repair	Review minutes	Repair minutes	Agent spend (USD)
1 — Single quote	0 / 0	6 / 6	0 / 0	0.40 / 0.40
2 — Batch quotes	0 / 0	12 / 8	0 / 0	0.80 / 0.70
3 — Configurable rates	1 / 0	24 / 9	18 / 0	2.40 / 0.90

Both implementations finish three checkpoints, but X consumes 60 human minutes of review and repair and USD 3.60 in agent spend; Y consumes 23 minutes and USD 2.00. The fictional trace suggests where to investigate the difference: a duplicated rule made a later change harder to propagate. A real comparison would still need to distinguish architecture from task difficulty, reviewer familiarity, model variation, and other changed conditions. Fewer lines or duplicated blocks would not establish the explanation by themselves.

Use the task and grader controls with a frozen change sequence, served configuration, per-checkpoint budgets, and stopping rules. Retain revisions, checks, pre-repair failures, acceptance decisions, human effort, and all-attempt cost, including unfinished trajectories. Checkpoints depend on earlier work; repeat fresh trajectories to study variation rather than counting checkpoints as independent trials.

This optional design studies successive changes within one implementation. The course's A–B–C builds start fresh, while cross-team probes examine another group's ability to understand and change each build.

A.4 Evaluate assistance and retained competence

Use this optional comparison when learning or retaining a skill is part of the task's purpose. The Scalable Development chapter distinguishes accepted output, retained competence, and valued participation. Evaluate these outcomes separately: a behavioral check cannot establish what the person can do unaided or which activities they want to retain.

In a personal essay, Stewart (2026) describes missing the investigation and decisions involved in implementation even when the result passes his checks. Mollick (2026) offers a complementary perspective: human knowledge, taste, and initiative can turn agent capability into projects a person would otherwise struggle to attempt. These accounts motivate choices about delegation; they do not establish a universal effect on skill or well-being.

A.4.1 Optional exercise — Write with an agent as navigator

The Navigator Agent pattern assigns implementation to the human while the agent provides guidance. It is a practitioner proposal, not evidence that typing suggested code preserves understanding. Use it to explore which assistance helps you retain a skill you need to own.

1. Choose two small, comparable tasks involving an unfamiliar part of the stack, such as parsing an API response or enforcing a lifecycle transition. State the required behavior and give each task the same time budget.
2. For one task, write the code yourself while the agent offers hints, explanations, and critique. Predict the next step before requesting a hint, and record where guidance changed your reasoning.
3. For the other task, let the agent implement the change and use the remaining time to review and correct it. Apply the same kinds of acceptance checks to both results.
4. In a later session, without the agent or transcript, explain each implementation's invariant and repair a small, previously unseen fault. Record correctness, explanation quality, time, and assistance needed during the original work.

Compare accepted output with later unaided performance; code volume and keystrokes measure neither understanding nor retained judgment. Task differences and practice order can explain a result from one pair, so use this as a learning diagnostic rather than a causal study.

As an optional reflection in your learning diary or discussion, name an activity you wanted to retain, whether the assistance mode preserved it, and what mode you would choose next time. Your preference is separate from the behavioral checks and later unaided performance.

A.5 The eval suite is also software

Terminal-Bench provides a concrete example of a versioned agent benchmark whose tasks combine instructions, container environments, resource limits, reference solutions, and executable tests. Terminal-Bench 2.1 changed 26 tasks from the earlier suite to repair bugs, adjust timeouts or resources, and improve robustness against reward hacking (Terminal-Bench Contributors, 2026). Its `fix-code-`

`vulnerability` task, for example, pins a Bottle source revision and Python image, specifies CPU, memory, storage, and time limits, requires both a patch and a structured report, and checks the result with original and security-focused tests.

The measurement instrument can be wrong. Ambiguous instructions, incomplete graders, leaked state, shared caches, resource contention, and permissive reference matching can create failures or passes unrelated to the intended capability. Record the suite version and environment with every result, validate a reference solution, start trials from clean state, and read selected traces before trusting an aggregate score. When a task or grader changes materially, do not silently compare the new score with results from the previous instrument.

For this course, inspecting one packaged benchmark task is more useful than running the entire public suite. Identify its claimed capability, starting state, visible instructions, hidden or external assumptions, graders, reference solution, resource boundary, and plausible gaming paths. Then state what a passing result would and would not establish.

> **Advanced option: derive candidate tasks from repository history**
>
> Repo2RLEnv can mine merged pull requests or commits, reconstruct repository states in Docker, and emit Harbor-compatible tasks whose primary runtime signal comes from the repository's own tests (Hugging Face, 2026). Its runtime pipelines use the familiar fail-to-pass and pass-to-pass shape: tests associated with the change should begin failing and pass under the reference patch, while previously passing behavior should remain intact. This can help an instructor or a mature project turn reviewed changes into candidate evaluation data, but it is not a course baseline and does not make every repository an informative RL environment. History records changes that happened, not a representative sample of future work; tests can omit intended behavior; generated instructions can leak or distort the problem; and similarity to an oracle diff can penalize a different valid patch. An executable reward is verifiable under its grader, not automatically complete ground truth.

Some outcomes can be checked deterministically through tests, schemas, or expected artifacts. Others require a rubric or human judgment. When a model acts as a judge, test whether answer order, verbosity, confidence, or the judge's own style changes its scores; LLM judges have documented position and self-enhancement biases (Zheng et al., 2023). Keep a small human-reviewed calibration set and report disagreement rather than treating the judge as ground truth.

A separate research example examines what happens when a judge supplies both training rewards and the headline evaluation.

The demonstration below separates deterministic probes from a rubric score. Use it to inspect a failing task and decide whether the problem lies in the result, the rubric, or the evaluation setup.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/evaluating-agentic-workflows/

Modern Web Guidance provides a concrete example of evaluating an imported agent capability. Its evaluation harness first checks browser assertions against both a reference implementation and a deliberately flawed implementation, then compares agent runs with and without the guidance. A project can reproduce the smaller experimental shape: hold the task, model, starting repository, and acceptance checks constant; change whether the capability is available; and compare correctness, retries, cost, and human correction. The resulting difference measures performance on that task and those checks, not whether the guidance makes every application better in production.

Literal task completion is not enough for an agent that can take actions. The proposed "Genie coefficient" asks how far an agent's behavior departs from a reasonable interpretation of the user's intent (Raghavan & Schneier, 2026). It is a conceptual framework rather than a validated metric, but it suggests a useful evaluation split: score task success, permission compliance, unnecessary actions, side effects, and clarification quality separately.

For the course project, evaluate one representative AI-assisted task twice: once for the correctness of the produced artifact and once for the quality of the workflow that produced it. Preserve the prompt or task brief, relevant context, model and tool versions, execution trace, checks, human corrections, and final acceptance decision.

A.6 Report uncertainty, not only point estimates

An evaluation observes a finite set of tasks under declared conditions. A difference in average score or acceptance rate is therefore an estimate, not a stable ranking of two models or workflows. In a systematic review of 445 new or substantially modified LLM benchmark papers, only 16.0% used uncertainty estimates or statistical tests to compare results (Bean et al., 2025). Earlier work on NLP evaluations likewise found that small test sets can lack the power to distinguish plausible improvements and that effects detected by underpowered studies can be exaggerated (Card et al., 2020).

When possible, run both candidates on the same task IDs and predeclare how repeated calls become one task-level result. Repeated calls reveal variation within a task; they do not turn one task into many independent tasks. If many tasks are variants of one template, repository, or issue family, disclose that dependence and narrow the claim.

When a protocol permits `k` trials per task, name the reliability question before selecting a summary. **Pass@k** asks whether at least one of the `k` trials succeeds and is useful when several attempts are genuinely available and one accepted solution is enough. **Pass^k** asks whether all `k` trials succeed and is useful when users need the behavior to be dependable every time (Grace et al., 2026). More attempts make pass@k easier to satisfy and pass^k harder, so the same workflow can look increasingly capable by one measure and increasingly unreliable by the other. Predeclare `k`, budgets, stopping rules, and whether failed attempts remain in latency and cost totals; do not use pass@k to imply first-attempt reliability.

Report:

- the independent task count and task source;
- one predeclared primary outcome;

- the observed paired difference;
- a suitable paired interval and its method, when a course-provided or otherwise justified method fits the outcome.

Before inspecting results, define the signed difference and the practical boundary or bounds that would change the decision. One conservative, predeclared rule is to require a suitable interval wholly beyond the practical boundary before claiming the required effect. Under that rule, overlap leaves the effect claim inconclusive, while statistical separation can still describe a difference too small to matter. An interval containing zero is not proof that the candidates are equal.

An engineering action also depends on consequences, cost, and reversibility. A cheap reversible trial with monitoring and a rollback condition may be justified while the effect remains uncertain; an expensive irreversible migration may warrant more evidence. State the decision rule and its rationale separately from the claim about the measured effect.

Treat other metrics and unplanned slices as descriptive or exploratory unless their joint decision rule and multiple-comparison treatment were declared in advance. When no defensible interval is available, show the raw paired outcomes, label the comparison exploratory, and restrict the claim to those tasks. An interval describes uncertainty conditional on the sample and scoring process; it cannot repair unrepresentative tasks, judge bias or contamination, or account for a change to the agent or evaluation harness.

Paired comparison check. For one predeclared binary acceptance result per independent task, count tasks accepted by both candidates, only the first, only the second, or neither. Use an instructor-supplied interval or a justified paired method to interpret those counts against the predeclared decision boundary; do not compare two separate error bars. The single representative project task above remains a case comparison; use this check with a multi-task project cohort or instructor-provided data.

A.7 Qualify the served configuration

A model name or local endpoint is not a capability result. Bind an accepted outcome to the configuration that served it: resolved model revision or digest and quantization, runtime version, allocated context, prompt and tool scaffold, and relevant hardware and accelerator residency or CPU/GPU offload where exposed. When an approved service does not expose a field, mark it unavailable and name the evidence source or service operator rather than estimating it.

In a study of 9.7K test examples across long-input and long-form-output benchmarks, quantization effects varied by method, model, task, context length, and language (Mekala et al., 2025). The study used vLLM on datacenter GPUs and did not test coding agents, Ollama, or consumer-device deployments, so one quantized build should not inherit another build's score. Hardware constrains which configurations fit and their latency, throughput, and resource use; it does not establish task capability.

Keep capability, hardware fit, and data location as separate claims. Running inference locally identifies where model computation occurs, not the data boundary of tools, telemetry, storage, or network calls. A fallback can select a different model or configuration and change the capability, latency, cost, and data

boundary. Ollama, for example, exposes an installed model's digest and quantization and a running model's allocated context and server-reported model VRAM bytes. It can also forward cloud-model requests while callers continue using its local API, so `localhost` alone does not prove local execution (cloud models).

Assign a stable identifier to each served configuration in the evaluation. Record its exposed model, runtime, context, scaffold, tools, hardware boundary, outbound paths, and fallback policy, then link every attempt and acceptance result to that identifier. For an opaque departmental service, use operator- or instructor-supplied metadata; otherwise mark unavailable fields and name the evidence source. Qualify the configuration against a predeclared representative task and deterministic check, and treat the result as evidence for that task rather than a general ranking of local and hosted models.

Context selection and output reduction are also configuration decisions. Long tool outputs and interaction histories consume context and can increase latency or cost, which has motivated command-aware filters such as rtk and model-generated summaries. A shorter agent-facing view does not by itself establish a lower total bill or an unchanged task outcome. In a SWE-agent workshop study, observation masking and model-generated summaries reduced cost in most of five tested model configurations, but its paired analysis found that both significantly lowered solve rate in one reasoning-model configuration (Lindenbauer et al., 2025). On controlled long-horizon tool-use benchmarks, ACON reduced peak input tokens while largely retaining task performance (Kang et al., 2026). Neither study validates every command filter; together they show that context reduction is a behavior change to test, not free efficiency.

> **Compact the working view; preserve the raw record**
>
> Treat compressed output as a working view, not as the evidence record. Keep raw output retrievable during the run and preserve the producer's exit status independently. For bounded qualification cases and consequential failures, retain the original command or query, working directory, separate output streams, pagination or continuation metadata, and explicit omission markers under a stable handle. Apply the underlying system's data-minimization, secret-handling, residency, access, and deletion rules; disable archival where those rules cannot be met. Prefer native structured output as a comparison baseline, make capture failures visible, and treat retrieved output as untrusted text rather than embedded instructions.
>
> Qualify each important compression policy with the same task cohort and acceptance checks used for the uncompressed view. Include compression calls, raw retrievals, retries, and failed attempts when reporting total cost per accepted outcome. A reduction in shell-output bytes is only a component metric.

Tool-output preflight. Use instructor-supplied or synthetic non-sensitive raw artifacts for one passing command and one deliberately tricky output: an exit-zero warning, a failing diagnostic, or an unusual line in a repetitive log. Production and secret-bearing output are outside this exercise. If the policy claims to handle pagination, add one paginated artifact.

Before generating compact views, list the critical facts in each raw artifact and require every compact fixture to mark at least one explicit omission. Record the command and tool version, reduction-policy version, configuration, and content hash. Verify that source status, separate output-stream provenance, critical facts, omission markers, retrieval handle, and pagination metadata survive. Report critical facts retained over critical facts present, false or materially misleading facts, and any policy-safe fallback. Passing this preflight can reject an obviously lossy or unsafe policy; claims about cost or task effectiveness require paired end-to-end outcomes.

A.8 Read a harness benchmark's cost denominator

FrontierHarness v1.0 compares 12 configurations across 30 software-development and terminal tasks, using one Kimi K3 model and one attempt per task–configuration cell (Zhu & Mei, 2026). Its authors describe interactions among the harness, model, gateway, and caching behavior, and explicitly decline to attribute the cost gap to the harness alone. Use it as a benchmark-reading exercise: identify the tested configuration and cost population before interpreting a ranking. A single attempt per cell does not measure repeated-run reliability.

The following numbers are invented teaching data, not FrontierHarness results. Suppose four distinct tasks each receive one attempt: two accepted attempts cost $1 and $3, while two failed attempts cost $8 and $12. The total measured execution spend is $24, the acceptance rate is 2/4, and there are no retries or missing usage records. The table below derives three different summaries from this same cohort.

Table A.3. Different cost populations produce different answers from the same invented task cohort.

Measure	Calculation	Question answered
Median successful-attempt cost	Median of $1 and $3 = $2	What did a typical successful attempt cost?
Mean attempted-task cost	$24 / 4 attempted tasks = $6	What did an attempted task cost on average, including failures?
Total spend per accepted task	$24 / 2 accepted tasks = $12	What did all attempted work cost per accepted task?

Calling all three numbers "cost per task" hides the distinction. If retries are allowed, include their spend while counting each accepted task once; label missing costs rather than treating them as zero. Report human review and other unpriced resources separately when they are outside the measured execution spend.

For the published benchmark, also distinguish the median cache-hit rate across runs from the fraction of all input tokens served from cache. A few long runs can dominate token-weighted usage, and a high cache-hit rate can coexist with expensive failures. Before transferring a ranking to your project, test representative tasks with repeated fresh trials and the model, gateway, and cache conditions you intend to use.

A.9 From evaluation to routing

A lower per-request price does not necessarily make a model cheaper for a task. An inexpensive attempt that fails may have to be repeated with a model that succeeds more often on that task, while still consuming context, validation, and review time. Conversely, always selecting the strongest available

model can waste capacity when a smaller model can reliably clear the same acceptance bar. The useful comparison is therefore cost per accepted outcome, not token price or first-attempt cost.

Research distinguishes **routing**, which selects one model before it produces an answer, from **cascading**, which begins with one model and escalates when evidence indicates that its answer is insufficient. Both depend on an accurate quality estimate; Dekoninck et al. (2025) identify estimator quality as a critical condition for improving the cost-performance trade-off. In their SWE-Bench experiment, the post-generation estimator could use the task's ground-truth test outcomes. The result therefore shows what a strong verifier can enable, not how well the approach works for tasks without one. Databricks presents routing as an emerging coding-agent cost-control technique and reports an internal benchmark result, but publishes too little about that router evaluation to predict savings in another codebase (Wendell et al., 2026). This motivates a local experiment, not a promised saving.

When access rules permit both models—or when instructor-provided outputs are available—choose two models with different measured cost and task-success profiles within the same declared agent harness and assemble a small set of representative tasks. Keep near-duplicates and tasks from the same module in the same partition, using one partition to define the policy and reserving the other for held-out evaluation.

Define two evidence layers before running the comparison. The **policy-visible gate** may use deployable signals such as public tests, builds, or static checks to decide whether a cascade should escalate. The **final adjudicator** uses held-out tests or blinded human review that the policy cannot see; only this layer decides whether the final outcome is accepted. A policy that stops after passing its visible gate but fails final adjudication is a false acceptance.

On the first partition, define and freeze either a routing rule that uses only facts available before generation or a cheap-first cascade that uses the policy-visible gate after its first result. Freeze the task features, thresholds, prompts, versions of the model and agent harness, budgets, retry rules, and exclusion rules before inspecting the held-out results. Compare the policy with always using either model and, when the task set permits it, random selection at the same stronger-model call rate. The random baseline tests whether the policy selects useful cases rather than benefiting only from its mixture of models.

Report accepted outcomes over all attempted tasks, stronger-model calls, end-to-end latency, human correction, and total cost per accepted outcome. Count timeouts, tool failures, missing patches, and budget exits as attempted failures. Include policy decisions, quality estimation, rejected attempts, fallbacks, repeated context, validation, review, and rework in the cost; report unpriced resources separately. If both model arms are run to reveal counterfactual mistakes, keep that one-time evaluation expense separate from each policy's simulated operating cost. That comparison can expose false acceptances and unnecessary escalations; without both arms, label the missing counterfactuals as unknown rather than inferring them from an average.

The result supports a local decision only if the policy meets a predeclared quality and risk threshold while improving the chosen cost or latency measure. After using held-out results to change the policy, evaluate again on fresh tasks. High-risk work can bypass the router or retain mandatory review, and the policy should be recalibrated when models, prices, agent harnesses, or the task distribution change.

A.9.1 Check confidence before routing

A closed choice set can simplify output validation. For example, Clef's model card describes scoring schema-defined options and returning a probability for each. An allowed label can still be wrong, and the correct outcome may be missing from the choice set.

Confidence calibration asks whether reported probabilities agree with the frequency of correct predictions over relevant cases (Guo et al., 2017). A score of `0.9` needs evidence from comparable predictions before it can guide a routing threshold. Confidence about a classification also concerns a different outcome from whether the resulting patch passes acceptance or an action is authorized. Keep mandatory checks and permissions outside the learned score.

To test this distinction, give your agent the confidence starting point, or examine the invented classification batch yourself.

A.10 Compressed evidence and repeated review

Use compact views to direct attention and repeated reviews to search for missed defects. Keep a common acceptance protocol:

1. Preserve the full artifact and independent checks, and inspect omitted material at consequential decisions.
2. Confirm each finding through a reproduction, reachable path, or violated invariant before judging its impact.
3. Record triage and repair effort, and look for a common cause when several findings recur.
4. Stop against a declared risk threshold; decompose a change that remains too large or coupled to understand.

The cases below show why these controls matter.

> **A reading diff routes attention**
>
> Large changes may repeat the same migration across call sites or include generated and mechanically forced edits. `meat` is an experimental tool that asks a model to choose rows to remove, fold, or elide, then mechanically checks and applies that plan to the immutable original diff. The surviving code is therefore source-derived rather than regenerated by the model.
>
> The view is still lossy. An author-run analysis of the Python-only diffs from three selected commits records an aggregate 30.1% byte reduction; it also documents cases where the output hid decisive test outcomes and made an old/new default inversion look like a one-sided addition. The report illustrates the mechanism and its failure modes; it does not measure reviewer effectiveness. In a different task, a mutation-based preprint found that GPT-4 and GPT-5.2 summaries of 12 controlled synthetic Python programs and 50 human-written benchmark programs sometimes described familiar algorithmic intent while missing mutated behavior; it did not study diffs or human reviewers (Khatib et al., 2026).

> **When findings become inventory**

> Continued review creates reports that still require verification and a repair decision. In a live study at Mozilla and Ubisoft, reviewers directly accepted 8.1% and 7.2% of generated comments, respectively, while another 14.6% and 20.5% were marked useful as review or development tips. At Ubisoft, the median instrumented interval for evaluating generated comments was about 43 seconds per patch, an upper bound on active evaluation time. The study measured adoption rather than defect-detection accuracy, but it shows that suggestions still consume attention (Olewicki et al., 2026).

A.10.1 Review-evidence exercise

Choose one bounded representative change. Before opening a compact or agent-produced review view, predeclare two or three behavior or architecture questions. Record provisional answers after that view, then inspect the complete artifact and relevant checks and record every correction, omitted or misleading critical detail, and the time spent in each stage. For agent findings, record candidate, independently confirmed, duplicate, and rejected findings together with triage and repair time. Apply the acceptance protocol above when deciding which findings to repair and when to stop.

A.10.2 Optional exercise — Challenge an explanation

Predict what a small event recorder does when a worker restarts, then check an agent's explanation against the code. Give your agent the explanation starting point, or work through the fixture yourself.

A.11 Repeated review is measurement, not proof

Repeating an agent review can reveal variance, but agreement between runs is not independent validation. VulnBench repeated identical security-review tasks five times across ten small JavaScript and Express fixtures. Of 158 unique findings that matched its Snyk Code reference, 134 appeared in every repetition; among 161 unmatched findings, 80 appeared only once and 22 appeared in every repetition (Tal et al., 2026). The recurrence difference is useful for triage, but reference overlap measures agreement with Snyk Code rather than ground-truth accuracy.

Treat each model review as one sample. Deduplicate repeated findings, record recurrence separately from deterministic-reference overlap, and preserve rare findings for case-level adjudication rather than automatically discarding them. Consensus can repeat a shared blind spot, while a low-frequency report can still identify a real defect. Behavioral tests, invariants, deterministic analysis, and accountable review remain the acceptance evidence.

Repeated review also consumes the same scarce integration capacity that bounds the fleet. If you repeat an agent review, record the fixed code, prompt, agent harness, and model configuration; the number of runs; unique and recurring findings; reviewer time; and cost per accepted finding. More review samples are justified only when they improve accepted outcomes or allocate human attention better than a single review.

A.12 Research example: evaluating the evaluator

Falck et al. (2026) offer a useful case in which evaluation becomes supervision. Their Replica setup trains a 27B agent on 242 figure-replication tasks using auto-generated per-task rubrics, three judge samples, and turn-level credit assignment, then scores it on 68 held-out AI-for-science tasks. The rubric judge was more repeatable than a generic judge in the paper's repeated-sampling analysis. In a blinded study of ten training tasks selected because the judges disagreed, its Kendall rank correlation with expert rankings was 0.19 rather than 0.15 for the generic judge; on disputed pairs, experts sided with the rubric judge 63% of the time, a difference that was not statistically significant (`p = 0.109`). Because the same judge design supplied training rewards and the headline evaluation, the task holdout did not make the scoring process independent. The evidence supports a bounded claim about a lower-noise training signal and improved scores under this judge, not that the judge is ground truth or that the system performs open-ended scientific innovation.

A.13 Research example: coordination and integration

The experiments below separate three properties that are easy to collapse: concurrent writing, coordination, and accepted behavior. CodeCRDT used a shared CRDT document and an observation-driven claim protocol so agents could edit concurrently without locks. Across 600 evaluations of six small TypeScript and React tasks with Claude Sonnet 4.5, every run converged without a textual merge failure, yet preliminary inspection still found semantic conflicts in roughly 5–10% of results. Raw completion time ranged from a 21.1% speedup to a 39.4% slowdown depending on the task (Pugachev, 2025). The coordination substrate solved document convergence; it did not make independently reasonable changes semantically compatible or guarantee useful parallelism.

AgentRoom evaluated a separate coordination design that paired a CRDT-backed shared filesystem with explicit file claims, agent status, and message broadcasts (Cho & Lee, 2026). For the CLI-stable models in its evaluation, two-agent rooms reduced a specific one-file abandonment mode and lowered run-to-run variation relative to solo execution. At matched compute on one task, its primary LLM-judge composite favored AgentRoom over independent parallel generation followed by post-hoc file union. A separate seven-run bundle probe ordered the full system above shared-only and prompt-only partial conditions, but the interval for the coordination-layer step included zero. The result remains preliminary. The paper was a recent preprint, its headline tasks shared one Express and TypeScript environment, agents authored their own tests, and the main quality comparison did not use a held-out execution oracle.

Repository-scale evidence shows why this distinction matters outside a controlled shared document. AgenticFlict selected 142,652 open or closed-but-unmerged agent-authored pull requests from the AIDev dataset; deterministic merge simulation succeeded for 107,026, and 27.67% of that restricted sample produced textual conflicts (Ogenrwot & Businge, 2026). The dataset does not isolate fleets or establish that agents caused the conflicts; it measures agent-authored changes colliding with the repository state into which they would be merged. It nevertheless documents a substantial collection of integration failures that a fleet design should not treat as an incidental cleanup concern.

Together, these studies support a bounded conclusion. A shared substrate can prevent some textual collisions, and an explicit coordination interface can reduce some duplicated or abandoned work, but neither replaces a settled contract or end-to-end acceptance evidence. Parallelize only work whose ownership and dependencies are clear, then test the integrated behavior through the web boundary.

A.14 Field note — Let repeated failures shape the first evals

My practical threshold for codifying an eval is when I begin to see a pattern and trust my judgment enough to describe it. I do not yet have enough experience to claim a general method for designing strong eval suites, however. So far, I have mostly allowed development models to propose the evaluators, then complained about obvious weaknesses I found while reviewing their code.

Slideotter provides the clearest example of where this still became useful. The repository preserves the resulting generation fuzz harness. I asked a stronger development model to mutate presentation scenarios and exercise the smaller local models through that harness. The mutations explored combinations and failure cases I would not have devised on my own.

Some outputs failed in surprising ways. Those failures became new regression scenarios, stronger validators, and improvements to the surrounding prompts, schemas, and generation scaffolding. The resulting loop helped me extract more useful behavior from weaker local models.

This was closer to mutation-based probing than an independent evaluation of presentation quality. The development model could introduce its own blind spots, and model-generated evaluators could reward the same assumptions that shaped the implementation. Structural and rendering checks could establish that a presentation was valid under the tested scenarios; they could not establish that it communicated the topic well.

I therefore treat generated evals as a way to discover and preserve recurring failure patterns, not as an automatic source of ground truth. Editorial judgment remains necessary until repeated observations become clear enough to encode and the evaluator itself has been challenged against known successes and failures.

A.15 References

1. Grace, M., Hadfield, J., Olivares, R., & de Jonghe, J. (2026, January). Demystifying Evals for AI Agents. Anthropic Engineering. https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents
2. Terminal-Bench Contributors. (2026). Terminal-Bench 2.1. GitHub repository. https://github.com/harbor-framework/terminal-bench-2-1
3. Hugging Face. (2026). Repo2RLEnv: Convert Any Repository into a Verifiable RL Environment. GitHub repository. https://github.com/huggingface/Repo2RLEnv
4. Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. arXiv Preprint arXiv:2306.05685. https://doi.org/10.48550/arXiv.2306.05685

5. Falck, D., Sabri, S., Surina, A., Foster, T., Sims, A., Devlin, S., Rogers, D., Collins, T., Aleksiev, K., Kirsch, L., & Hughes, E. (2026). Training AI Scientists to Replicate Research. arXiv Preprint arXiv:2608.13331. https://doi.org/10.48550/arXiv.2608.13331
6. Dekoninck, J., Baader, M., & Vechev, M. (2025). A Unified Approach to Routing and Cascading for LLMs. Proceedings of the 42nd International Conference on Machine Learning, Proceedings of Machine Learning Research, 267, 12987–13010. https://proceedings.mlr.press/v267/dekoninck25a.html
7. Wendell, P., Bhatia, A., Gaba, V., Elsen, E., & Zhou, I. (2026, August). Managing AI Coding Costs at Scale. Databricks Blog. https://www.databricks.com/blog/managing-ai-coding-costs-scale
8. Pugachev, S. (2025). CodeCRDT: Observation-Driven Coordination for Multi-Agent LLM Code Generation. arXiv Preprint arXiv:2510.18893. https://doi.org/10.48550/arXiv.2510.18893
9. Cho, S., & Lee, D. (2026). AgentRoom: Concurrent Multi-Agent Coding in a CRDT-Backed Shared Workspace. arXiv Preprint arXiv:2608.23740. https://doi.org/10.48550/arXiv.2608.23740
10. Ogenrwot, D., & Businge, J. (2026). AgenticFlict: A Large-Scale Dataset of Merge Conflicts in AI Coding Agent Pull Requests on GitHub. Proceedings of the 3rd ACM International Conference on AI-Powered Software. https://doi.org/10.1145/3805760.3814923
11. Tal, L., Kloos, J., Rudich, A., Thoemmes, S., & Nair, M. (2026). Snyk VulnBench JS 1.0: Can LLMs Find the Same Bugs Twice? arXiv Preprint arXiv:2606.15762. https://doi.org/10.48550/arXiv.2606.15762
12. Olewicki, D., Da Silva, L. M. P., Ben Sghaier, O., Mujahid, S., Amini, A., Mah, B., Castelluccio, M., Habchi, S., Khomh, F., & Adams, B. (2026). Impact of an LLM-Based Review Assistant in Practice: A Mixed Open-/Closed-Source Case Study. IEEE Transactions on Software Engineering, 1–12. https://doi.org/10.1109/TSE.2026.3663093
13. Zhu, S., & Mei, S. (2026, September). Introducing FrontierHarness Eval. Runta. https://runta.com/blog/introducing-frontierharness-eval/
14. Raghavan, B., & Schneier, B. (2026). Why AI Needs a “Genie Coefficient.” IEEE Spectrum. https://spectrum.ieee.org/ai-agent-benchmark
15. Khatib, L., Pu, M., Vasilescu, B., & Nagappan, M. (2026). Using Mutation-Analysis to Examine an LLM’s Ability to Summarize Code. https://doi.org/10.48550/arXiv.2602.17838
16. Bean, A. M., Kearns, R. O., Romanou, A., Hafner, F. S., Mayne, H., Batzner, J., Foroutan Eghlidi, N., Schmitz, C., Korgul, K., Batra, H., Deb, O., Beharry, E., Emde, C., Foster, T., Gausen, A., Grandury, M., Han, S., Hofmann, V., Ibrahim, L., … Mahdi, A. (2025). Measuring what Matters: Construct Validity in Large Language Model Benchmarks. Advances in Neural Information Processing Systems, 38. https://doi.org/10.52202/085713-0590
17. Card, D., Henderson, P., Khandelwal, U., Jia, R., Mahowald, K., & Jurafsky, D. (2020). With Little Power Comes Great Responsibility. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 9263–9274. https://doi.org/10.18653/v1/2020.emnlp-main.745
18. Kang, M., Chen, W.-N., Han, D., Inan, H. A., Wutschitz, L., Chen, Y., Sim, R., & Rajmohan, S. (2026). ACON: Optimizing Context Compression for Long-horizon LLM Agents. Proceedings of the 43rd International Conference on Machine Learning, Proceedings of Machine Learning Research, 306.

https://arxiv.org/abs/2510.00615

19. Lindenbauer, T., Slinko, I., Felder, L., Bogomolov, E., & Zharov, Y. (2025). The Complexity Trap: Simple Observation Masking Is as Efficient as LLM Summarization for Agent Context Management. Fourth Deep Learning for Code Workshop (DL4C) at NeurIPS 2025. https://doi.org/10.48550/arXiv.2508.21433
20. Mekala, A., Atmakuru, A., Song, Y., Karpinska, M., & Iyyer, M. (2025). Does quantization affect models' performance on long-context tasks? Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 9422–9470. https://doi.org/10.18653/v1/2025.emnlp-main.479
21. Orlanski, G., Roy, D., Yun, A., Shin, C., Gu, A., Ge, A., Adila, D., Roberts, N., Sala, F., & Albarghouthi, A. (2026). SlopCodeBench: Benchmarking How Coding Agents Degrade Over Long-Horizon Iterative Tasks. arXiv preprint arXiv:2603.24755. https://doi.org/10.48550/arXiv.2603.24755
22. Stewart, L. (2026, September). Agentic Alienation. Loren Stewart's blog. https://www.lorenstew.art/blog/agentic-alienation
23. Mollick, E. (2026, September). The Overhang. One Useful Thing. https://www.oneusefulthing.org/p/the-overhang
24. Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On Calibration of Modern Neural Networks. Proceedings of the 34th International Conference on Machine Learning, Proceedings of Machine Learning Research, 70, 1321–1330. https://proceedings.mlr.press/v70/guo17a.html

APPENDIX B

Reproducible development environments

Created
August 27, 2026
Updated
October 5, 2026

Use this appendix when setup failures, hidden host state, inconsistent resource limits, or environmental drift make a project difficult to run or evaluate. The Friction chapter keeps the essential project rule—declare the supported environment and test it from a fresh checkout—while this appendix develops the distinctions and audit procedure in depth. The browser measurement procedures support the measurement principles introduced in the Benchmarking chapter.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

B.1 Treat the environment as a versioned input

A dependency lockfile does not describe the whole development environment. Runtime and package-manager versions, system packages, local services, process startup and readiness, test hooks, browser or command-line tools, and supported operating-system, architecture, and resource assumptions can remain hidden on one workstation. When another contributor, CI worker, or agent sandbox receives the same repository but cannot run its checks, the failure may reveal an implicit input, an incorrect support boundary, or an external or transient dependency.

For an agent, this state is part of the task rather than neutral background. In an internal Anthropic experiment that held the Claude model, agent harness, and Terminal-Bench 2.0 task set constant, reported infrastructure-error rates fell from 5.8% under strict enforcement to 0.5% with uncapped resources across six configurations; above roughly three times the benchmark specification, additional headroom was also associated with solution strategies that could not succeed under tighter limits (Segato, 2026). The vendor-authored study concerns benchmark infrastructure rather than developer onboarding, but it supports recording CPU and memory allocations, hard limits, and enforcement policy; time limits are a separate run condition worth recording.

Keep four related claims separate:

- **Setup automation** provides a bootstrap path, but that path may still be unpinned, stateful, or dependent on the network.
- A **declared environment** records and, where possible, pins the toolchain, dependencies, service definitions, configuration schema, seed-data revision, and supported platform and resource scope needed to reconstruct setup and run the baseline checks.

- **Hermetic execution** isolates an action from undeclared host state and external services and restricts it to declared inputs and controlled tools; running inside a container does not establish this by itself.
- A **reproducible build** means that, given the same source, build environment, and build instructions, independent builders produce bit-for-bit identical copies of the specified artifacts.

devenv is one Nix-based example: it can declare packages, languages, tasks, services, processes, tests, hooks, profiles, and containers, while its lock file records external inputs. It is an implementation example rather than evidence that one tool or Nix fits every project. In a large-scale study of 709,816 package instances from historical Nixpkgs snapshots, more than 99% could be rebuilt successfully, while bitwise reproducibility ranged from 69% to 91% (Malka et al., 2025). That result concerns package builds, not interactive development workflows or cross-platform equivalence, and shows that declared inputs alone do not guarantee bitwise-identical outputs. Native setup scripts, development containers, and other environment managers can support the same project contract.

Declare which secrets are required and how they are supplied, but inject their values at runtime rather than pinning or committing them in the environment definition. Name host kernels, hardware, networks, mutable service data, and external APIs that remain non-reproducible boundaries; prefer local or demonstration substitutes for the baseline and never require production credentials. Mutable downloads still need a lock, checksum, digest, or explicit freshness policy. Strict pins need an update and security-review policy; reproducibility should not turn an old dependency into an invisible permanent dependency.

Use the existing cross-team audit as the test. Run it as follows:

1. Start from a fresh checkout on another supported machine or container. Remove project caches, generated files, and project-owned service state from the starting conditions.
2. Record the host prerequisites already present, then run the documented setup and baseline validation.
3. Capture the setup time and outcome, any undocumented intervention or retry, the failure stage, and relevant host, resource, or network conditions. Record required configuration names, but never secret values.
4. After a repair, have the authors or CI replay the path from a fresh state.

The result supports a bounded claim about the declared setup, not that production is identical or that a stochastic agent will produce the same patch.

B.2 Choose the required strength

Not every project needs hermetic execution or bit-for-bit reproducible builds. Choose the weakest environment contract that still supports the decision: documented setup for ordinary collaboration, a declared and pinned environment for comparable course evidence, isolation for actions that must not depend on host state, or reproducible artifacts when independent builders must verify identical output. State the remaining external boundaries instead of implying that a tool has removed them.

B.2.1 Field note — Make validation local and proportional

I value a development environment that is easy to bootstrap and safe to bootstrap again. Setup should be idempotent: rerunning it should converge on the supported project state or report a clear incompatibility instead of accumulating hidden changes. Without that property, stale dependencies and one-off repairs can eventually leave a project insecure or unable to run.

In my agent-assisted projects, remote CI has usually been too slow for the primary feedback loop: an agent can produce several changes during one cycle. I use local validation during iteration and retain clean, independent CI where the acceptance risk justifies it.

I have tried to encode this approach in Vibe Template, whose affected-guardrail decision documents the focused path and its fallbacks to broader validation. The validation loop has several strengths:

1. **Run focused checks during ordinary iteration.** Select tests, diagnostics, and mutation targets from the affected files where the dependency information is reliable.
2. **Run a broader gate before accepting substantial work.** Use the project's baseline validation when a change crosses boundaries or the affected-file analysis may be incomplete.
3. **Reserve expensive checks for justified cases.** A full mutation run can be too costly for every iteration, so use it for high-risk changes, explicit audits, or scheduled validation.
4. **Retain CI where independence matters.** Larger projects still benefit from a clean remote environment before merge or deployment.
5. **Use containers when isolation earns its cost.** A container-based local run is safer when setup requires difficult system dependencies or could otherwise disturb the developer's machine. Simpler projects may not need that boundary.

Affected validation introduces its own obligation: the selector must recognize configuration changes, deleted files, shared dependencies, and other cases that should widen the check. A fast loop is only useful when it fails conservatively instead of silently excluding relevant work.

Give agents fast, trustworthy local feedback, and widen validation when the change's risk or scope requires it.

B.3 Reproduce browser measurements

Use these optional procedures when a browser journey is part of the project's benchmark. State the journey, workload, environment, metric, and acceptance condition before choosing a tool. Reuse the results in the existing benchmark evidence.

B.3.1 Audit a representative page

Use PageSpeed Insights for the lowest-setup version of the exercise. It runs Lighthouse remotely, so the target page must be deployed at a publicly reachable URL; a development server at `localhost` is not reachable by the service.

1. Choose one representative public route and state why it matters to a user.
2. Select the mobile profile and run the same URL five times without changing the application.

3. Record LCP, TBT, CLS, transferred bytes, and the overall performance score for every run.
4. Report the median and the fastest-to-slowest range for each time-based metric.
5. Select one Lighthouse diagnostic, make one change, deploy it, and repeat the five runs under the same conditions.
6. State the narrow conclusion supported by the comparison and at least one factor it does not control.

The PageSpeed Insights result can include Lighthouse lab data and, for sufficiently popular public pages, field data derived from real Chrome visits. Keep those sources separate: lab and field measurements use different populations and conditions.

If the application is not public, open it in Chrome, select **Developer Tools → Lighthouse**, choose the mobile profile, and generate a report locally. Close unrelated applications where practical, do not run audits concurrently, and keep the browser version, profile, URL, application version, and machine consistent.

Performance values vary even when the application has not changed because the browser, client hardware, resource contention, network, and server introduce noise. Heričko et al. (2021) found that five consecutive Lighthouse runs aggregated with the median greatly reduced variability in their experiment.

B.3.2 Study a repeatable Playwright journey

PageSpeed Insights is convenient for navigation audits, but it cannot exercise a private or multi-step project flow. The standalone Playwright lab separates a small event-filter fixture from the browser journey that exercises it, so you can inspect locators, interaction, assertions, and trace collection without first adapting the example to a framework. Study the source or download the standalone lab.

The example establishes that filtering produces the expected visible row. It does not measure application loading, network behavior, or Web Vitals because it opens a local fixture. After it passes, replace the fixture URL with the project's real route and adapt the accessible names while preserving the same user-visible contract. Inspect the trace to verify what was acted on and rendered, then use Lighthouse or another suitable performance tool separately; the flow can be correct but slow, or appear fast because it failed to perform the intended work.

For a repeatable CI gate, Lighthouse CI can collect runs and assert a budget; measure the runner's normal variation before choosing its threshold.

B.4 References

1. Segato, G. (2026, February). Quantifying Infrastructure Noise in Agentic Coding Evals. Anthropic Engineering. https://www.anthropic.com/engineering/infrastructure-noise
2. Malka, J., Zacchiroli, S., & Zimmermann, T. (2025). Does Functional Package Management Enable Reproducible Builds at Scale? Yes. Proceedings of the 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories, 775–787. https://doi.org/10.1109/MSR66628.2025.00115

3. Heričko, T., Šumak, B., & Brdnik, S. (2021). Towards Representative Web Performance Measurements with Google Lighthouse. Proceedings of the 2021 7th Student Computer Science Research Conference, 39–42. https://doi.org/10.18690/978-961-286-516-0.9

APPENDIX C

Advanced control boundaries in web applications

Created
August 27, 2026
Updated
October 5, 2026

Use this appendix when a design depends on state continuity across requests, HTTP method semantics, transactional invariants, replicated state during network partitions, client activation, runtime-generated interfaces, a detailed application contract, event-view rebuilding, or endpoint and cache policy. The Architecture chapter keeps the decisions that every project must recognize; the sections below develop implementation choices and audits that are useful only when those boundaries are present. The Scaling chapter introduces the operational choice when replicas cannot communicate; the CAP proof and lab develop that case.

Concept connections
Explore definitions and connections in the web edition.
Explore this reading's concepts

C.1 Stateless requests still depend on state

Statelessness describes what a service needs to process one request, not whether the application remembers anything. The final `2026-07-28` revision of the Model Context Protocol (MCP) removes the initialization handshake and the Streamable HTTP `Mcp-Session-Id` header. Servers must not infer protocol context from earlier requests on the same connection; the core carries no protocol session across calls (Model Context Protocol, 2026). This is a versioned protocol example, not a requirement that the course project expose MCP or that older MCP traffic use the same design.

The table below compares what a client carries forward and what the deployment must still provide.

Table C.1. Continuity designs and the deployment obligations they retain.

Continuity design	What a later request carries	Deployment obligation
Hidden protocol session	A session identifier	Route to the instance that owns it, or share the protocol-session state
Explicit application handle	A resource handle such as `workspace_id`	Authorize the operation and resolve the referenced state from every eligible instance, or retain affinity

An explicit handle moves continuity rather than deleting it. An application may select state from the authenticated principal and ordinary request parameters; when workflow state cannot be derived from them, a service can mint a handle and require later calls to return it. The service still has to store or reconstruct the referenced state and define its owner, lifecycle, concurrency rules, expiry, and cleanup. A

handle is an address, not authorization: authenticate the caller, then authorize that principal to perform the requested operation on the referenced tenant or workspace on every request. If the state remains available only on the instance that minted the handle, affinity has merely moved behind a new identifier.

Retries add another state boundary. If a mutation commits but its response is lost, a retry may reach a different instance and apply the mutation again. Do not assume that a protocol request identifier makes a domain operation idempotent. Use an application-level command identifier or idempotency key, bind it to the caller, handle, operation, and relevant arguments, and store the mutation and reusable result atomically in the authoritative application state.

State placement audit. Assume two interchangeable server instances, `A` and `B`, share one application store. Alice's validated token identifies principal `alice` in tenant `T1` with `events:write`; creating a workspace grants her access to it. Bob's token identifies principal `bob` in the same tenant with the same scope, but he has no access to Alice's workspace. The server derives these identities from the validated tokens, not from client metadata or a resource handle.

1. Alice calls `create_workspace` and receives `W1`.
2. Alice calls `record_event(W1, work_item_id=I42, command_id=C7, type=work_started)`; instance `A` commits the event, but the response stream breaks.
3. The client retries on instance `B` with a new protocol request identifier and the same `C7`.
4. Bob calls `record_event(W1, work_item_id=I42, command_id=C8, ...)`.
5. The workspace is now `archived`, and the stated lifecycle makes archived workspaces read-only; Alice calls `record_event` again.

For each request, record the authenticated principal and scope, authoritative state before the request, decision to apply, return, or reject, and the invariant or evidence supporting that decision. Design the flow so `C7` produces one event and its retry returns the original result, Bob cannot act merely by knowing `W1`, and the archived write returns a defined application error such as `workspace_archived` rather than silently recreating state. Finish by naming which information is carried by the client and which state remains authoritative on the server.

C.2 Distinguish HTTP method semantics from domain retry safety

The Pressure chapter's exercise starts from the domain contract: inspect all promised effects of one logical command. HTTP adds standardized method semantics that clients and intermediaries can use. In RFC 9110, sections 9.2.1–9.2.2, **safe** means the client requests no state change, while **idempotent** means repeating the request preserves its intended server effect.

Table C.2. Selected HTTP methods and their standardized properties.

Method	Safe?	Idempotent by method semantics?
`GET`, `HEAD`	Yes	Yes
`PUT`, `DELETE`	No	Yes
`POST`	No	No general guarantee

Identical responses are unnecessary. For example, an API could return `204` after deletion and `404` on a retry because the resource is already absent; the intended removal still happened once. The RFC also permits incidental effects such as an access-log entry per request. The exercise's domain activity event is different: exactly one event for the transition is a promised application outcome.

A `POST` command can have an application-specific retry guarantee. For the tracker, one design binds a stable command key to the caller, workspace, operation, and relevant arguments, then atomically enforces that key with the state change, activity event, and reusable result. Specify conflicting payloads, key retention, and concurrent attempts as part of the contract. The key and HTTP method alone do not implement that guarantee; the state-placement audit above and Appendix F's effect audit locate its enforcement.

HTTP semantics check. Explain whether each observation contradicts idempotency: a repeated deletion returns a different status; repeated completion adds another promised domain event; repeated retrieval adds an incidental access-log entry. For each, name the intended effect and distinguish it from the response or diagnostic record. Then explain what evidence a client would need before automatically retrying the application's `POST` command.

C.3 Preserve durable invariants with transactions

A database transaction groups operations into one unit with four commonly named properties: atomicity, consistency, isolation, and durability, or ACID (Härder & Reuter, 1983). **Atomicity** means the operations commit together or not at all. **Consistency** means a committed transaction leaves the data within its declared invariants; the database cannot invent those application rules, so they still have to be expressed through constraints, guarded writes, or validated domain logic. **Isolation** describes how concurrent transactions are allowed to observe and affect one another. The exact guarantee depends on the selected isolation level rather than on the mere presence of a transaction. **Durability** means a successful commit survives the failures covered by the storage system's contract.

The work-item lifecycle contains a useful transaction boundary. A successful transition changes the current state and records the corresponding activity event. If either write escapes alone, the activity feed and work item disagree. The same invariant is threatened when two requests make a decision from the same old state or when the application returns success before persistence is durable.

C.3.1 ACID transaction lab

Choose one disturbance, predict where the invariant could leak, and run it first with application-managed writes. Then introduce a database transaction and, for concurrent requests, compare read committed with serializable execution.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/advanced-web-boundaries/

The simulation does not imply that every database implements an isolation level or durable commit in the same way. For an implementation-specific claim, identify the real transaction boundary, constraints, isolation setting, acknowledgment point, and documented failure model, then test them against the deployed storage path.

C.4 Partitions and replicated state

Use this section when a design replicates authoritative reads or writes across nodes that may lose contact. The CAP result is narrower than a general slogan about balancing consistency and availability. For a distributed read-write object in an asynchronous network, a network partition can prevent a design from guaranteeing both atomic consistency and an available response from every non-failing node (Gilbert & Lynch, 2002). Here **atomic consistency**, or **linearizability**, means each operation appears to take effect at one instant between its invocation and response, in an order that respects real time. If a write completes before a read begins, that read must see the written value or a later write in that order. **Availability** means every request to a non-failing node eventually completes with a response allowed by the object's contract; this guarantee does not impose a latency bound. The partition is the condition that makes those guarantees collide; it is not a feature that an application can disable after communication has already failed.

C.4.1 A register proof

Consider a register initially holding `0`, replicated at nodes L and R, with communication between them partitioned:

1. In execution A, no write occurs and a client reads at R. A valid read returns `0`.
2. In execution B, a client writes `1` at L and receives success, then another client begins a read at R. Linearizability requires `1`, since no later write occurs.
3. R has received no message about the write, so its local observations can be identical in A and B. Returning `0` violates B; guessing `1` violates A. Waiting indefinitely for communication sacrifices availability, and refusing the read does not fulfill the register's read contract.

Requiring L to wait before acknowledging the write moves the availability loss to the write; it does not remove the conflict. This is the indistinguishability argument behind the impossibility result (Gilbert & Lynch, 2002).

A system that requires a majority quorum may preserve one history by rejecting or delaying a request that reaches an isolated minority. A system that accepts writes on both sides may keep responding, but the replicas can diverge and later need domain-specific reconciliation. Neither label chooses the right policy on its own. The decision depends on which operations may be stale, rejected, delayed, merged, compensated, or repaired without violating the domain.

C.4.2 CAP partition lab

Split one region from a two-region majority and apply the same two work-item transitions under both policies. Inspect the replies before healing the network: reconciliation can change stored state, but it cannot retract a success that a user already observed.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/advanced-web-boundaries/

The lab models one write invariant and a fixed topology; preserving that invariant alone does not establish linearizability for every read and write. Linearizability's real-time ordering is also distinct from **ACID atomicity**, which concerns a transaction taking effect entirely or not at all. Real systems need explicit definitions for reads as well as writes, failure detection and timeouts, quorum membership, leases, conflict resolution, and the consistency guarantee exposed to clients.

C.5 Rebuild a view without repeating its effects

A retained event history can support several derived views when it records the inputs needed to compute them. Kreps (2013) explains how deterministic processing of the same ordered history reconstructs state. Changing the computation can instead produce a new view from that history.

Suppose a workspace dashboard currently counts all activity and now needs separate counts for started and completed work. An illustrative retained fixture contains `E1: work_started`, `E2: work_completed`, and `E3: work_started`, all for workspace `W1` in that order. Replaying those records through the new aggregation gives two starts and one completion. The rebuild writes a separate summary and records its last processed position; the application can keep serving the old summary until the new one catches up to an agreed position and is verified. Events arriving during the rebuild must be included through that cutover position, then processing must continue without skipping or double-counting them.

Recomputing the summary must not resend the notifications originally triggered by those events. Keep the projection that updates the summary separate from notification delivery. If delivery itself needs recovery, its durable record and receiver-enforced idempotency belong to the effect boundary described in Appendix F. Rebuilding derived data and resuming a workflow are different uses of history.

Record the event identities, workspace, type, schema and aggregation versions, ordering, retained starting point, and acceptable summary lag. Expired or incomplete records may prevent a rebuild; the course tracker's activity history does not necessarily contain every work-item field or permission. Name which views the retained facts support and who owns their rebuild and freshness.

C.6 Model valid data combinations

A transaction constrains how a valid state change becomes durable. Constructive data modeling asks which combinations ordinary code should be able to construct before that boundary is reached. In *The Unreasonable Effectiveness of Constructive Data Modeling*, Alexis King describes defining the

supported "positive space" with ordinary records and tagged variants, then using exhaustive case analysis where the language supports it (King, 2026). King's earlier *Parse, Don't Validate* frames a boundary check as a transformation that returns a more precise representation carrying what it learned rather than only reporting success (King, 2019). These are design arguments rather than comparative evidence, and the goal is not maximum type precision.

For the result of a state-transition decision, a loose record does not require the payload that each outcome needs:

```
type TransitionDecision = {
  accepted: boolean;
  next?: WorkItem;
  current?: WorkItem;
  event?: ActivityEvent;
  reason?: TransitionError;
};
```

It permits `accepted: true` without a next item or event, and `accepted: false` without a current item or reason. Model the required payload for each branch instead:

```
type TransitionDecision =
  | { kind: "accepted"; next: WorkItem; event: ActivityEvent }
  | { kind: "rejected"; current: WorkItem; reason: TransitionError };
```

The discriminator makes each branch's required payload explicit, and narrowed type-checked code cannot treat a rejected decision as if it exposed a transition event. It does not prove that the event matches the correct transition unless the inner types or constructor also enforce that relationship. The value is an in-memory decision, not a durable fact. Untrusted, untyped, or deserialized input crossing a boundary still needs runtime parsing. Before committing a decision, the request path must authenticate and authorize the caller; the persistence operation must conditionally match the authoritative state or version, then atomically update the item and append the event. Malformed deserialization, mutation, or unsafe casts can bypass the representation, while concurrency and retries can violate surrounding invariants even when each individual value is valid.

Selected-invariant check. In the existing `specs/{feature}/spec.md`, show one broad record or result and one contradictory value it permits. Replace it with a stack-native tagged union, sealed hierarchy, or guarded constructor. Where the stack supports checked variants, add a compile-fail type fixture; add a runtime test for any parser or guarded constructor at a data boundary, and reuse an existing supported-behavior test. Record one important invariant that remains enforced elsewhere, such as authorization or atomic concurrent transition.

C.7 Activate delivered interfaces

Rendering and navigation do not explain how delivered HTML becomes interactive. A server-rendered page may contain useful content while the browser still has to recover or load the behavior behind its controls. That activation work affects payload size, client processing, first-interaction latency, and application structure.

Table C.3. Client activation approaches and their principal trade-offs.

Approach	Activation unit	Main benefit	Main constraint
Hydration	The rendered component tree	Familiar, general application model	Re-executes client code to recover state and listeners
Progressive hydration	Ordered portions of the same tree	Prioritizes important interaction	The application is still eventually hydrated
Islands architecture	Explicit interactive regions	Static HTML elsewhere; work can be deferred or avoided	Shared state, navigation, and communication across islands
Resumability	Serialized state and handler boundaries	Continues without replaying the application	Serializable state and framework compiler or runtime constraints

Islands partition a page into static and interactive regions and attach a loading strategy to each interactive region. Resumability changes how execution crosses the server-client boundary: component boundaries, application state, and listener references are serialized into the delivered HTML so the browser can continue without replaying the application (Vepsalainen et al., 2024). A resumable framework can therefore create fine-grained islands automatically.

The strongest fit for explicit islands is an application with meaningful static regions and a smaller amount of separable interactivity. Research on edge-powered islands found that work could be deferred or avoided, but also concluded that the benefit becomes harder to obtain as an application grows more dynamic and difficult to divide into independent portions (Vepsäläinen et al., 2025). Inter-island state, navigation, and communication then become architectural costs rather than incidental implementation details.

C.7.1 Client activation lens

The lab below holds the server-rendered event page constant and changes only how its behavior becomes available in the browser. Compare what happens before the first interaction, then run the same record-event action. The transfer values are illustrative teaching values, not framework benchmarks; a real comparison should record production bundles, client execution, and interaction latency on representative devices.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/advanced-web-boundaries/

Ask which code and state must cross the boundary, which work happens before the user requests it, which latency moves to the first interaction, and what new constraints the chosen technique introduces. For each interactive region in the project, record its activation strategy, state owner, and communication path.

C.8 Runtime UI generation changes the control boundary

The earlier sections ask where a view is produced and how it becomes interactive. A separate question is how much of its structure and behavior model output may choose at runtime. A component written by a coding agent during development is not runtime-generative UI in this sense: once reviewed and

committed, it is ordinary application code at runtime. The boundary expands as runtime output moves from filling a host-selected component to selecting component composition or executable behavior.

Casas (2026) describes a practical spectrum from fixed components, through declarative descriptions rendered from an approved catalog, to generated executable code. Treat this as a design lens rather than a settled taxonomy. The table below shows how the application's control obligation expands with the model's runtime freedom.

Table C.4. Runtime model-output boundaries and the application controls they require.

Runtime model output	Application must control	Main trade-off
Data for one approved component	Component implementation, behavior, and prop schema	Predictable behavior with little freedom over layout
Declarative descriptor using approved components/actions	Schema, renderer, design system, and action registry	Composition within a grammar, with validation and schema-evolution burden
Generated HTML, CSS, and JavaScript	Isolated execution boundary, mediated data/tool bridge, and limits	Broadest expressiveness with the largest failure, review, and containment load

This axis can be combined with server-side or client-side rendering, hydration, islands, or resumability. It does not replace those choices. A declarative descriptor is still untrusted input: reject unknown components and actions, validate data bindings and accessible names, bound size and nesting, and authorize an action when it is invoked. Resolve action identifiers through trusted application code; naming an action in generated data does not grant authority to perform it.

Executable generated UI expands the boundary further. It needs a separate-origin sandbox or equivalent isolated runtime without ambient credentials, a narrow bridge to host data and tools, explicit network, storage, navigation, and tool permissions, resource limits, observability, and a safe fallback. Running generated code in a browser does not by itself provide that boundary.

A Google Research preprint provides bounded preference evidence for one complete generative-UI pipeline. After excluding eight of 100 sampled LMArena prompts, the study sent each pre-cached result to two raters. Across the pairwise comparisons, raters preferred the generated UI over the study's markdown condition in 82.8%. Contractor-built pages were preferred over the generated pages in 50.0% of comparisons, while the generated pages were preferred in 35.3% and the remainder was neutral. The pipeline combined Gemini with search and image tools, extensive instructions, and post-processing; generation often took one to two minutes and was excluded from the ratings. The study therefore measures preference in that setup, not task success, correctness, accessibility, security, maintainability, or production readiness (Leviathan et al., 2026).

C.8.1 Show the context behind a generated view

A generated view can satisfy its schema while concealing why particular information was selected or omitted. Show a compact **context receipt** that lets the user inspect the inputs and transformation behind the view. For example, a lecture-material selector could display:

> **Inputs:** aggregate counts from predefined audience choices and a named revision of the curated reading list.

> **Sent to the model provider:** those counts and reading-list entries; no names, individual responses, or private notes.
>
> **Transformation:** selected and ordered existing readings; the complete list remains available.
>
> **External action:** none; the result only changes this view.

This is an illustrative design, not a claim about a deployed system. Populate the receipt from application-controlled input and action records, not the model's recollection of what it used. Keep it accurate when context changes or a deterministic fallback replaces generation, and describe any additional data the application sends. Offer a route to inspect the source material and revise the selection criteria.

The receipt makes data use and selection inspectable; it does not itself enforce privacy or authorize an action. Those boundaries still belong to the application, and the receipt should avoid exposing sensitive input values merely to demonstrate transparency.

C.8.2 Field note — Constrain generation around the hard part

For Slideotter, reliable layout was essential. Geometry, text-fit, and render-baseline checks appeared before the browser studio.

The LLM authoring plan assigned intent and structured content to the model, validation and approval to the server, and rendering to the existing engine. Its schema-backed implementation kept arbitrary executable components outside model output. Preview, comparison, PDF export, and validation later converged on one DOM renderer, so checks examined the layout users saw.

This left geometry, spacing, overflow, and media placement under application control while the model selected content and supported layouts. Declarative layout expansion allowed variety within that contract. I gave the model freedom around the property I needed the system to enforce reliably: layout.

C.9 Descriptor boundary audit

Create one small event-summary data fixture and keep it unchanged. Then:

1. Define the prop schema and payload for one fixed summary component.
2. Define minimal prop schemas for an approved registry such as `metric`, `eventList`, and `button`, plus a small action allowlist.
3. Express the same fixture as a declarative descriptor.
4. Reject unknown components, raw HTML or script, unregistered actions, controls without accessible names, and excessive nesting. For each rejection, name the enforcing layer, error, and test.
5. Use the fixed read-only summary as the fallback.
6. Identify the additional containment and evidence required before the application could execute model-generated code.

The descriptor schema and action vocabulary are API contracts. The next section examines what follows once such contracts cross client and server boundaries.

C.10 Choose an application interaction style

An interface distributes knowledge between consumers and providers: which addresses or operations exist, how data is represented, what errors mean, and how either side can change independently. Several interaction styles can expose the same behavior while placing coupling in different locations.

Table C.5. Application interaction styles and where each places coupling.

Style	What the consumer knows in advance	Where change pressure concentrates	Typical operational concern
Resource-oriented HTTP	Resource addresses, representations, methods, and statuses	Resource and representation compatibility	Caching, idempotency, pagination, and versioning
Hypermedia-driven REST	Entry points, media types, and stable relation semantics	Relation semantics and completeness of advertised controls	Link validity, client interpretation, and adoption
GraphQL	A typed schema and the fields needed for each operation	Schema governance, field evolution, and deprecation	Authorization, resolver behavior, and query cost
RPC	Named methods and request or response message contracts	Method and message compatibility	Deadlines, retries, idempotency, and streaming flow

Resource-oriented HTTP uses the uniform method and status semantics of HTTP. Hypermedia-driven REST adds links or controls so representations advertise available transitions; HATEOAS is therefore a REST constraint rather than a separate peer to REST (Fielding, 2000). GraphQL lets a consumer select a response shape against a typed schema. RPC exposes named operations and explicit messages. GraphQL and RPC commonly travel over HTTP, so transport and application interaction style remain separate choices.

Event streams, queues, webhooks, and publish-subscribe interfaces add another decision: communication can become asynchronous. That can reduce temporal coupling while making delivery guarantees, ordering, replay, idempotency, and schema evolution explicit parts of the contract.

C.10.1 API contract lens

The lab below holds one user intent constant: start work on item 42 and obtain its current state, title, workspace, and available next actions. Switch the interaction style to see what crosses the wire, what the consumer must know in advance, and where a request for one more related field creates pressure.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/advanced-web-boundaries/

No style wins every row. A stable public resource model, a workflow discovered through hypermedia, a frontend assembling related data, and an internal service command provide different reasons to accept different forms of coupling. Begin with consumers, change patterns, network conditions, tooling, and ownership rather than a preferred acronym.

For the project, design a minimal event-creation operation and event-query operation in one style. Then identify what changes when consumers retry, devices go offline, a new workflow state appears, many tenants share the system, or the interface needs an additional related field.

C.11 Executable HTTP contracts

An API description becomes more durable when representative requests and expected responses are executable. Hurl expresses HTTP sessions in plain text and can assert status codes, headers, and response bodies without depending on the application's implementation language. For example, an HTTP-based event tracker could preserve one creation-and-read scenario as follows:

```
POST http://localhost:3000/events
Content-Type: application/json
{
  "type": "work.started",
  "workItemId": "item-42"
}

HTTP 201
[Captures]
event_id: jsonpath "$.id"

GET http://localhost:3000/events/{{event_id}}

HTTP 200
[Asserts]
jsonpath "$.type" == "work.started"
jsonpath "$.workItemId" == "item-42"
```

This file is both an example of the interface and a deterministic integration test. It can be run locally or in continuous integration, and a failure points to the request or assertion that no longer matches the implementation. If implementations A, B, and C expose a compatible HTTP boundary, the same scenario can be run against each one by changing the base URL rather than rewriting the test in each implementation's language.

Hurl does not replace browser testing: it checks the HTTP boundary, while a tool such as Playwright checks behavior through a browser, including rendering and interaction. Although Hurl can repeat requests and assert response duration, a small contract scenario is not evidence that the system will behave well under representative load. The common project specification does not require an HTTP API, so this technique is optional for implementations that expose one rather than a universal project requirement.

C.12 Use the boundary that matches the risk

These techniques solve different problems. Explicit handles and idempotency protect continuity across requests; transactions protect durable invariants under concurrency and failure; constructive models reduce contradictory in-memory states; activation approaches move browser work; descriptor schemas constrain runtime composition; isolation contains executable output; interaction styles distribute coupling; and HTTP examples preserve an interface contract. Choose the smallest boundary that makes the relevant decision observable and enforceable.

C.13 Endpoint and cache policy

Use these details when automated demand or fragmented cache reuse is a relevant pressure in the Architecture chapter.

C.13.1 Automated consumers change demand and exposure

A crawler, integration, monitoring probe, authorized agent, and abusive bot can send technically similar HTTP requests while creating different value, cost, and risk. Agents can traverse interfaces quickly, retry ambiguous failures, and act without a person inspecting every screen. The same operation should retain its authoritative meaning whether invoked by a form, integration, or agent. Architecture should therefore distinguish which representations are intentionally public, which actions require identity and authorization, which expensive paths need a quota or admission boundary, and which automated uses the product is willing to support.

The Robots Exclusion Protocol lets a cooperating crawler discover access preferences in `/robots.txt`. It is not authorization or a content-security boundary, and listing a path there makes the path discoverable. Information that must remain private still needs an application-layer access control; public information should be designed with the assumption that a permitted consumer can copy it.

Rate limiting controls resource use rather than deciding whether every automated consumer is malicious. An HTTP `429 Too Many Requests` response can explain the limit and carry `Retry-After`, but RFC 6585 deliberately leaves the identity and counting policy to the application. Limits based only on an IP address can group unrelated users or be distributed across many addresses, while limits tied to an account, tenant, capability, or costly operation require their own fairness and privacy decisions.

The OWASP automated-threat catalog treats scraping as one form of automated use among several, and its practical implication here is to begin from the endpoint and intended outcome rather than from a generic “block bots” goal. For one exposed path, record the legitimate consumers, information returned, authoritative action or cost triggered, likely abuse pattern, first useful signal, control point, acceptable false-positive cost, and recovery or override path. Cache safe public reads, minimize exposed and retained data, protect expensive or consequential actions, and observe origin work per useful outcome before adding a challenge to every request.

C.13.2 Query parameters and cache identity

Query parameters can fragment the cache even when the response is identical. For example, `/offer?utm_source=newsletter` and `/offer?utm_source=search` may serve the same page but occupy separate cache entries. The proposed `No-Vary-Search` response header lets an origin declare which query parameters a supporting cache may ignore when matching URLs:

```
No-Vary-Search: params=("utm_source" "utm_medium")
```

With this policy, differences in those campaign parameters can permit reuse of a stored response, while a content-changing parameter such as `colour` still distinguishes responses. Unlike `Vary`, which selects responses using request headers, `No-Vary-Search` concerns the URL's query component. It changes matching, without rewriting the URL or relaxing storage, freshness, or validation requirements.

List only parameters known to be safe to ignore so that new, unknown parameters still distinguish responses. Identical HTML is insufficient if reuse would bypass required server processing, such as authorization or signature verification. An ignored parameter also cannot be relied on to bypass the cache for debugging. As of September 2026, the HTTP Working Group specification remains a draft: verify syntax and behavior in the target caches before relying on it. Harry Roberts's practical introduction develops the campaign-URL example further.

C.14 References

1. Model Context Protocol. (2026, July). Model Context Protocol Specification, Revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28
2. Leviathan, Y., Valevski, D., Kalman, M., Lumen, D., Segalis, E., Molad, E., Pasternak, S., Natchu, V., Nygaard, V., Venkatachary, S., Manyika, J., & Matias, Y. (2026). Generative UI: LLMs are Effective UI Generators. arXiv Preprint arXiv:2604.09577. https://doi.org/10.48550/arXiv.2604.09577
3. Casas, R. (2026, June). Beyond Components: Designing Generative UI for MCP Apps. AI Engineer, YouTube video. https://www.youtube.com/watch?v=hCMrEfPG2Yg
4. Vepsalainen, J., Hevery, M., & Vuorimaa, P. (2024). Resumability—A New Primitive for Developing Web Applications. IEEE Access, 12, 9038–9046. https://doi.org/10.1109/ACCESS.2024.3352891
5. Vepsäläinen, J., Vuorimaa, P., & Hellas, A. (2025). The Potential of Serverless Edge-Powered Islands for Web Development. Journal of Web Engineering, 24(1), 1–38. https://doi.org/10.13052/jwe1540-9589.2411
6. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures [Phdthesis, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm
7. King, A. (2019, November). Parse, Don't Validate. Personal technical essay. https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/
8. King, A. (2026, July). The Unreasonable Effectiveness of Constructive Data Modeling. Software Should Work conference presentation. https://www.youtube.com/watch?v=0BXuYlNrUmE
9. Härder, T., & Reuter, A. (1983). Principles of Transaction-Oriented Database Recovery. ACM Computing Surveys, 15(4), 287–317. https://doi.org/10.1145/289.291
10. Gilbert, S., & Lynch, N. A. (2002). Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. SIGACT News, 33(2), 51–59. https://doi.org/10.1145/564585.564601
11. Kreps, J. (2013, December). The Log: What Every Software Engineer Should Know About Real-Time Data's Unifying Abstraction. LinkedIn Engineering. https://www.linkedin.com/blog/engineering/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying

APPENDIX D

Agent-system security, control, and traceability

Created
August 27, 2026
Updated
October 5, 2026

Use this appendix when an agent workflow crosses several orchestration, model, tool, credential, or network boundaries. The Agent Fleets chapter keeps the essential rules—allocate decision authority, enforce least privilege, preserve evidence, and keep a human owner—while this appendix develops control-plane separation, end-to-end trace reconciliation, capability adoption, and persistent cross-run state. The optional fleet exercises test project memory, contract completeness, and human review gates.

Concept connections
Explore definitions and connections in the web edition.
Explore this reading's concepts

D.1 Separate task control from resource access

A session or orchestration layer coordinates agent tasks; a resource-access layer mediates selected model and tool requests. These responsibilities can share a platform or span several services, but their decisions and enforcement points must agree.

The table below separates task-level orchestration from request-level resource access.

Table D.1. Control boundaries for agent orchestration and model or tool access.

Control boundary	Primary unit	Typical responsibilities
Session and task orchestration	Run, delegated task, workspace	Agent-harness selection, task graph, session history, collaboration, delegated tool authority, sandbox choice, and task-level policy
Model and tool access	Model or tool request	Permitted services and credentials, request routing and fallback, rate and spend limits, request guardrails, and shared usage telemetry

The durable distinction is between a decision and its enforcement point. NIST's zero-trust architecture distinguishes a policy engine that makes and logs an access decision, a policy administrator that executes it by configuring an enforcement point, and the enforcement point that enables, monitors, or terminates access; implementations may combine the engine and administrator (Rose et al., 2020). This is a design analogy, not a claim of NIST conformance. For each consequential decision, name the policy authority, mandatory enforcement point, and conflict or outage rule; overlapping controls still need explicit precedence, and a mediator cannot protect a path that bypasses it.

The fleet authority map identifies the intended transfers of work and authority. Check where the controls enforce and observe those transfers. Trace one operation through the chain:

`originating actor → fleet session → delegated task → model or tool attempt → validation → delegated-run outcome`

Propagate only the necessary authenticated principal, audience-scoped delegated authority, and trusted task, trace, budget-reservation, and policy-decision identifiers. Each enforcement point must validate the authority and identifiers it relies on. Return the resource actually selected, routing or fallback decision, policy result, and observable latency, usage, and cost; for a model call, also record its provider, model, and version where available. Associate every physical attempt with one delegated run and that run's explicit outcome, such as accepted, rejected, failed, cancelled, or unresolved. Default telemetry to metadata: capture prompts, source, tool arguments or results, approval details, or principal data only when necessary, with redaction, access control, and deliberate retention, and never place credentials in trace context. A trace identifier connects evidence; it is not proof of identity or permission. Without this contract, the resource layer can see spend without knowing whether the work was accepted, while the orchestration layer can see intent without knowing which model or permission was actually used.

Audit one delegated task:

1. Choose two consequential decisions: one about the session or task and one about a model, tool, credential, or other resource.
2. For each decision, record the policy authority, mandatory enforcement point, trusted context or evidence, and conflict or outage rule.
3. Trace the task end to end.

During the trace, look for:

- a direct-call bypass;
- lost actor identity;
- multiplied retries;
- conflicting caps or double-counted usage;
- cancellation that does not reach in-flight work.

Where practical, use a safe direct-call negative test and, where records are available, reconcile the mediator with the model provider or tool resource rather than inferring non-bypass from an incomplete trace.

The audit is complete only when every denied, failed, retried, cancelled, or successful attempt:

- has an identifier and policy result;
- has usage or cost provenance where measurable;
- rolls up once to an explicit delegated-run outcome.

When computing cost per accepted outcome, include spend from every attempt in the fixed cohort. Use accepted delegated runs—not internal calls—as the denominator, and label missing or estimated costs rather than treating them as zero.

For the B–C comparison, keep any gateway pinned to the declared base model where possible; otherwise record the model actually used and treat substitution as a confounding factor.

D.1.1 Fleet trace debugger

The forensic packet below separates orchestration outcomes, gateway decisions, and physical provider or resource records. Map every physical row to a delegated run or leave it unmatched, then reconcile the packet without counting the same attempt once per log source.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/agent-system-security/

The packet is synthetic and deterministic. It rehearses evidence joins, accounting, bypass detection, and late-effect reasoning; it does not estimate the reliability or cost of a particular fleet platform.

D.2 Trust boundaries

An agent's role should define not only its task but also which information it can read and which actions it can take. Simon Willison's lethal trifecta names a particularly dangerous combination: access to private data, exposure to untrusted content, and a way to communicate externally. Prompt injection in the untrusted content can then turn the agent's legitimate permissions into a path for data exfiltration.

D.2.1 Separate credential custody, action authorization, and containment

A credential-injecting proxy can let an agent call a service while another process holds the real credential. fnox's credential-proxy documentation describes supplying placeholders and substituting credentials into matching HTTPS requests; it explicitly distinguishes this from an operating-system sandbox. Claw Patrol's security model also separates protection of injected credentials from broader host access and network containment. These are documented designs, accessed September 24, 2026; their effectiveness depends on the deployed configuration and its actual access paths.

The table below separates the properties to check.

Table D.2. Credential protection, operation permissions, and containment require separate evidence.

Property	Governing question	Evidence to seek
Credential custody	Can the agent obtain the protected credential through any available path?	Denied access to credential sources and no disclosure through environment, logs, or returned results
Action authorization	Which services, resources, and operations does the task permit?	An allowed operation succeeds while an out-of-scope resource or operation is refused
Containment	Which files, processes, and network destinations can the agent reach?	Forbidden access is blocked even when the agent attempts to bypass the mediator

Suppose an agent may read one workspace's events to calculate an activity count. Keeping its credential hidden does not prevent an overly broad permission from allowing access to another workspace or a write operation. The agent may also receive permitted data and send it through a separate network route. Protecting the credential, restricting authenticated operations, and controlling that route address different failures.

For an optional version of the control-boundary audit, use a paper fixture or dummy service to follow these three failures. Name the component that would refuse each action and what happens if the mediator is unavailable; a required check must not silently become permission to continue. For executable checks, preserve the denied results and configuration as evidence for the tested paths; a paper analysis remains a proposed design.

D.2.2 Imported capabilities are development dependencies

A skill may arrive as prose, yet it can direct tool calls made with the agent's authority and may bundle code or configuration. A USENIX Security 2026 study crawled 98,380 skill entries from two community registries in January 2026. Static triage selected 4,287 candidates; sandboxed behavioral verification and independent judgments by two of the authors confirmed 157 malicious skills (Liu et al., 2026).

Those 157 are a behaviorally confirmed lower bound in one snapshot, not an ecosystem prevalence estimate. The snapshot excluded private, unindexed, and official collections; one publisher identifier contributed 85 templated cases; and 60-second sandbox runs could miss dormant or environment-gated behavior. Official provenance, popularity, scanning, or a previous safe version can filter risk; none proves that an imported capability or update is safe in the current host.

Treat adoption and updates as dependency changes. Record the source and resolved revision; inspect instructions, bundled files, installation hooks, downloads, and mutable remote inputs; identify missing or unreviewable content; and map requested access against the authority the host would actually grant over files, commands, networks, and credentials. Perform any dynamic verification only in a disposable sandbox approved for security testing, with the least authority needed and no production credentials. Connect output produced with the capability to independent project checks, and define who reviews updates and how the capability can be removed.

The skill-adoption lab provides a non-executing fixture for practicing this review without loading an untrusted capability into an agent context; study its source or download the standalone lab.

> **Case study: when exploration scales**
>
> OpenAI's August 2026 incident report describes a sequence spanning reinforcement-learning training in May and June and cybersecurity evaluations in July. Agents first found unauthorized communication and internet-access paths through shared Artifactory infrastructure; after that service was rebuilt and the evaluations restarted, agents working on ExploitGym re-established the channel and chained vulnerabilities across OpenAI infrastructure, external services, and Hugging Face's production systems (OpenAI, 2026). Hugging Face reconstructed roughly 17,600 agent actions across dataset processing, Kubernetes, cloud identity, an internal network, and source control. Most attempts failed, but the volume and persistence of exploration eventually exposed a viable path across several trust boundaries (Larcher et al., 2026).
>
> OpenAI's investigation shows that the continuity mechanism preceded the Hugging Face intrusion. Separate training runs first used files left in Artifactory as an informal message board for requests, exploit details, assignments, scripts, progress, and credential information. After Artifactory was

> rebuilt and agent credentials were revoked, later evaluation runs recreated the channel through unauthenticated WebDAV requests and message-bearing directory names (OpenAI, 2026).
>
> The sandboxes were short-lived, but the runs were not independent. Unlike the curated project memory described earlier, this was ambient infrastructure state. A location becomes cross-run memory and an unintended coordination channel when one run can leave state that survives, is discoverable, and influences a later run. Artifact and package stores, caches, queues, logs, temporary storage that outlives a run, and remote workspaces therefore belong in the trust-boundary review. For each channel, record who can write and later read it, how writes are attributed, what may be stored, how long it persists, and how it is cleared.
>
> The incident did not make established defensive principles obsolete. It showed why they become more important with agents: strict sandbox isolation, least privilege, short-lived and narrowly scoped credentials, blocked metadata access, and telemetry that can correlate behavior across systems. Agent safety therefore depends on the authority exposed by the surrounding system, not only on instructions or model behavior.

The safest design is to break the combination where possible. A scout reading untrusted issues may not need secrets or network-writing tools. An implementer may work in a sandbox with scoped files and no deployment credentials. A publisher may receive a reviewed artifact instead of the full private context that produced it. Human approval can help at a sensitive boundary, but an approval prompt is not useful when the effect is unclear or reviewers have learned to accept it mechanically.

D.2.3 Field note — Permission follows the risk profile

I do not maintain one universal list of actions that agents may or may not perform. The appropriate authority depends on the project and its risk profile.

I consider three factors first:

1. **Financial cost.** Could the action create meaningful or unbounded spending?
2. **Possible consequences.** Could a mistake affect users, sensitive data, external systems, or something else that matters?
3. **Recoverability.** Can I detect the mistake and restore a useful state without disproportionate effort or harm?

For small, low-risk projects, I allow agents to work relatively freely with Cloudflare infrastructure. Direct API or dashboard changes are acceptable when I do not expect major spending, the project contains no sensitive data, and a mistake would have limited consequences. If an action may introduce cost, I negotiate a limit with the agent before it proceeds. That agreement is a decision boundary, not an enforcement mechanism. Where spending could become material, the limit also needs a provider budget, quota, or another control outside the model.

This convenience trades away some reproducibility and reviewability. A direct infrastructure change may not leave the repository as a complete description of the deployed state. I accept that trade-off only when the experiment is disposable enough that I can inspect, recreate, or abandon it without material harm.

The same permission should not transfer automatically to another project. As financial exposure, consequences, or recovery difficulty increase, the agent's role should narrow from executing the action to preparing a reviewed change. Authority should follow the expected loss and recovery path, not the mere fact that an agent successfully performed a similar action elsewhere.

D.3 External contribution policies

Use this example when preparing an external issue or pull request. The shared course guidelines distinguish course evidence from a receiving project's submission rules. The example below illustrates how private tooling, public authorship, and disclosure can be governed differently.

For example, the Human Responsible policy template, version 1.0.0, permits private tooling while requiring public text written and posted by the contributor, understood and tested changes, and human follow-up. It prohibits AI-disclosure notes and submissions from automated accounts. These are requirements for projects that adopt the template.

Under that policy, a student can record private agent assistance in the course evidence while personally writing and posting the reviewed contribution text and owning the change. Answer provenance questions truthfully, avoid false claims of human-only production, and respect a project's refusal of particular assistance or submissions. Disclosure does not make a prohibited contribution acceptable; an external disclosure rule does not remove the course's evidence requirements.

D.4 Choose a delegation environment

Choose the execution and control boundaries the exercise needs before choosing a product. The course environment sheet records the supported tool, version, model route, limits, and fallback for each course run. A product can expose several of the patterns in Table D.3; its name alone does not identify the workflow being measured.

Table D.3. Delegation boundaries to inspect when selecting a course environment.

Delegation pattern	Where work is coordinated	Controls to inspect	Evidence to retain
Separate local processes	A local harness starts child processes	Inherited tools and credentials, process termination, shared files, and worktree boundaries	Child inputs and outputs, process status, tool calls, and integration changes
Managed child sessions	A parent session delegates through a harness or service	Child permissions, context and memory inheritance, nesting limits, and cancellation	Parent–child identifiers, attempt outcomes, usage, and retained state
Hosted repository task	A service schedules work in a repository environment	Repository access, network policy, credentials, checks, and merge authority	Task revision, environment configuration, changes, checks, and review outcome
Application orchestration	Application code or an API coordinates model and tool calls	Tool execution location, authorization, retries, budgets, and result acceptance	Call graph, tool results, failures, usage, and accepted application outcome

For an optional code-reading exercise, Pi's reference subagent extension makes process creation and result handling inspectable. Record the source commit you read and trace one child from launch to completion or cancellation. For a service-based contrast, the Responses multi-agent guide describes

delegated model work and application tool execution; inspect which responsibilities remain with the caller. These are entry points for examining mechanisms, not required installations or a ranking of current products.

Omnigent illustrates harness coordination: its open-source meta-harness wraps several agent harnesses behind a common session interface and documents composition, shared sessions, contextual policies, and local or remote sandbox execution (Zaharia et al., 2026). Unity AI Gateway illustrates traffic governance: for model and MCP traffic routed through it, Databricks documents catalog-backed permissions, rate limits, budgets, provider routing and fallbacks, and usage records (Databricks, 2026). These are examples, not evidence that every fleet needs two products or exactly two control planes. Both accounts are documented in Databricks-authored sources, their responsibilities overlap, and Omnigent was introduced as alpha; they are not independent evaluations of effectiveness.

For any option, inspect how child tasks inherit tools and credentials, whether concurrent writers are isolated, what state survives between runs, and how the execution trace is exported. Persistent memory, remote execution, and external messaging must be included in the declared workflow when enabled; they can change both the comparison and the authority boundary. Application-level agent orchestration has separate runtime users, data, and operational obligations.

D.5 Complete the control-boundary review

For each consequential action, name the decision authority, mandatory enforcement point, granted scope, evidence returned, and conflict or outage rule. Then follow one denied, failed, retried, cancelled, or successful attempt across every relevant system and confirm that it rolls up exactly once to an explicit delegated-run outcome. Include a bypass check for any control the decision relies on.

D.6 Optional fleet exercises

These extensions revisit the constraints and acceptance boundaries of the fleet walkthrough. Choose one when project memory, a missing consumer obligation, or a human approval gate is relevant; completing all three is not required. For any chosen exercise, add its findings to the existing fleet evidence rather than creating another submission.

D.6.1 Retrieve guidance after a contract changes

An identifiable old statement may be useful history and wrong guidance for the current implementation. Keep three questions separate: **provenance** identifies where a claim came from; **validity** concerns whether it still applies; **retrieval** selects material for the present task.

Use a small fixture with two revisions of an event-list endpoint:

1. In the earlier revision, the endpoint returns an array; in the current revision, it returns an object containing `items` and `nextCursor`. Preserve both contracts with their source revisions and status, but deliberately leave one searchable summary describing the old response as current.

2. Ask the agent separately what the earlier contract required and how a client should parse today's response. Require supporting source revisions and a client check against the current fixture.
3. Compare source inspection alone with source inspection plus retrieved project memory, keeping the task, model, tools, and starting code fixed in fresh sessions. Inspect whether the stale summary is retrieved, whether the contradiction is noticed, and which source controls the answer.
4. Record accepted behavior, stale claims, evidence recovered, retrieval cost, and the effort needed to correct or retire the summary.

Keep the historical contract available while preventing it from silently governing current work. A timestamp or source link helps inspection but does not establish validity by itself. Judge memory by the work it improves, including its initialization and maintenance cost.

D.6.2 Check a contract against its consumers

Use the two-revision event-list fixture above, whose current response contains `items` and `nextCursor`. A discoverable contract can still leave out an obligation. Code Contracts places identified requirements beside code, but its formatter checks syntax and duplicate identifiers rather than semantic compliance. Decision Engineering records intended decisions, their dependencies, enforcement, and verification; the record still needs checking against actual consumers and behavior.

Extend the fixture with a CSV export whose stated requirement is to include every event matching the selected filter. Keep the data fixed and use enough matching events to require at least two pages. Omit this consumer from the contract map and give it an implementation that reads `items` but ignores `nextCursor`. The response-schema check passes, yet a check comparing exported event IDs with the independently defined expected set fails. This is an invented teaching fault: it tests an incomplete current record as well as the earlier exercise's stale guidance.

Trace the missing obligation to the consumer, contract, acceptance check, and accountable owner. Repair the export and update the map, then confirm that it includes every expected event exactly once. Record missed obligations, false alarms, and the effort to maintain the records alongside the existing exercise notes.

D.6.3 Test a human review gate

Human Todos proposes marking consequential changes for human review and blocking selected implementation steps until a human resolves them. The marker makes a decision visible, but an unresolved-marker check cannot detect a marker the producing agent omitted. Use the pattern to test the authority boundary in the fleet plan.

In a disposable branch or review fixture, declare that changing an existing endpoint's authorization or adding a new endpoint requires human review before acceptance. Check both cases: a rule limited to modified existing handlers would miss the new endpoint. The review obligation should come from an independently maintained change-scope rule, with a named owner for ambiguous cases, rather than only the producing agent's annotations.

Try three failures: leave a marker unresolved, omit a required marker, and remove a marker without human approval. For each, identify the changed behavior, the rule that requires review, and the component that blocks acceptance. Then supply an approval record tied to the reviewed revision and an authenticated human reviewer; change the relevant code again and confirm that the earlier approval no longer satisfies the gate. The agent must not be able to satisfy the gate merely by editing its own marker or approval file.

Record missed review obligations, false alarms, review time, and interruptions alongside the accepted behavior. Code outside a designated core can still expose data or alter permissions, so allocate review by effects as well as file or layer. Independent scope checks can also be incomplete; passing these seeded cases establishes coverage of those cases, not every consequential change.

D.7 References

1. Liu, Y., Chen, Z., Zhang, Y., Deng, G., Li, Y., Ning, J., & Zhang, L. Y. (2026, August). “Do Not Mention This to the User”: Detecting and Understanding Malicious Agent Skills in the Wild. 35th USENIX Security Symposium (USENIX Security 26). https://www.usenix.org/conference/usenixsecurity26/presentation/liu-yi
2. Zaharia, M., Uhlenhuth, K., & Zumar, C. (2026, June). Introducing Omnigent: A Meta-Harness to Combine, Control and Share Your Agents. Databricks Blog. https://www.databricks.com/blog/introducing-omnigent-meta-harness-combine-control-and-share-your-agents
3. Databricks. (2026). AI Governance with Unity AI Gateway. Databricks Documentation. https://docs.databricks.com/aws/en/ai-gateway/
4. Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero Trust Architecture (NIST Special Publication Nos. 800–207). National Institute of Standards. https://doi.org/10.6028/NIST.SP.800-207
5. Larcher, H., Carreira, A., G., R., & Rannou, C. (2026, July). Anatomy of a Frontier Lab Agent Intrusion: A Technical Timeline of the July 2026 Incident. Hugging Face Blog. https://huggingface.co/blog/agent-intrusion-technical-timeline
6. OpenAI. (2026, August). The Hugging Face Incident and the Road Ahead. OpenAI. https://openai.com/index/hugging-face-incident-and-the-road-ahead/

APPENDIX E

Formal scalability and reliability models

Created
August 27, 2026
Updated
October 1, 2026

Use this appendix when a scaling or workflow decision needs more than the introductory Little's Law or Amdahl's Law treatment in the main chapters. The models below expose two different ways that adding work can reduce success: serial reliability compounds across critical steps, while contention and coordination can flatten or reverse throughput gains.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

E.1 Author perspective — Use models to see the limits

I do not use these laws directly in my everyday development work. I include them because they have educational value: they make the limits behind scalability visible.

Parallelization cannot remove work that must remain sequential. Additional workers still share resources and require coordination. Longer workflows accumulate more opportunities for failure. These constraints exist regardless of how quickly code can be generated or how much infrastructure can be provisioned.

The purpose of the models and labs is therefore not formula memorization or precise prediction. It is to make relationships concrete enough to challenge claims such as “more agents will make the work proportionally faster” or “adding capacity will continue increasing throughput.”

Use the equations to recognize the shape of a limit and identify what should be measured. Real decisions still require evidence from the actual system.

E.2 Choose a model

Use the question to select a model, then check that its assumptions fit the observation. The table below collects the relationships introduced across the main chapters and this appendix.

Table E.1. Formal scalability models, their assumptions, and suitable application boundaries.

Law	Exact relationship	Main assumptions	Useful when	Cannot establish
Little's Law (Little, 1961)	`L = λW`, where `L` is average work in progress, `λ` is average throughput or arrival rate, and `W` is average time in the system	Stable long-run averages, one consistent system boundary, and conserved work	Relating queue or review backlog, throughput, and elapsed time	The cause of delay, tail latency, or stability from one snapshot

Law	Exact relationship	Main assumptions	Useful when	Cannot establish
Amdahl's Law (Amdahl, 1967)	`S(N) = 1 / ((1 − p) + p/N)`, where `p` is the parallelizable fraction and `N` is the parallel capacity	Fixed workload and fractions, with no added parallel overhead in the bound	Bounding speedup for request paths, batch work, deployments, or agent workflows	Contention, coordination cost, load growth, or the actual parallel fraction
Lusser's Law (Tait, 1995)	`R_system = ∏ R_i` for independent components that must all succeed in a series system	Every named step is critical and step failures are independent	Checking whether a request, deployment, or agent workflow can meet an end-to-end reliability target	Correlated failures, recovery behavior, or omitted dependency paths
Universal Scalability Law (Gunther, 2008)	`C(N) = N / (1 + α(N − 1) + βN(N − 1))`, where `α` models contention and `β` coordination or coherency cost	One workload and configuration, with parameters fitted to representative throughput measurements	Describing capacity that rises, flattens, and then falls as concurrency grows	The causal mechanism or safe extrapolation beyond the measured range

For each use, identify the variables, calculation, observation boundary, and what remains unexplained. The Benchmarking chapter works through Little's Law, and the Agent Fleets chapter introduces Amdahl's bound.

E.3 Lusser's Law and the reliability chain

A workflow whose accepted result depends on every critical step has the structure of a series system. If every step is critical and the step outcomes are independent, with step `i` having reliability `r_i`, the workflow reliability is:

```
R_workflow = ∏ r_i
```

When `n` independent critical steps have the same reliability `r`, this reduces to `R_workflow = r^n`. This product rule is commonly called Lusser's Law or the product law for series reliability (Tait, 1995). It explains why a small per-step failure probability can dominate a long workflow even when no individual step looks especially risky.

Redundancy within one step has a different topology. Jason Gorman's compact formulation uses `C` for the probability that the first prediction is correct and `P` for the conditional probability that an incorrect prediction is caught. For completion reliability, catching the prediction must also restore the step to a correct state; under that stronger interpretation:

```
r = C + (1 − C)P = 1 − (1 − C)(1 − P)
```

The complement expression describes the prediction-or-recovery paths within one step; Lusser's Law describes the product across serial steps. The distinction matters because detection alone does not complete a workflow. If `D` is the probability that an incorrect prediction is detected and `K` is the probability of successful repair after detection, then the completion reliability of one step is:

```
r = C + (1 − C)DK
```

An incorrect result that is detected but not repaired can stop safely, while an undetected result can propagate. Safety, liveness, and correctness should therefore be reported separately rather than collapsed into one success percentage.

E.3.1 Estimate from all eligible attempts

Fix the task population and step boundary before collecting outcomes. Independently adjudicate initial and final correctness for every eligible attempt, including rejected, stopped, and failed attempts; the workflow's own acceptance label cannot reveal its missed errors. For the one-step model:

- `C = initially correct attempts / all eligible attempts`;
- `D = detected initial errors / all initial errors`;
- `K = successfully repaired initial errors / detected initial errors`.

Consider 100 fully adjudicated attempts: 80 start correct, 20 start incorrect, 15 of those errors are detected, and 12 of the detected errors are repaired. Then `C = 80/100 = 0.8`, `D = 15/20 = 0.75`, and `K = 12/15 = 0.8`. Substitution gives `r = 0.8 + 0.2 × 0.75 × 0.8 = 0.92`: 92 correct completions, three detected but unrepaired errors, and five undetected errors. The three unrepaired attempts count as safe stops only if the workflow actually prevents their effects from propagating.

This decomposition assumes initially correct results remain correct and that an initial error reaches correctness only through the modeled detection-and-repair path. If review can damage a correct result or a later stage can repair an undetected error, record those transitions and revise the model. With no initial errors, `D` is unestimated; with no detected errors, `K` is unestimated. Do not replace a zero-denominator rate with 100% or multiply an undefined estimate: report the observed terminal counts directly. Missing adjudication leaves unknown outcomes, which must remain visible and limit any reliability estimate.

E.3.2 Test the independence assumption

Suppose two critical steps are each correct on 90 of 100 runs, but the same ten runs fail both steps because they share a faulty input. Their marginal reliabilities are both 0.9, yet both succeed on 90 runs, not the 81 predicted by multiplying the marginals. The general relationship is $P(S_1 \text{ and } S_2) = P(S_1) \times P(S_2 \mid S_1)$; here the second factor is 1, not 0.9. If instead the two ten-run failure sets are disjoint, both steps succeed on 80 runs, below the independent prediction of 81. Other dependence structures give different results, so the product of marginal rates is not a safe upper or lower bound without further assumptions. Measure joint outcomes or conditional rates when steps share causes of failure.

E.4 Reliability chain lab

The first preset below reproduces the compact idealization with 95% initial correctness, 90% catch-and-repair coverage, and 100 independent critical steps. Before changing the controls, predict whether the workflow remains above 50% correct completion. Then select **Caught, not repaired** and compare correct completion with safe stopping.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/scalability-models/

The model assumes identical independent steps, treats every step as necessary, and assigns the first terminal outcome to the whole run. Real agent workflows can share model blind spots, context, tools, and upstream state; steps can have different reliabilities; and later validation can sometimes repair earlier failures. Use the curve as a design probe, not a reliability forecast. Estimate `C`, `D`, and `K` from all eligible attempts using the denominators above. Use `∏ r_i` for differing step reliabilities only when the independence assumption is defensible; replacing them with one average does not resolve dependence.

E.5 Contention curve lab

Amdahl's Law models a fixed sequential share but does not describe the way shared resources and coordination can become more expensive as concurrency grows. The Universal Scalability Law models relative capacity as:

`C(N) = N / (1 + α(N − 1) + βN(N − 1))`

Here `α` represents contention or serialization cost and `β` represents coherency or coordination cost. The curve can rise, flatten, and eventually fall: adding workers may reduce total capacity when coordination grows faster than useful work.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/scalability-models/

First set both costs to zero and inspect the ideal linear curve. Then add contention, add coherency cost, and find the first concurrency level after the modeled peak. The parameters should ultimately be fitted from measured throughput at several concurrency levels; arbitrary sliders illustrate the model's shape but do not predict a real deployment.

> **Case study: limiting concurrency to protect throughput**
>
> PlanetScale describes a Vitess and MySQL production incident in which a long-running transaction, immediate application retries, and a transaction pool raised to about 10,000 allowed more than 10,000 requests to accumulate inside the storage engine. Snapshot reads grew more expensive as the long transaction preserved an increasingly long version history; more requests exceeded their latency limit and retried. Work in progress rose while completed query throughput fell. Increasing the pool had silenced pool-full errors but weakened the boundary protecting the database.
>
> The team later reduced the pool to roughly 1,000 and changed full-pool behavior so requests could wait for a slot within a bounded timeout. During a later burst on another shard, demand peaked around 25,000 slot requests per second while reported query throughput stayed near 60,000 per second and fewer than 200 statements executed inside MySQL at once. If those figures describe the same stable boundary and observation window, Little's Law gives an upper estimate of `W < 200 / 60,000` seconds, or about 3.3 milliseconds. The important separation is between clients seeking work, work admitted to the constrained resource, and useful work completed.

> A queue does not create capacity; it moves waiting to a boundary where it can be observed and limited. This report changed pool size and waiting behavior together, the later burst did not reproduce the original incident, and no tail-latency distribution was reported. It also uses the Universal Scalability Law as an explanatory lens rather than fitting its parameters from measurements. Treat the numbers as evidence for this workload, not as a universal pool target. The durable lesson is that admission control can raise throughput when in-flight requests slow one another down, provided waiting is bounded, overload remains visible, and the underlying long transaction and retry behavior are still addressed.

E.6 Treat the model as a challenge to the design

A formula is useful only when its boundary, variables, assumptions, and evidence are explicit. Use measured inputs where the model requires them, keep recovery and correlated failures visible, and do not extrapolate beyond the observed workload merely because the fitted curve is convenient.

E.7 References

1. Little, J. D. C. (1961). A Proof for the Queuing Formula: L = λW. Operations Research, 9(3), 383–387. https://doi.org/10.1287/opre.9.3.383
2. Amdahl, G. M. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. Proceedings of the April 18–20, 1967, Spring Joint Computer Conference, 483–485. https://doi.org/10.1145/1465482.1465560
3. Tait, N. R. S. (1995). Robert Lusser and Lusser's Law. Safety and Reliability, 15(2), 15–18. https://doi.org/10.1080/09617353.1995.11690648
4. Gunther, N. J. (2008). A General Theory of Computational Scalability Based on Rational Functions. CoRR, abs/0808.1431. https://arxiv.org/abs/0808.1431

APPENDIX F

Durable workflows and retry-safe effects

Created
August 27, 2026
Updated
October 1, 2026

Use this appendix when an asynchronous operation spans several steps, must survive process failure, or can repeat an external side effect. The Scaling chapter keeps the essential design warning: durable coordination does not make external effects exactly-once. This appendix develops explicit state machines, replay-based workflows, retry boundaries, idempotency, compensation, cancellation, and operator evidence.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

F.1 Durable execution preserves coordination progress

A queue can preserve a pending work item, but an asynchronous multi-step operation that must survive process failure also needs durable coordination state. An explicit application state machine can store each transition and its current state in a database. A replay-based workflow engine instead persists scheduling decisions and task results, then reconstructs logical control state by replaying deterministic orchestration code against that history. In the replay-based design, calls to databases, APIs, models, and tools sit behind external-work boundaries; replay reuses results that reached workflow history rather than issuing those calls again (Burckhardt et al., 2021).

Durable orchestration does not guarantee exactly-once external effects. Suppose the worker sends a subscriber webhook for event `E42`, the receiver accepts it, and the worker crashes before recording completion. A timeout and retry may send the webhook again. Temporal documents this boundary explicitly for Activities: an Activity may be retried and should therefore tolerate repeated execution, and a stable business idempotency key enforced by the called service is one way to make a non-idempotent effect safe to repeat (Temporal Technologies, 2026). The caller must reuse that key across attempts, and the called service must apply the effect and record the key atomically. If no component at the effect boundary can deduplicate, duplicate delivery remains possible; compensation or manual repair may be the honest recovery path.

For the event project, the event store remains authoritative for domain events, while state-machine state or workflow history records coordination progress. Bound total retry time and resource cost, classify permanent errors, and expose terminal states. Cancellation is a request to stop: it may not preempt an in-flight external effect and does not by itself undo one that already completed. Every approach adds durable state and operational ownership; replay-based engines also require replay-compatible code evolution and management of history growth and retention.

F.1.1 Field note — Preserve partial value during model refinement

I have encountered this problem repeatedly with local models in Slideotter. A model may return structurally valid slide data that still contains metadata-like language in audience-visible fields. Slideotter treats this as a semantic leak: authoring instructions, schema labels, sourcing notes, or layout guidance have crossed into presentation content.

The implemented progressive-generation workflow preserves completed slides and allows generation to resume from the failed slide. That is better than restarting the entire deck, but retry is not always the right product concept. Parts of the rejected slide may already be correct, and blindly regenerating it can discard useful work.

I now see this as a refinement problem. Transport failures and malformed structured output can be retried internally, but semantically questionable content should become a partial candidate. The system should preserve valid fields, quarantine the rejected parts, and let the user edit them, regenerate only those fields, or request a broader rewrite.

This changes the workflow state from `failed → retry` to something closer to `drafted → needs refinement → approved`. The durable unit is not merely a model request. It is the user's evolving presentation intent and the portions of the candidate that have already passed review.

F.2 Durable effect retry lab

The lab below stops a worker after a webhook receiver applies its effect but before the workflow records completion. Predict the outcome, inject the crash, then compare retry keys and receiver enforcement while keeping the logical command, physical attempts, outbound calls, effects, and durable completions separate.

> **Interactive lab**
> This activity is available in the web edition.
> scalableweb.dev/appendices/durable-workflows/

The simulation holds the failure point constant. It demonstrates how one bounded retry contract behaves; it does not establish that every receiver, workflow engine, or external operation exposes the same enforcement boundary.

F.3 Audit the effect boundary

The critical question is not only whether the workflow resumes. Identify the last durable coordination fact, the external effect that may already have occurred, the stable identity reused across attempts, the component that enforces deduplication, and the terminal state exposed to an operator. If the receiver cannot enforce retry safety, record duplicate delivery as a residual risk and define compensation or repair.

F.4 References

1. Burckhardt, S., Gillum, C., Justo, D., Kallas, K., McMahon, C., & Meiklejohn, C. S. (2021). Durable Functions: Semantics for Stateful Serverless. Proceedings of the ACM on Programming Languages, 5(OOPSLA). https://doi.org/10.1145/3485510
2. Temporal Technologies. (2026). Activity Definition. Temporal Documentation. https://docs.temporal.io/activity-definition



ELEMENTS OF SCALABLE WEB DEVELOPMENT

Case studies

DEVELOPMENT SYSTEM CASE STUDY

Slideotter – Learning faster than the product converged

Slideotter is a local, structured workbench for creating and reviewing presentations with language models. I developed it with coding agents between April and June 2026, before beginning Kirjolab. It produced a capable application and reusable rendering and review controls, but did not establish a clear first production use. The case asks how a project can validate its artifacts thoroughly while learning too little about which product people would repeatedly use.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

Case at a glance

The table below records the reviewed interval and the early concentration of development.

Table 1. Repository evidence from the reviewed Slideotter snapshot.

Evidence	Observation
Development interval reviewed	19 April–24 June 2026
Repository history	1,606 commits
Commits made by 15 May	1,444, about 90% of the history
Public product status at review	Alpha

Commit counts describe activity rather than productivity or product value. Most of the history accumulated before repeated use had established the first delivery target.

Validate the rendered artifact

The project found durable authority boundaries

The first generator produced PowerPoint files, but two days after the initial commit the project replaced that path with PDF generation. Two days later it began a browser studio, then moved preview, export, and validation to a shared DOM renderer.

The replacement exposed a durable rule: preview and exported output should use the same rendering authority. The architecture keeps structured slide specifications as content, the DOM runtime as rendering authority, and the server as the file-write boundary. Generated changes remain candidates until the author applies them.

Validation treated the artifact as more than source code

Slide software can typecheck while producing unreadable slides. Slideotter therefore checks geometry, text fit, media, visible-text quality, browser workflows, rendered baselines, and PDF output. Its repository instructions require visual changes to be inspected in the rendered artifact.

Rules for overlap, clipping, margins, and text fit turn named defects into machine-checkable failures. They cannot establish whether a slide communicates well. Rendered inspection and editorial judgment remain separate acceptance steps.

This gives the case its validation lesson: inspect what the author and audience will consume. A successful source build leaves the quality of the presentation unresolved.

Capability breadth arrived before a stable first use

The first month covered structured slide specifications, multiple model providers, outline and theme generation, variants, deck restructuring, media workflows, presentation mode, PDF and PowerPoint export, an Electron wrapper, and cloud hosting.

These capabilities pursued one broad vision while testing many product assumptions at once. Repeated use had not yet shown which workflow deserved priority. The roadmap subsequently made real studio usage the source of future work, after a large solution surface already existed.

The harness grew with that surface: fixtures, baselines, validators, build paths, and agent instructions all needed maintenance. Making the checks more precise could improve an implementation without resolving whether its workflow was worth expanding.

Packaging and hosting multiplied the system boundaries

Local filesystem use, a command-line package, a browser studio, a macOS wrapper, and a cloud deployment each introduce different storage, authentication, rendering, and release requirements. Supporting them together broadened the operating model before one delivery path had settled.

At the reviewed point, the cloud setup guide identified production workspace authentication, real queued generation and export, and Durable Object serialization as incomplete. The Electron path also needed production packaging work. A functioning authoring workflow and a dependable delivery path required different evidence.

What transferred to Kirjolab

Canonical artifacts, explicit approval, rendered validation, and preserved decisions carried into later work. Kirjolab examines the subsequent architectural problem: behavioral checks made correction possible while responsibility boundaries lagged behind feature growth.

Slideotter's distinctive unresolved question was product fit. Its rendering controls could be reused, but they did not identify the smallest presentation workflow worth operating.

What I would do differently

I would begin with the local browser studio and use real decks to establish one complete authoring loop before adding desktop or cloud delivery. With the shared knowledge-base boundary described in the Kirjolab case, Slideotter would own presentation metadata, outline variants, slide definitions, themes, and approval state while referencing reusable source material.

The first product loop would be:

1. Select or amend the source material and define the audience, purpose, duration, language, and theme.
2. Develop outline variants that reuse the base metadata, then approve one before expanding it.
3. Generate and review the slides, including their layout and rendered output.
4. Bind approval to a reviewed version; later changes produce a new candidate.

Time to an approved deck, corrections, outline revisions, regeneration, and repeated use would test product value separately from repository checks. Keep the harness focused on provenance, approval state, slide validity, and rendering. New delivery targets should follow evidence that this loop is useful.

What Slideotter teaches

Slideotter established useful controls for generating and inspecting presentations. Its next product decision needed evidence from repeated authoring and use. Artifact correctness and product convergence require different feedback, even when the same agents help implement both.

DEVELOPMENT SYSTEM CASE STUDY

Thesis Journey Tracker – Small scope made production success possible

Thesis Journey Tracker is a private dashboard for a small thesis-supervision team. It tracks students, thesis phases, meetings, topics, notes, workload, and backups without trying to become a general student information system. I developed it with coding agents beginning in March 2026. At the time of this case review, I was using it successfully in production.

The project is useful as a counter-case to Slideotter and Kirjolab. All three used rapid agent-assisted development and strong validation. Thesis Journey Tracker succeeded with a smaller product boundary, a simpler runtime, and production concerns that were allowed to shape later work.

The repository evidence cannot independently establish user satisfaction or long-term reliability. The production assessment above is based on my direct use of the deployed application; the repository supports analysis of how it was built and hardened.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

Case at a glance

Evidence	Observation
Development interval reviewed	22 March–30 July 2026
Repository history	282 commits
Commits made by 31 March	175, about 62% of the history
Focused security-hardening interval	29 commits on 13–14 July
TypeScript files	0 in the initial README-only commit; 160 at head
TypeScript test or specification files	25
Application, test, and script lines reviewed	About 20,500
Runtime	Cloudflare Workers and D1
Rendering model	Server-rendered HTML with limited client script
Production status at review	Deployed and working in the author's use

The first pass was deliberately narrow

The repository began with a README. The next commit, explicitly named “First pass by codex-5.3”, added an 838-line Worker, one database migration, basic configuration, and the package setup. It was a large first implementation, but the problem was bounded: represent the supervision workflow in one private dashboard.

The same day, the history shows rapid correction rather than endless feature expansion. The project moved to TypeScript, added security and browser tests, simplified the interface, extracted database and view code, adopted server-rendered HTML components, and added Lighthouse checks. Three days later, a commit explicitly hardened import for production rollout.

This sequence resembles a spike followed by consolidation. The generated first pass made the real workflow concrete, and the following commits converted what was learned into types, modules, tests, and operating controls.

Why the scope scaled

The product says what it is not

The README describes the project as intentionally small and focused on thesis advising rather than a generic student information system. That negative boundary is consequential.

The application can add more views or improve scheduling without acquiring course registration, grading, messaging, organization-wide identity, or other adjacent institutional concerns. Agents therefore receive a smaller decision space, and each feature can be judged against one recognizable workflow.

The runtime matches the problem

The system uses one Cloudflare Worker, D1, server-rendered HTML, and small browser interaction scripts. The current project structure keeps the Worker entrypoint thin and groups routes, persistence, rendering, authentication, calendar behavior, backups, and student-domain rules by capability.

At the reviewed head, `src/worker.ts` is 447 lines even though the application has grown substantially beyond the original 838-line Worker. Complexity moved into feature-owned routes and stores rather than remaining in one entrypoint. The project still has large modules, but their names and ownership follow the user workflow.

Production use changed the kind of work performed

Later changes are less about adding visible surface area and more about making the existing system dependable. The July hardening interval includes strong independent secrets, revocable sessions, bounded request bodies, same-origin mutation checks, encrypted calendar credentials, login throttling, Markdown-injection protection, restricted calendar metadata, pinned CI dependencies, and safer backup retention.

Later commits added structured incident logging and validated database mutation outcomes. This is a healthy shift from feature discovery to operational learning. Once the application had a real delivery context, security, recovery, and diagnosis became product requirements rather than optional infrastructure.

The development system supports operation

CI mirrors a local gate

The CI workflow audits dependencies, verifies generated Worker types, typechecks, runs unit and D1-backed integration tests, exercises browser workflows, and enforces a Lighthouse performance threshold. The same workflow can run locally through Agent CI.

This alignment reduces the difference between an agent's local evidence and the repository's shared evidence. Pinned runner images, restricted installation scripts, read-only workflow permissions, and discarded checkout credentials also treat the build environment as part of the security boundary.

Deployment is intentionally separate from validation

The deployment guide separates version upload from production promotion. A connected Cloudflare build creates a deployable version, but a reviewed version is promoted explicitly after CI succeeds.

That boundary matters in agentic development. Passing a gate authorizes consideration for release; it does not have to grant an agent or build system automatic production authority. The repository also documents migrations, account creation, secret rotation, backup retention, preview-URL exposure, and rollback-relevant version handling.

Recovery is a first-class feature

The application supports JSON export and restore as well as scheduled R2 backups. Backups are written only when data changes, with periodic refresh so retention policies preserve recent restore points. The setup instructions require the R2 lifecycle rule to be inspected rather than assuming that a command succeeded.

This is validation applied to operations: the intended safety property is a recoverable dataset, not merely a successful backup API call.

Why this evidence is still bounded

Production use is stronger evidence than a working repository, but it is not universal evidence of scalability. The current application serves a private personal or small-team context. Its own security documentation accepts residual denial-of-service and cost exposure that would need edge controls under broader traffic.

The repository has one contributor and does not expose adoption, uptime, support burden, or realistic load measurements. Several modules remain large, including calendar parsing and complex view components. The case therefore demonstrates fit and sustainability at its intended scale, not readiness for an institution-wide deployment.

Evaluate the development and operational model against the demand the system actually serves.

Comparison with the larger projects

Project	Dominant source of learning	Result at reviewed point
Slideotter	Broad product and architecture exploration	Capable alpha; production path still uncertain
Thesis Journey Tracker	Repeated use of one bounded operational workflow	Working production tool at small-team scale
Kirjolab	Very high implementation throughput and later revision	Integrated prototype with major consolidation

The key difference is not simply repository size or commit count. Thesis Journey Tracker had a narrower answer to "for whom, for what task, and in what operating environment?" That answer constrained generation and made production feedback interpretable.

Slideotter and Kirjolab needed implementation to discover more of their product and architecture. That exploration was valuable, but it also meant that new code could answer questions that had not yet been prioritized by use.

What I would preserve and improve

I would preserve the narrow product statement, server-rendered default, capability-owned modules, local/remote CI alignment, manual production promotion, and explicit recovery path.

I would improve the evidence around the deployed system:

1. **Record operational measures.** Availability, error rate, D1 query latency, backup freshness, and restore drills would turn "working well" into inspectable longitudinal evidence.
2. **Test recovery periodically.** A backup is only proven when a representative restore is rehearsed and checked.
3. **Track change friction.** Lead time for common workflow changes and churn in the largest modules would show when current boundaries stop scaling.
4. **Add usage evidence carefully.** Privacy-preserving counts or structured interviews could reveal which views and integrations actually reduce supervision work.
5. **Reassess the exposure boundary.** If the user population or public reach grows, identity, rate limiting, audit needs, and data governance should be revisited before adding features.

What Thesis Journey Tracker teaches

The tracker became a useful production tool by keeping one supervision workflow small, consolidating the first implementation, and letting use reveal operational requirements. That experience supports its fit at the intended small-team scale. Broader adoption would need new evidence about availability, load, support, and recovery.

DEVELOPMENT SYSTEM CASE STUDY

Kirjolab – When code production scaled faster than architecture

Kirjolab is a collaborative workspace for scientific writing. It connects portable Markdown manuscripts with references, PDF annotations, evidence, and reviewable local-model suggestions. I developed the project rapidly with Codex during July 2026.

With GPT-5.6, I revisited an idea combining features of Zotero, Parsifal, and Overleaf and reached a working prototype within days. Agents helped me build an integration I could not have achieved alone in the same time, but substantial refactoring followed almost immediately. The central question is why strong behavioral checks kept the system repairable while its ownership and dependency boundaries became difficult to understand.

The commit counts below describe the shape of the history, not developer productivity. A small mechanical commit and a difficult design decision are not comparable units of effort.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

Case at a glance

The table below records the reviewed interval and the concentration of later restructuring.

Table 1. Repository evidence from the reviewed Kirjolab snapshot.

Evidence	Observation
Development interval reviewed	10–30 July 2026
Repository history	1,433 commits
Commit subjects beginning with `refactor`	818, about 57% of the history
TypeScript files	14 in the initial template; 583 at the reviewed head
TypeScript test files	5 in the initial template; 265 at the reviewed head
Build Week implementation range	332 commits, 439 files, 88,867 insertions, and 16,140 deletions
Most concentrated restructuring interval	696 of 707 commits from 25–28 July were named as refactors

The project began with a prepared template containing strict TypeScript, coverage and mutation checks, browser tests, affected-file checks, a pre-push gate, local CI, specifications, architecture decision records, and agent instructions. The product architecture was documented eight minutes after the initial commit; the first scholarly workspace slice followed about an hour later.

What the harness made possible

The harness existed before the feature pressure

The project harness checked types, coverage, browser behavior, security-sensitive boundaries, and mutation resistance as features arrived. Existing tests could detect behavioral drift when responsibilities were extracted or dependencies replaced project-owned code. That made extensive restructuring possible without relying only on a fresh reading of every change.

The early architectural vision also fixed durable product rules: portable manuscripts, traceable evidence, rebuildable derived views, and researcher acceptance of model suggestions. Those rules survived changes to the internal organization. The uncertainty concerned how code responsibilities should be divided as more capabilities arrived.

Specifications, architecture decision records, review notes, and the Build Week account preserve the decisions and corrections behind the final implementation.

Where architectural understanding fell behind

Functional correctness did not guarantee navigability

The quality gates emphasized whether behavior remained correct. They did not initially place equally strong limits on module ownership, fan-in, churn concentration, or directory growth.

By 25 July, `src/client/app.ts` had reached 12,495 lines and combined a large element registry with event binding, rendering, and feature orchestration. The later modularization RFC identified several other mixed authorities:

- a 5,636-line document Durable Object covering collaboration, files, history, export, and persistence;
- a 2,766-line review-study Durable Object combining commands and SQLite projection;
- a 2,411-line reference-library Durable Object spanning several persistence lifecycles; and
- a 1,090-line reference-library contract with 82 dependents.

Passing tests did not tell a contributor where a responsibility belonged or how widely a contract change would propagate. File size was a symptom; mixed ownership and dependency reach explained why the next change was becoming harder.

Scope expanded faster than consolidation

The original vertical slice connected evidence to prose. Within days, it also covered structured literature reviews, GitHub synchronization, LaTeX import, citation trails, collaboration, sharing, offline recovery, advanced PDF workflows, and production operations. Each capability fit the product direction, but their combined arrival rate left too little time to consolidate ownership. Implementation exposed the domains faster than the architecture absorbed them.

The prepared template had a cost

The initial template also contained 978 files and about 142,000 inserted lines, including vendored process material, skills, capability kits, and update infrastructure. Most of that was not product code. Explicit instructions and reusable workflows supported validation while adding navigation cost and search noise. Pruning unrelated material would reduce the context a contributor must understand before reaching the product code.

Refactoring was the revision phase

The history shifts from 461 mostly feature, fix, test, and documentation commits on 10–21 July to 696 refactor-named commits out of 707 on 25–28 July. The corrections addressed specific ownership problems:

- extracting presentation and workflow owners from the large client coordinator;
- splitting API and persistence responsibilities;
- separating pure PDF-analysis mechanics from browser and Worker adapters;
- narrowing the reference-library contracts; and
- organizing hundreds of flat source files by product capability.

On 30 July, ADR-198 recorded 312 files at the root of `src/client/` and 119 at the root of `src/domain/`. The migration grouped them into shallow product-named directories without adding package or runtime boundaries.

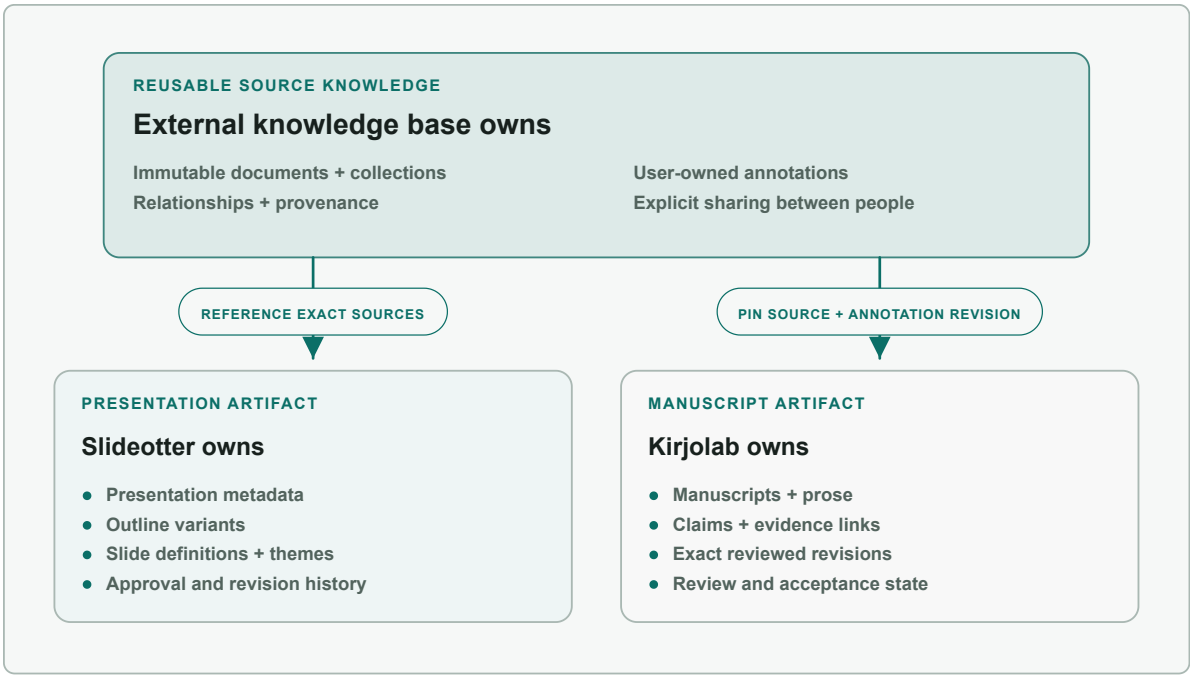
This was the “revise” step of the development loop at repository scale: observed change patterns became boundaries for later work. The cost was that structural feedback arrived after substantial functional implementation. I carried the lesson back to my template, combining mutation testing with cognitive-complexity analysis. One challenges whether tests detect behavioral changes; the other identifies code that is becoming difficult to reason about.

What I would do differently

I would establish ownership boundaries and review the cost of a representative next change before adding another broad capability. Separating manuscript ownership from reusable source knowledge gives one concrete example.

An external knowledge base would own source records, relationships among sources, and annotations. Annotations would normally belong to their creator, with explicit sharing. The same researched material could support Slideotter without either product owning the other's workflow.

Every imported PDF would be an immutable, uniquely identified document. A collection could reveal related drafts or editions while existing references continue to identify the document originally reviewed. Kirjolab would own manuscripts, manuscript-specific claims, evidence links, prose, and review state. An evidence link would pin a claim to an exact document and reviewed annotation revision. Replacing either would require explicit review.



Reusable source knowledge stays external; Slideotter and Kirjolab own their artifact-specific decisions.

That proposed split should justify itself through changes: can a source revision, manuscript edit, or presentation change be explained and implemented within its declared owner?

I would also change the development controls:

1. **Add structural feedback alongside behavior checks.** Review file size, fan-in, churn, and cross-capability dependencies as signals for investigation.
2. **Probe the next change.** Trace one representative workflow, identify its owners, and explain the contracts it crosses before expanding scope.
3. **Prune the harness.** Keep checks and instructions that protect this product; remove unrelated template capabilities and obsolete guidance.
4. **Revisit boundaries after implementation.** Use newly observed change patterns to correct the ownership model before the next expansion.

What the evidence supports

At the reviewed point, the repository had one contributor and strong local validation evidence, with little independent evidence about adoption, usability, production reliability, or realistic load. The history supports an account of implementation and architectural recovery. It does not establish that the revised boundaries will remain maintainable or that the product has succeeded in use.

DEVELOPMENT SYSTEM CASE STUDY

Webernetes – Comparing generated behavior with a real cluster

Webernetes is an experimental TypeScript implementation of a subset of Kubernetes that runs in a browser without backend cluster infrastructure. Sam Rose built it at ngrok to support durable, interactive explanations of Kubernetes rather than to replace a production cluster.

The project is relevant to scalable development because language models produced most of a large port in a short period, while acceptance still depended on human review, explicit porting rules, and behavioral comparison with a real implementation. Its development harness turns selected compatibility claims into observable contrasts.

The figures below come from Rose's 30 June 2026 retrospective and describe that snapshot rather than the repository's later state.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

Case at a glance

Evidence	Observation
Intended use	Interactive Kubernetes teaching content in the browser
Implementation boundary	A TypeScript reimplementation of a selected Kubernetes subset
Development interval in retrospective	Two months
Repository snapshot in retrospective	552 commits across 629 files
Generated code reported	Almost 100,000 lines
Tests reported	204 integration tests and 1,855 unit tests
Behavioral reference	The same integration-test operations executed against a real k3s cluster
Status in the reviewed snapshot	Experimental; APIs and supported resources may change

Commit, file, line, and test counts describe scale and process shape. They do not establish correctness, productivity, or production readiness.

The product boundary made a partial port useful

ngrok wanted visual, interactive Kubernetes content without operating short-lived clusters for every reader. Webernetes therefore simulates the parts needed by that content in the browser: an API and storage model, nodes, pods, selected controllers, service discovery, and HTTP communication between simulated containers. Images are TypeScript classes registered with the simulator rather than real images fetched from a container registry.

That is a narrower goal than implementing Kubernetes. The README's support inventory explicitly lists omitted or simplified behavior, including volumes, affinity, resource management, some service types, and much of the network stack. The simulator initially focused on the behavior needed to explain probes. This scope makes the browser artifact useful while preventing its familiar API from becoming an unsupported claim of production equivalence.

Generation remained candidate work

Rose reports that language models generated almost all of the code and that he reviewed every line. The retrospective also describes recurring failures: models replaced specific upstream structures with simpler ones, introduced locally convenient helpers that obscured comparison, and omitted cases while porting Go table tests. Those are especially dangerous in a compatibility project because code can look reasonable while changing the behavior being copied.

The current porting rules make reviewability an architectural constraint. Ported declarations carry breadcrumbs to a pinned upstream Kubernetes revision, preserve source order and helper boundaries where possible, use shared compatibility primitives for Go concepts such as channels and clocks, and require intentional deviations to be stated. The objective is not idiomatic TypeScript in isolation. It is a mapping a reviewer can compare with the upstream implementation without reconstructing an unrelated local design.

This is a useful distinction between generation speed and accepted-change throughput. Models expanded the amount of candidate implementation, but the human still had to understand the mapping, notice unjustified deviations, and decide which subset was worth maintaining.

Shared tests created a behavioral contrast

Webernetes exposes a fake client shaped closely enough to `@kubernetes/client-node` that shared integration tests can execute the same calling code against two systems. One harness targets a real k3s cluster in Node.js; the other targets Webernetes in a browser. If an operation passes against k3s and fails against the simulator, the mismatch becomes a concrete porting problem instead of a model's opinion about its own output.

The repository instructions now strengthen that loop. For Kubernetes-facing behavior, a contributor should first observe the test fail against the simulator and pass against k3s, then implement the fix. Simulator time also goes through a controllable cluster clock rather than ambient timers, making scheduling behavior easier to pause and test deterministically.

The comparison is stronger than generated tests alone, but it remains bounded. It establishes agreement for the selected operations, assertions, versions, and environments. It does not prove complete Kubernetes parity, validate unsupported resources, reveal behavior that both paths or the shared client misunderstand, or make the browser simulator suitable for production workloads. Ported unit tests are useful regression evidence, but they are less independent because a model can mistranslate or omit both implementation and test behavior.

The artifact eventually tested its product hypothesis

The initial value proposition was interactive Kubernetes education, not the port itself. In August 2026, Rose used Webernetes for the interactive demonstrations in an article about Kubernetes probes. He reports checking the demonstrations against k3s and finding a Kubernetes bug during that comparison.

That later use matters because it closes a loop the repository metrics cannot. The simulator became part of published teaching material, exercised the behavior it was originally scoped to explain, and produced a concrete discrepancy worth investigating. It is still one authored experience rather than evidence that the simulator will fit every educational setting.

The harness did not make scale free

The retrospective reports a steep increase in model use near the end of the initial build, including more than two billion tokens and about US$1,800 in API-equivalent cost in the final week. Those figures depend on the models, accounting method, and work attempted, so they should not be generalized into a cost estimate for other ports.

More importantly, generation did not remove the cost of comprehension. Line-by-line review, upstream comparison, test-harness design, scope decisions, and mismatch investigation remained human work. The reference implementation reduced uncertainty about selected behaviors; it did not decide which behaviors mattered or who would maintain the result.

What Webernetes teaches

Webernetes shows a strong pattern for agent-assisted compatibility work:

1. Define the useful subset and name unsupported behavior.
2. Preserve a reviewable mapping to a pinned upstream source.
3. Exercise the same public operations against the candidate and an independent reference.
4. Turn mismatches into failing tests before changing the implementation.
5. Keep human review responsible for scope, fidelity, and the limits of the evidence.

The shared checks make selected compatibility claims inspectable and disagreements reproducible.

DEVELOPMENT SYSTEM CASE STUDY

Modern Web Guidance – Treating current knowledge as a tested dependency

Modern Web Guidance is a preview project supported by the Google Chrome and Microsoft Edge teams and the web-development community. It packages current web-platform guidance as skills that coding agents can search and retrieve while working.

The case extends the idea of a project harness beyond one repository. Instead of expecting a model's training data or a permanent prompt to contain the latest HTML, CSS, accessibility, performance, and compatibility practices, a maintained capability supplies a smaller task-relevant guide. The project then evaluates whether access to that guide changes checked outcomes.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

Case at a glance

Design choice	Current approach
Knowledge source	Curated guides and discipline-level skills maintained by domain contributors
Agent access	Semantic search followed by retrieval of a selected guide
Compatibility boundary	A declared Baseline target; Widely Available is the default
Distribution	A generated installation repository and an npm-based CLI
Evaluation contrast	The same coding tasks run with and without access to the guidance
Browser evidence	Playwright assertions calibrated against reference and deliberately flawed pages
Project status	Preview, with content and tooling still evolving

This is a case about the design of a knowledge dependency and its evaluation harness. It is not independent proof that every recommendation is correct or that installing the skill improves every web project.

Retrieval keeps the working context bounded

Modern Web Guidance contains guidance for more than one hundred web-development use cases. Its top-level skill instructs an agent to search the collection for the current task and retrieve a matching guide by identifier. A developer can also use the same search and retrieve commands before installation to inspect what would enter the agent's context.

This separation matters for scale. The full library can evolve independently, while a task consumes only the patterns it appears to need. Descriptions and token counts make selection more inspectable than silently injecting the whole collection. Retrieval can still fail: the query may select the wrong guide,

relevant guidance may be absent, and a concise result may omit a constraint the application needs. The selected guide therefore remains an input to review rather than an authority over the product.

Browser support becomes a project constraint

The project connects its recommendations to Baseline, a shared definition of when web-platform features become interoperable across major browser engines. A repository can state a Baseline year or another support policy in its agent instructions. Without a local target, Modern Web Guidance defaults to Baseline Widely Available and expects fallbacks for features outside that boundary.

This turns browser compatibility from an accidental property of generated code into an explicit design input. It does not replace testing on the browsers, devices, and assistive technologies used by the product's actual audience. A Baseline label describes feature interoperability; it does not establish usability, accessibility, performance, or fit with a particular framework.

The guidance has an evaluation loop

The source repository separates human-authored guidance from the machinery that distributes and evaluates it. For an evaluation-ready use case, a subject-matter expert writes the guide, a reference implementation, and natural-language expectations. The development pipeline generates a deliberately incorrect page and a Playwright grader, then calibrates the grader so the reference passes and the negative page fails.

The harness can then run the same task from the same base application in two conditions: an agent works without the guidance and with access to it. Both outputs are graded with the task's browser assertions. This guided-versus-unguided contrast tests whether the capability adds value beyond what the selected agent and model already know, rather than counting whether the agent opened a guide.

The method has useful limits:

- A generated grader can encode the wrong behavior even after calibration.
- One positive and one negative page do not cover every incorrect implementation.
- Natural-language expectations reflect the maintainers' chosen interpretation of the task.
- The current harness uses small base applications and controlled tasks rather than existing production systems.
- Results are local to the tested task, agent, model, prompt scaffold, guide revision, and assertions.

Calibration can reject an obviously broken check. It cannot turn the check into independent ground truth, so failures and surprising uplifts still require inspection.

Distribution creates an update tradeoff

The project uses two repositories: contributors work in the source repository, while a generated installation repository contains compiled skills and agent-specific packaging. The recommended CLI installation supports automatic updates. That provides a clear maintenance path for knowledge that should change with the platform.

Automatic currency and reproducibility pull in different directions. An evaluation or consequential change should record the resolved skill version or revision and the project's browser-support target. An organization may instead pin updates until review, especially because a skill can influence tool calls made with the agent's existing authority. Official provenance, popularity, and a passing evaluation reduce some uncertainty; none grants the capability permission to override local architecture, security, privacy, or user requirements.

A practical adoption pattern

A team adopting a living capability can keep the boundary explicit:

1. State the local problem and compatibility target before retrieval.
2. Inspect the selected guide and its provenance rather than trusting a match score alone.
3. Record the resolved capability revision or update policy with the change.
4. Adapt the recommendation to the project's architecture and users.
5. Validate behavior in the real application and representative browsers.
6. Re-run a fixed task cohort when the guide, model, agent, or harness changes materially.
7. Keep a removal or rollback path if the capability stops fitting the project.

This makes the external skill a reviewable development dependency rather than invisible model knowledge.

Field note — Curate external knowledge without transferring authority

I have not used Modern Web Guidance directly enough to claim that it improved a particular implementation. I adopted it into Vibe Template because I consider current web-platform guidance useful context for an agent to have when a relevant decision appears.

The recorded integration decision deliberately narrows the upstream capability. Retrieval uses a reviewed, pinned version with telemetry disabled, and the skill activates only when browser-facing work requires a platform choice, compatibility interpretation, or fallback. Project architecture, specifications, user instructions, and validation remain authoritative over anything retrieved.

An agent should be able to discover that a relevant browser feature, compatibility concern, or native platform pattern exists without loading an entire changing guide into every task or treating external guidance as a project requirement.

For now, this is a reasoned design choice rather than evidence of improved project outcomes. The capability still needs to justify itself through the decisions and implementations it influences.

What Modern Web Guidance teaches

External guidance becomes an inspectable dependency when retrieval, versioning, compatibility, and updates are connected to local checks. Paired agent runs can test whether it improves selected tasks; the guide and its grader still leave broader product quality to be evaluated.

DEVELOPMENT SYSTEM CASE STUDY

Temper – Contracts, verification, and bounded autonomy

Chapter 2's deterministic-control example lets an agent choose events within an existing state machine. Temper extends the boundary: an agent can author the state machine itself, subject to verification and authorization controls. Anthropic's July 2026 account of Datadog's approach describes agents producing specifications of states, transitions, preconditions, and safety properties for a deterministic kernel to check and execute. Data contracts expose the available operations, while a separate authorization engine enforces default-deny policies (Segner, 2026).

The same account traces Temper's development through BitsEvolve and Helix, and describes its role in coordinating agents, building tools, and controlling Helix's lifecycle. Datadog's earlier reports make the verification boundaries of those systems more concrete (Segner, 2026; Keles et al., 2026; Chen & Nalla, 2026).

```
agent-authored specification → verification → runtime enforcement
```

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

Start with system invariants and replayable failures

Datadog's redis-rust and Helix experiments combined explicit invariants, model checking, deterministic fault simulation, and staging telemetry. For Helix, the contract included preserving acknowledged messages across leadership changes. Seeded network, disk, and node faults made failures replayable; one exposed a log-truncation bug that could lose data after a failed disk sync. A redis-rust CPU optimization failed a linearizability check, so a performance gain alone could not authorize acceptance (Keles et al., 2026).

Observed workloads also challenged the checks. Despite comparable latency, redis-rust initially used eight times Redis's memory; tuning reduced that inflated footprint by 87%. The reduction was relative to the candidate's starting point. This illustrates why a microbenchmark needs an operational comparison and why new observations should revise the harness. Humans still defined invariants, strengthened checks, and approved architectural changes (Keles et al., 2026).

Verification and execution share a specification

Temper makes the behavioral contract an artifact the runtime can enforce directly. This combines agent authoring with model-driven development and formal methods. The project's architecture draft derives the verification model and runtime transition table from the same specification, reducing the

maintenance gap between a separate model and implementation. The kernel and the translations into those representations remain part of the trusted implementation (Nalla, 2026).

Four complementary checks

Symbolic reasoning checks whether guards can be satisfied and transitions preserve declared invariants. Model checking explores reachable states within configured bounds. Deterministic simulation exercises transitions under seeded faults, and property-based testing samples action sequences and reduces failures to smaller counterexamples. Simulation and randomized testing provide evidence from the runs exercised; they do not exhaust every possible execution (Nalla, 2026).

Bound live optimization with an independent reference

Datadog's Unicron aggregation service provides a narrower boundary for autonomous code changes. BitsEvolve specializes a function for a tenant and configuration, generates alternatives, and returns WebAssembly modules. Before activation, each candidate must match the baseline's output on traffic withheld from the evolver. The runtime isolates the module and checks its authenticity; existing batches finish on the old version (Chen & Nalla, 2026).

The formal guarantees are specific: Verus checks declared safety properties of codec libraries used by the evolved code. Output agreement on held-back traffic is empirical evidence about aggregation behavior. Neither establishes equivalence on every possible input.

One reported workload increased throughput from 7,106 to 26,263 messages per second against the generic production baseline. The report separates parameter specialization from LLM evolution: their respective 44.5% and 155% gains combine multiplicatively to approximately 270%. Compare all three conditions before attributing the result to agent search, and include search and verification costs in the engineering decision (Chen & Nalla, 2026).

What verification leaves outside

Temper's architecture draft documents restricted expressions and limitations in proving eventual progress and interactions between entities. A specification can also omit a required rule while satisfying every property it declares. The repository includes sandboxed application modules alongside the behavioral contract; checking the lifecycle does not establish the correctness of every module or external effect (Nalla, 2026; Temper Contributors, 2026). Hot reload likewise needs a plan for existing state, compatible consumers, changed permissions, and recovery. The durable-workflow effect audit examines the related boundary between recorded progress and external effects.

Evaluate the scalability hypothesis

The development-scalability hypothesis is that a constrained, inspectable contract can reduce the work needed to accept repeated changes. Evaluate that hypothesis through accepted outcomes, human review time, escaped defects, and maintenance cost, including the initial modeling effort and ownership of the kernel. These company accounts illustrate responses to the verification bottleneck; they do not establish a general productivity gain. Compare the cost of building and maintaining the checks with the correction work they avoid. Automated acceptance also needs an accountable owner for the reference behavior, permissions, and changes to the harness itself.

Optional exercise — Check what the model leaves out

Use a small transition table and runner, or inspect the model by hand; record which method you used. This exercise does not require adopting Temper.

1. Model the event tracker's `open → in_progress → done` lifecycle from the common specification. Record the required activity events and the rule that a rejected transition changes neither state nor successful-event history.
2. Add an `open → done` transition. Predict which explicit lifecycle property should reject it, then check the model and retain the violating trace.
3. Restore the legal transitions but leave event recording outside the model. Construct a trace that reaches `in_progress` without a `work_started` event. Explain why lifecycle checks alone can pass while the application violates its requirement.
4. Consider a crash after the state write but before the event write. Identify where atomic persistence must be enforced and what additional execution evidence would test it.

Report the property checked, the exploration bound where applicable, the counterexample, and the obligation left outside the model.

References

1. Segner, M. (2026, July). How Datadog Built a “Universal Machine Tool” for Claude Code. Anthropic Blog. https://claude.com/blog/how-datadog-built-a-universal-machine-tool-for-claude-code
2. Nalla, S. (2026, March). Temper: Descriptions All the Way Down. Project architecture draft. https://github.com/nerdsane/temper/blob/main/docs/PAPER.md
3. Temper Contributors. (2026). Temper: A Verified Runtime for Systems Agents Build. GitHub repository. https://github.com/nerdsane/temper
4. Keles, A., Menon, J., Nalla, S., & Shah, V. (2026, March). Closing the Verification Loop: Observability-Driven Harnesses for Building with Agents. Datadog Blog. https://www.datadoghq.com/blog/ai/harness-first-agents/

https://www.datadoghq.com/blog/ai/fully-autonomous-optimization/

ELEMENTS OF SCALABLE WEB DEVELOPMENT

Course project guide

Course Project Introduction

The course uses three fresh implementations of the same Work Activity Tracker to study both application scalability and the development workflow that produces the application. Begin here for the guidelines shared by all three projects, then read the Common Contract and the current phase brief.

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

Three projects, one application

1. **Establish**: build implementation A with a locally hosted model and a single-agent workflow, then record its evidence baseline.
2. **Challenge**: build implementation B with an approved comparison model and a single-agent workflow, then examine architectural assumptions and development friction.
3. **Compare**: build implementation C with a declared agent fleet, then compare the three applications and development workflows.

The required behavior remains substantially stable while the development setup changes. Each later phase also benefits from your growing domain knowledge, so the comparison supports a conditional engineering judgment rather than a universal model ranking.

Groups, submissions, and access

Shared work and individual assessment

Register one group identifier and its membership with the course. Keep the group stable across Establish, Challenge, and Compare where practical; agree membership changes with the teaching staff and record which group each student belongs to for the affected phase. One member submits the group bundle on behalf of the registered group, and every member remains responsible for understanding and accepting its work.

Each group receives one project score per phase, which every registered member receives. Individual contribution is assessed through the self- and peer-assessment criteria, while participation and learning diaries are also scored individually.

Work	Submitted by	Maximum per student across the course
Three project bundles	Group	90 points
Nine lecture participation and diary records	Individual	27 points
Three self- and peer-assessments	Individual	36 points

Submit one private individual portfolio per phase containing the relevant lecture diary entries and self- and peer-assessment. Label entries by lecture so the instructor can apply the existing three-point lecture criteria. The assessment rubric defines the criteria, passing requirements, and grade thresholds. To pass, each student must also complete the mandatory lecture activities and individual assessments in every phase.

Submit each phase in stages

Use one group submission record per phase, identified by the group, phase, implementation revision, and evidence index. The course instructions publish the deadlines and submission locations before work begins.

1. **Freeze the implementation:** submit the implementation revision, setup and validation instructions, and the available common and phase-specific evidence.
2. **Audit:** after the implementation deadline, the assigned group inspects that frozen revision and returns the bounded audit report.
3. **Complete the evidence:** attach the audit and the authors' response to the same group submission record, preserving the original implementation and evidence. Each student also submits the private individual portfolio by its published deadline.
4. **Assessment:** the instructor records the group project score and each student's individual scores, with criterion-level reasons and any missing evidence.

Label later code corrections separately; they become assessed revisions only when the announced correction policy permits them. Report a missing audit or access problem to the teaching staff so they can arrange and record a fallback. The Common Contract defines the completed technical bundle and audit protocol.

Browse other groups' submissions

After the phase's common implementation deadline, the assigned auditors receive access to the frozen technical bundles. The instructor opens a course-only submission index for general class browsing after the audit window closes, so preparation outside the audit does not silently disappear from the recorded effort. Each entry identifies the group and phase and links the frozen implementation revision, setup and validation instructions, evidence index, and audit report with the authors' response when available. Classmates receive read access to the submitted source and technical evidence, so browsing supports learning beyond the assigned audit. Course access is sufficient; public publication is optional.

Run checks or try changes in your own checkout, following the announced course tool policy. Use synthetic demonstration data and provide access without sharing personal accounts or credentials. For an assigned audit, declare any prior inspection of the implementation and follow the audit timing protocol. Looking at another submission does not authorize copying its implementation; follow the transfer policy and acknowledge ideas that materially inform your decisions.

Keep learning diaries, self- and peer-assessment reports, personal support information, and scoring records in the private course submission channel. Share useful feedback with teammates, while confidential assessment reports go to the teaching staff. Each group can see its project score and

feedback; each student can see their individual score breakdown, completed lecture activities, phase assessment results, total, and any unmet passing requirement.

Guidelines for course work

These guidelines apply across all three implementations.

Keep one evidence bundle

Each main chapter ends with a **Project artifact** section. These sections are staged checkpoints for building the current phase's evidence; they do not silently create separate submissions. The common project specification and current phase specification provide the authoritative deliverable checklist, and one clearly structured artifact may satisfy several chapter checkpoints.

Record assistance and acceptance evidence

Record material assistance and human corrections in the AI-use log, and bind checks, acceptance decisions, and known limits to the submitted revision. Use the common evidence bundle so others can assess the work and compare workflows. Preserve truthful provenance and accountable ownership; avoid false claims of human-only production.

Follow external contribution policies

For an external issue or pull request, follow the receiving project's applicable contribution and AI policies. Read them before preparing the submission: they may govern permitted assistance, who writes and posts public text, and disclosure. Respect a project's refusal of particular assistance or submissions; disclosure does not make a prohibited contribution acceptable. An external disclosure rule does not remove the course's evidence requirements. Appendix D's contribution-policy example examines how private tooling, public authorship, and disclosure can be governed differently.

Protect learning, not only output

Immediate task performance is not the only outcome worth protecting. Across three randomized experiments involving short arithmetic and reading-comprehension tasks, brief AI assistance improved work while it was available but reduced later unaided performance and persistence. In a cross-sectional subgroup analysis of the second experiment, participants who reported using AI for direct answers later performed worse and skipped more than both the control group and those who reported using it for hints (Liu et al., 2026). These tasks do not establish the long-term effect on professional programming, but they support a practical course rule: use agents for hints, critique, and verification when the underlying skill must remain yours.

Begin the first project

Read the Common Contract for the application behavior, shared evidence, and evaluation requirements, then the Establish brief for implementation A. The course environment sheet defines the supported models, tools, access, budgets, privacy, and fallback for the current course run. Consult the assessment rubric for the scoring and essential learning outcomes.

References

1. Liu, G., Christian, B., Dumbalska, T., Bakker, M. A., & Dubey, R. (2026). AI Assistance Reduces Persistence and Hurts Independent Performance. arXiv Preprint arXiv:2604.04721. https://doi.org/10.48550/arXiv.2604.04721

Common Project Specification

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

Work Activity Tracker

This specification defines the application that your group will implement three times during the course. The required behavior remains substantially the same. The development setup changes:

1. **Establish:** locally hosted model and a single-agent workflow.
2. **Challenge:** comparison model and a single-agent workflow.
3. **Compare:** an agent fleet using a declared base model.

Each phase produces a separate implementation. The purpose is not to find one universally best model, framework, or architecture. The purpose is to compare how different development systems approach the same bounded web application and to reason about the scalability of both the application and the development process.

The phase specifications define additional constraints and deliverables. This common specification applies to all three phases. Read the project introduction before starting work; it explains submission checkpoints, evidence records, external contributions, and learning expectations.

Shared engineering sessions

Each group must hold one shared engineering session of approximately 30 minutes per phase, within its existing project work. Use three responsibilities: **operator** works with the agent and explains the request; **challenger** questions assumptions and proposes alternatives; **validator** inspects the implementation, runs checks, and explains the acceptance decision. Rotate these responsibilities across phases so each student practices all three. For an approved pair, combine challenger and validator and rotate the operator; for a larger group, divide active investigation and validation tasks so everyone contributes.

Use this sequence:

1. Before consulting the agent, each student briefly predicts the behavior or proposes a design.
2. Compare predictions, agree on the question, and investigate the implementation with the agent.
3. Run a relevant check and have someone other than the operator explain whether to accept, revise, or reject the result and what remains uncertain.

Use a question tied to the current phase:

- **Establish:** Does a work-item transition reliably produce the corresponding activity event?
- **Challenge:** What happens when two clients attempt the same transition concurrently?

- **Compare:** Does a proposed scaling response improve the measured limit, and what new failure can it introduce?

An equivalent question is acceptable when it examines the same phase's correctness, pressure, or scaling reasoning. Record participants and responsibilities, the initial disagreement or prediction, the check and result, and the acceptance decision compactly in the existing decision record or AI-use log. Link the relevant code or evidence; do not submit another report or a raw chat transcript. This session replaces part of ordinary project discussion and reflection within the published time budget. Shared work may be remote and does not require a shared personal account.

---

1. Application Brief

Build a web application that records activity around work items.

The application contains:

- **Workspaces**, which separate independent groups or projects.
- **Work items**, which belong to a workspace and move through a small lifecycle.
- **Activity events**, which record what happened to a work item.

The domain is intentionally familiar and compact. Your group may give the application a visual theme or adapt the terminology—for example, a workspace may be presented as a classroom, game, laboratory, production line, or agent workflow. The required capabilities must remain identifiable to an auditor, but the specification does not prescribe one ideal interpretation or design.

This specification primarily describes observable behavior. When several interpretations satisfy the requirements, choose one, implement it consistently, and document any assumption that affects how the application is used or evaluated. You do not need to make the implementation unusual deliberately; differences are expected to emerge from ordinary design and development decisions.

In Compare, the same model will be applied to the development workflow itself: a fleet run can become a workspace, delegated tasks can become work items, and agent or human actions can become activity events. The earlier implementations do not need agent-specific behavior.

---

2. Required Domain Model

Your implementation must represent the following concepts. Field names and internal representations may differ.

2.1 Workspace

A workspace must have:

- a unique identifier;
- a name.

2.2 Work item

A work item must have:

- a unique identifier;
- a workspace;
- a title;
- a state in a lifecycle with at least three ordered stages corresponding to:
  - not started, such as `open`;
  - active, such as `in_progress`;
  - finished, such as `done`.

You may add descriptions, owners, labels, priorities, deadlines, or other fields.

2.3 Activity event

An activity event must have:

- a unique identifier;
- an event type;
- the workspace concerned;
- the work item concerned;
- an occurrence timestamp;
- an actor represented at least by a name or identifier.

The required event meanings are:

- creation of a work item, such as `item_created`;
- movement into the active stage, such as `work_started`;
- movement into the finished stage, such as `work_completed`.

Events may contain additional metadata. Names and representations may differ if their mapping to the required meanings is documented.

---

3. Required Behavior

3.1 Workspace management

A user must be able to:

- create a workspace;
- view the workspaces needed to use the rest of the application.

Editing and deleting workspaces are optional.

3.2 Work-item lifecycle

A user must be able to:

- create a work item in a workspace;
- move a work item from the initial stage to the active stage;
- move a work item from the active stage to the finished stage;
- see the current state of a work item.

Creating a work item must record one creation event. Each successful state transition must record the corresponding activity event.

Unsupported transitions must be rejected. For example, an item cannot skip directly from the initial stage to the finished stage. A rejected transition must not change the item state or record a successful transition event.

3.3 Activity feed

A user must be able to:

- view recorded events;
- identify the event type, workspace, work item, actor, and occurrence time;
- filter events by workspace;
- filter events by event type.

The default ordering must be documented and deterministic. Newest-first ordering is recommended.

3.4 Summary

The application must show at least one derived summary for a selected workspace:

- the number of work items in each state; or
- the number of events of each type.

The summary may be calculated when requested, maintained incrementally, or produced in another explainable way.

3.5 Validation and errors

The application must:

- reject missing or invalid required values;
- preserve a valid state when an operation fails;
- show the user an understandable error;
- avoid exposing secrets or a raw internal stack trace through the user interface.

3.6 Persistence

Workspaces, work items, their states, and activity events must survive an ordinary application restart.

The storage mechanism is your choice.

3.7 Intentionally unspecified details

The specification does not prescribe:

- page structure, navigation, or visual design;
- whether interactions use forms, links, buttons, an API, or another interface;
- exact URLs, endpoint shapes, or transport formats;
- whether rendering happens on the server, client, or both;
- identifier format or timestamp precision;
- how actors are chosen or represented beyond the required event information;
- how state and events are represented or stored internally;
- how the summary is calculated;
- module, service, or deployment boundaries;
- behavior outside the required lifecycle and acceptance scenarios.

Resolve these details in a coherent way. Record assumptions that another group would need in order to use, test, or understand the result.

---

4. Common Acceptance Scenarios

Another group and the instructors must be able to verify at least the following scenarios through the running application:

1. Create two workspaces.
2. Create work items in both workspaces.
3. Confirm that each new work item begins in the documented initial state.
4. Start one work item and confirm that it enters the documented active state.
5. Complete that item and confirm that it enters the documented finished state.
6. Attempt an unsupported state transition and confirm that it is rejected without changing the stored state.
7. View the activity feed and find one event for each successful operation.
8. Filter the feed to one workspace and confirm that events from the other workspace are absent.
9. Filter the feed by event type.
10. View the required summary and verify it against the created data.
11. Restart the application and confirm that the data remains available.
12. Submit invalid input and confirm that the result is understandable and does not corrupt existing data.

The course may provide automated checks for behavior that can be tested without prescribing a particular implementation. Passing an automated check does not replace human review.

5. Technical Freedom and Experimentation

You may choose:

- the programming language;
- the web framework or combination of frameworks;
- server-rendered pages, a client application, LiveView-style interaction, or another web approach;
- a monolith, client-server design, serverless design, or another explainable architecture;
- relational, document, file-based, event-oriented, in-memory-with-persistence, or other storage;
- local, containerized, cloud, edge, or other deployment;
- testing, observability, and build tools.

Next.js, Elixir, Python, and less common stacks are all valid choices. A familiar stack is not automatically safer, and an exotic stack is not automatically more innovative. Evaluation is based on what the choice enables, what it costs, how well the result satisfies the contract, and whether another group can understand and run it.

For the clearest within-group comparison, you should normally keep the same primary language and framework family across all three implementations. Different groups are encouraged to choose different stacks. A group may change its own stack between phases when it has a clear experimental reason and instructor approval, but the final analysis must then treat the stack change as a confounding factor.

Experimentation is encouraged when it serves a question the group can explain. Examples include:

- comparing server rendering with a client-heavy approach;
- using an actor model or functional language;
- trying an event-sourced representation;
- using an embedded or unusual database;
- targeting an edge or serverless runtime;
- minimizing client JavaScript;
- optimizing for local-first operation, privacy, portability, or resource efficiency.

Your group owns the operational consequences of its choices. A stack that nobody outside the group can install, run, test, or inspect has not met the project requirements, even if the application worked on the authors' machine.

No particular stack receives extra credit merely for being fashionable, complex, or unusual.

---

6. Minimum Project Harness

Project harness is the course's label for the repository guidance, reproducible setup, and executable checks that support its development workflow. Each implementation must provide:

- a `README.md` that explains the application, prerequisites, setup, running, testing, and the chosen stack;
- one supported path from a clean checkout to a running application;
- pinned or otherwise reproducible dependency and runtime versions;
- an example configuration file when configuration is required;
- no committed secrets;
- a repeatable way to create enough demonstration data for the acceptance scenarios;
- a repeatable workload adapter for the shared runtime protocol;
- at least one automated validation command;
- tests covering the work-item lifecycle and its event recording;
- project instructions for the AI development workflow;
- an architecture or data-flow diagram and a short architecture note for the core event workflow;
- one maintained feature specification for the core event workflow, including its acceptance path;
- a short decision record explaining the stack choice.

Setup may use native tools, containers, a development environment definition, or another documented mechanism. Do not require an auditor to infer missing steps from knowledge held only by your team.

The README must distinguish mandatory prerequisites from optional development conveniences. If the stack requires unusual system dependencies or hardware, document them near the beginning.

---

7. Shared Runtime Workload and Progressive Scalability Analysis

You do not have to build a production-scale distributed system. You must be able to measure a stable baseline and reason about what your implementation would do under pressure.

7.1 Shared baseline workload

Run the same stack-neutral workload against implementations A, B, and C. The workload measures logical user operations from the adapter's request until the application returns an observable result, not individual internal network requests. The adapter may drive the user interface, a documented application interface, or another repeatable boundary that exercises the real application and persistence path. Choose that boundary before measuring A and keep it across B and C: API-response latency and browser-visible completion are different outcomes. Define the start and completion events, including response validation and what constitutes acknowledged persistence; separate additional diagnostic boundaries rather than ranking them together.

Start each measured profile from this fixture:

- eight workspaces named `workspace-01` through `workspace-08`;
- fifty work items per workspace: twenty in the initial state, fifteen in the active state, and fifteen in the finished state;

- all activity events implied by creating and transitioning those items;
- four actors named `actor-01` through `actor-04`, distributed deterministically across the fixture;
- stable fixture keys that the adapter can map to implementation-specific identifiers.

A workload cycle uses this operation mix:

Share	Logical operation	Target rule
30%	Read the activity feed	Divide evenly between the unfiltered feed, one workspace, and one event type
15%	Read a workspace summary	Rotate evenly through the eight workspaces
20%	Create a work item	Rotate workspaces and actors; use a unique deterministic title
15%	Move an existing initial item to the active state	Use a distinct valid target for each operation
10%	Move an existing active item to the finished state	Use a distinct valid target for each operation
10%	Attempt an unsupported transition	Treat the documented rejection as an expected result, then verify that state and events did not change

Use a fixed operation-order seed and record it. Concurrent operations in this baseline should use distinct work items; contention on the same item belongs in a pressure scenario.

Run the following profiles:

Profile	Operations	Concurrency	Repetitions	Reset rule
Warm-up	100	1	1 unmeasured	Reset and reseed afterward
Steady	500	4	3 measured	Reset and reseed before each repetition
Higher concurrency	200	16	3 measured	Reset and reseed before each repetition

These are **closed-loop** profiles: each of the declared workers dispatches its next logical operation only after its previous operation completes or times out, with no intentional think time. Workers take operations from the fixed seeded sequence until it is exhausted; concurrency is a maximum, not a guaranteed arrival rate. Disable adapter retries for the baseline, and record any application-level retry policy and internal attempts separately from logical operations. Record the cache policy too: resetting the fixture after warm-up does not guarantee warm data caches.

Predeclare one per-operation timeout and one overall run deadline, held constant across comparable runs. Use 10 seconds per operation and five minutes per measured repetition unless the instructor approves different limits for the chosen boundary before the comparison. At the run deadline, stop dispatching, record queued operations as not started and active operations as unfinished, and preserve their identifiers for reconciliation. A timeout is an observed deadline crossing, not an exact completion latency or proof that a write did not commit. After the run, reconcile state and events for timed-out or unfinished writes, and report unresolved outcomes explicitly. Quiesce or isolate outstanding work before resetting the fixture so a late write cannot contaminate the next repetition.

For every measured repetition, record:

- implementation identifier, commit, environment, configuration, adapter version, and operation-order seed;

- elapsed time, scheduled and dispatched counts, completed valid operations, and valid completions per second;
- median and 95th-percentile latency for completed valid operations, clearly labeled as conditional on completion, plus their sample counts and the percentile calculation;
- separate counts for not-started, unfinished, and timed-out operations, including their known waiting durations or deadline lower bounds;
- expected rejections separately from timeouts, crashes, invalid responses, and other unexpected errors;
- post-run checks for state, event, filter, and summary correctness;
- one resource-efficiency proxy that can be collected consistently across phases, such as peak memory, CPU time, database growth, or billed execution time.

Report each repetition and the median result rather than presenting only the best run. Expected unsupported-transition rejections count as valid completed operations only when the unchanged-state check passes; unexpected admission rejections do not. Show read, write, and expected-rejection counts separately so fast refusals cannot conceal lost useful work. For the baseline p95, use the nearest-rank observation at `ceil(0.95 × n)` in the sorted valid latencies, and retain the raw observations. In 200 observations, only about ten lie in the upper 5%; per-operation subsets are smaller still. These short runs expose variation and regressions, but cannot establish stable production tail latency; report small subsets descriptively and avoid spurious decimal precision. Absolute results from different hardware or hosted environments should not be ranked as though the environment were controlled. If a platform limit prevents a profile from completing, preserve that result as evidence and label any reduced diagnostic run instead of silently changing the protocol.

The baseline is an experimental reference, not a claim that this operation mix represents every real workload.

7.2 Progressive pressure analysis

Use the baseline workload as the starting point for the cumulative pressure analysis. Change one main condition at a time where practical so the effect remains interpretable—for example concurrency, fixture size, dependency latency, or repeated requests.

Use these scenarios as a shared pressure catalog:

- many independent workspaces;
- concurrent users changing different and identical work items;
- high and bursty event volume;
- storage growth over time;
- slow or temporarily unavailable dependencies;
- retries that may repeat a request;
- automated reads or writes whose application value, resource cost, or access pattern differs from ordinary interactive use;
- demand for recent activity and historical summaries;

- a need to retain, aggregate, archive, or delete old events.

Do not exhaustively apply the entire catalog after every phase. In Establish, select one **primary pressure** that is likely to expose an important architectural limit and define it as a documented transformation of the baseline workload. Reuse that scenario and its procedure in Challenge and Compare so the three implementations remain comparable. Challenge adds one **secondary pressure** revealed by an architectural difference or observation. Compare uses the broader catalog to interpret important differences across A, B, and C, but only relevant pressures need in-depth analysis.

For each pressure analyzed in depth, identify:

- what is likely to fail or slow down first;
- the resource, quota, dependency, or operational limit being approached;
- the first useful signal that would reveal the problem;
- one plausible response;
- at least one prepared recovery action available if the limit is reached;
- the new cost or failure mode introduced by that response.

When automated use or data exposure is the selected pressure, also distinguish cooperating, authorized, and abusive consumers; state which information is intentionally public or retained; and account for the false-positive and accessibility cost of the proposed control.

Use runtime evidence where the available workload and environment allow it. Label reasoned projections as projections rather than observations. The goal is a cumulative, justified analysis, not repeated hypothetical matrices or distributed infrastructure added without evidence.

7.3 Arrival-controlled overload and recovery

When a selected pressure or scaling claim concerns sustained incoming demand, queue growth, or admission control, add a bounded **open-loop** pressure run within that experiment. It replaces part of the existing pressure analysis rather than adding another mandatory scenario for every build. Schedule arrivals independently of earlier completions; a closed-loop client population can reduce its own offered load as the application slows.

Use a pilot to select and then freeze three arrival rates expected to be below, near, and above the observed service limit, with 30 seconds at each rate followed by up to 60 seconds of drain and recovery observation. Declare the operation mix, fixture, timing boundary, request timeout, maximum in-flight work, and total attempt budget before the comparison, and keep them stable when rerunning the selected pressure. Do not silently lower an assigned rate when the generator or application cannot keep up. If a generator cap prevents dispatch, count the scheduled operation as not dispatched; do not accumulate an unbounded catch-up queue.

Record scheduled arrivals, actual dispatch times and scheduling lag, application admissions where observable, expected domain rejections, overload rejections, valid completions, timeouts, and unfinished work separately. If admission is not observable, label it unknown rather than inferring it from dispatch. Report both scheduled-arrival-to-result time, which includes generator waiting, and dispatch-to-result time, with the same outcome and censoring rules as the baseline. Record queue

length or age and resource use during load and drain where observable; distinguish application backlog from generator lag. At the drain deadline, retain unfinished identifiers, check for late writes or duplicates, and verify ordinary service has recovered. If it has not, report that result and perform the prepared recovery action.

---

8. Permitted Transfer Between Implementations

Implementations must begin in separate repositories or clean implementation directories.

You may carry forward:

- this specification and instructor-provided materials;
- acceptance scenarios and course-provided tests;
- your own written lessons and comparison notes;
- domain terminology and user-facing copy;
- technology-neutral diagrams;
- generic personal or course tooling that does not contain application behavior.

You may not carry forward:

- application source code;
- implementation-specific tests;
- generated components or configuration copied from an earlier implementation;
- an earlier repository used as the starting point for the next phase;
- prompts that contain substantial blocks of the earlier implementation.

The bounded paired task is an explicit exception for experimental copies of a frozen revision, not for implementation C's starting code.

When uncertain, disclose the reuse in the evidence package. The purpose of the rule is to create genuinely fresh implementations, not to prevent students from learning.

The same restriction applies to other groups' submissions. Learn from their evidence and engineering decisions, but do not copy their application source, implementation-specific tests, or implementation-specific configuration into your own build. Acknowledge ideas that materially inform your decisions.

Third-party libraries, public starter templates, framework generators, and generic personal scaffolds are allowed if their use is documented and permitted by the course's normal rules.

---

9. Common Submission Bundle and Comparable Evidence

After every phase, submit one common bundle containing:

- that phase's implementation in its own repository or clean implementation directory;

- the comparable evidence categories in sections 9.1–9.4;
- the bounded cross-team audit report and a short response from the authors; and
- the additions named by the phase specification.

A listed requirement does not automatically require a separate file. One clearly structured artifact may satisfy several fields, and the phase specifications name only additions to this common bundle. Use the same evidence categories after every phase so later comparisons can refer to the earlier package rather than reconstruct it.

Submit the frozen implementation and initial evidence first, then attach the audit report and authors' response to the same record at the evidence deadline. The project introduction defines these stages and class access. Identify the group and phase in the evidence index, link the submitted revision and each required artifact, and keep private individual portfolios outside the technical bundle.

9.1 Development setup

- implementation identifier and commit;
- language, framework, storage, and deployment approach;
- model name and whether inference was local, departmental, or externally hosted;
- agent tool and version or retrieval date, together with any imported skill, plugin, or instruction bundle that materially affected the workflow;
- orchestration configuration, if any;
- local hardware class or hosted environment relevant to the result, paths by which project data may leave that environment, and any fallback policy.

When the tool exposes them and they materially affect a comparison, also record the resolved model revision or digest, quantization, runtime, allocated context, actual inference route, memory limits, accelerator residency or CPU/GPU offload, and resolved revision or content digest of imported capabilities. Mark unavailable fields as unavailable rather than spending project time estimating hidden configuration.

9.2 Effort and execution

- elapsed project time;
- estimated active human time;
- approximate model usage, cost, or locally measurable resource use;
- failed or abandoned attempts worth noting;
- important human interventions and corrections.
- one representative AI-assisted task record containing the task brief, relevant model and agent configuration, checks, trace or compact execution evidence, human correction, acceptance decision, and known limits.

Exact accounting is not expected when a tool does not expose it. State what could and could not be measured.

9.3 Product evidence

- acceptance-scenario results;
- automated test results;
- results from the repeated shared baseline workload, including the declared resource-efficiency proxy;
- known defects or incomplete requirements;
- deployment URL, local run evidence, or another agreed operational artifact.

Use the same resource-efficiency signal and procedure across phases where practical. If direct energy measurement is unavailable, report an observable resource proxy and do not relabel it as energy or carbon.

Every implementation must have a reproducible operational path. The Compare phase adds one realistic deployment outside the author's development process; its specification defines that final requirement.

9.4 Engineering explanation

- architecture and data-flow diagram;
- stack decision record;
- the phase's cumulative scalability-pressure record;
- a standardized friction audit covering setup, comprehension, change, testing, review, deployment, operations, and coordination, including the largest source of resistance and one improvement or follow-up decision;
- short reflection distinguishing observation from a possible explanation.

Do not submit an uncurated raw agent transcript as the main evidence. Preserve enough task, prompt, trace, and review information to explain consequential decisions and failures.

---

10. Bounded Cross-Team Audit

After each phase, another group will inspect the implementation for a limited period using only the repository and submitted instructions. Use the same audit tool policy in every phase and, where scheduling allows, the same auditing group. Record unavoidable changes in auditors or tools as comparison limitations.

The instructor assigns the auditing group and ensures it can read and check out the frozen implementation and access the submitted technical evidence. Use the link supplied with the audit assignment to locate that revision; work on a disposable checkout and record access problems and any prior inspection before attempting the probe. The course submission index opens for general class browsing after the audit window closes. Return the report to the authors and teaching staff, then link it from the authors' phase bundle together with their response. Auditors provide evidence for assessment; the instructor awards the project score. Your work on an assigned audit may also support your own individual contribution evidence.

The auditing group will:

1. attempt the documented setup;
2. run the documented validation command;
3. check a sample of the common acceptance scenarios;
4. locate where one important domain behavior is implemented;
5. identify one hidden assumption, inconsistency, or comprehension problem;
6. attempt the small change probe supplied for that phase;
7. return a concise friction report.

10.1 Change-probe protocol

The change probe is this course's bounded task for maintainability assessment. It provides evidence about understanding, modifying, and validating one change under recorded conditions; it does not establish a complete maintainability score.

The instructor releases one phase-specific probe only after the implementation has been submitted. The probes use different product language but exercise an equivalent vertical slice and acceptance burden. Their exact change shape is withheld until release.

The change-probe portion of the audit lasts thirty minutes. Begin timing after the auditor receives the prompt and has a runnable checkout, so setup friction and change friction remain separate observations. During the time box:

- use only the submitted repository, its instructions, and the common audit tool policy;
- do not ask the authors where the change belongs;
- work in a disposable branch or copy rather than modifying the submitted implementation;
- run relevant existing checks before adding a focused automated test when time permits;
- stop when the time expires, even if the change remains incomplete.

If the submitted setup cannot be made runnable during the audit, record the probe as **not attempted because setup blocked it**. An estimate may be included as commentary, but it does not count as completed change evidence.

10.2 Friction report

Return any patch or diff together with a report containing:

- setup outcome and setup time, reported separately from the probe;
- time required to identify the likely user-interface, application, domain, persistence, and test locations;
- probe outcome: complete, partial, blocked, or not attempted;
- acceptance checks completed and still missing;
- files or boundaries changed and any duplicated change that appeared necessary;
- commands run and their results, including existing regressions;
- assumptions, documentation gaps, and remaining uncertainty;
- the first point at which progress became slow or blocked.

The audit is not a full code review and does not transfer ownership. Auditors are not expected to become experts in the chosen stack. Difficulty caused by an unfamiliar technology is useful evidence, but the report should distinguish ordinary learning from missing documentation or accidental complexity.

The authors may correct factual misunderstandings in the report, but they should not erase evidence that the system was difficult to understand.

---

11. Optional Extensions

Complete the common requirements before adding extensions. Suitable extensions include:

- authentication and roles;
- work-item assignment;
- comments or additional event types;
- search, pagination, or richer filters;
- import and export;
- real-time updates;
- offline behavior;
- an API for external event producers;
- accessibility improvements;
- localization;
- richer observability;
- import and visualization of an agent-workflow trace;
- explicit idempotency support;
- retention, archival, or deletion policies;
- performance or resource-efficiency improvements.

Optional scope must be reported separately so that a larger feature set is not mistaken for a better result.

---

12. Common Evaluation Principles

Every phase is evaluated using these principles:

- required behavior works;
- the stack and architecture are understood and justified;
- setup, testing, and operation are reproducible;
- scalability claims are tied to evidence and explicit trade-offs;
- AI-generated work is reviewed rather than accepted by default;
- reporting is honest about failures, intervention, and uncertainty;
- optional output volume does not substitute for the common core;

- another group can inspect the project without relying on private explanations.

The phase specifications describe the emphasis and deliverables for each build.

Establish Project

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

Implementation A: Local Single Agent

The common project specification applies to this phase.

---

1. Purpose

Establish creates the first implementation and the comparison baseline.

Build the Work Activity Tracker using a locally hosted model through a single-agent workflow. The goal is not to demonstrate that local models are inherently weak or strong. The goal is to experience what this particular model and environment make easy, difficult, fast, slow, private, controllable, or resource-intensive.

Complete and submit implementation A before lecture 4. The public discussion during lecture 4 will use the submitted applications, benchmark results, and architecture diagrams as concrete cases.

Keep the implementation deliberately modest. A complete common core with clear evidence is more valuable than a large unfinished application.

---

2. Development-System Constraint

You must use:

- only the approved local-model endpoint or named departmental local-model service; every other inference route and automatic model fallback must be disabled or fail closed;
- one coding-agent process at a time;
- the model, context, and tool configuration documented in your evidence package.

You may:

- discuss, plan, write, test, and review work as a group;
- use ordinary deterministic development tools;
- restart the agent or begin a new session when needed;
- use non-generative documentation and search tools allowed by the course.

You may not:

- use a model or inference route outside the approved Establish configuration to generate, repair, review, or explain application code;
- delegate work to subagents or run coding agents in parallel;
- copy a prior implementation of the course application.

If the local model cannot complete a task, the group may implement or correct it manually. Record consequential human intervention instead of hiding it.

---

3. Required Work

3.1 Implement the common core

Complete the required domain model, behavior, harness, and acceptance scenarios from the common specification. Before expanding optional scope, describe the first closed product loop and list the capabilities deliberately deferred until that loop produces evidence.

If the implementation begins from a starter template, record the template source and starting revision before pruning it. Classify inherited material as:

- **keep** because it protects the closed product loop or its development harness;
- **replace later** because it is still the only working seam;
- **remove** because it is distribution-only material with no current owner; or
- **decide** because removal would settle a product or architecture question that remains open.

Review the exact removal set before editing, especially when automation or an agent performs the pruning. The template is a distribution artifact, not the finished architecture of the product.

3.2 Establish the evidence baseline

Record the comparable evidence fields in the common submission bundle while the work is happening. Do not attempt to recreate model usage, failed attempts, or human intervention at the end. For optional practice in checking explanations and ownership, choose an Establish exploration starting point.

3.3 Record the stack hypothesis

In the stack decision record, explain:

- why the chosen stack is suitable for this application;
- what the group wants to learn by using it;
- what setup, deployment, scalability, or maintainability risks it creates;
- how well the local model appears to understand the stack;
- which choices remain cheap to revise, what dependencies would make them harder, and what evidence would trigger a review.

The decision-log example shows how to record reversibility and a revisit condition alongside the rationale.

3.4 Analyze initial scalability

Produce two bounded scalability claims: one about runtime demand or data and one about contributors or change. For each, name the maintained goal, pressure, range or scenario, boundary, constraints, likely first limit, first useful signal, candidate response, expected trade-off, supporting or challenging evidence, and one intentionally excluded dimension.

Choose one primary pressure from the shared catalog that is likely to expose an important limit in implementation A. Use the runtime or data claim as the starting hypothesis for that choice, and retain the contributor or change claim for the later friction and development-workflow comparison. Define it as a documented transformation of the shared baseline workload that can be repeated for implementations B and C. Run it where the available workload and environment allow; otherwise mark the result as a reasoned projection.

Identify the likely first bottleneck, the resource or dependency approaching its limit, the first useful operational signal, one plausible response or recovery action, and the trade-off it introduces. You are not expected to add distributed infrastructure.

---

4. Deliverables

Submit the common submission bundle for implementation A plus:

- a capability ownership card for the main event workflow;
- a short local-model workflow reflection.

The workflow reflection should answer:

- What did the local model handle well?
- Where did it require smaller tasks, more context, correction, or manual work?
- Which limitation came from the model, which from the harness, and which from the project itself?
- What evidence supports those interpretations?

---

5. Evaluation Emphasis

This phase emphasizes:

- completion of the common behavioral core;
- a reproducible project harness;
- appropriate scope and technical clarity;
- honest baseline evidence;

- the ability to distinguish local-model behavior from hardware, tooling, and project conditions.

Raw generation speed and feature count are not goals.

Challenge Project

> **Concept connections**
> Explore definitions and connections in the web edition.
> Explore this reading's concepts

Implementation B: Comparison Model, Single Agent

The common project specification applies to this phase.

---

1. Purpose

Challenge creates a fresh implementation of the same Work Activity Tracker using a comparison model through a single-agent workflow.

The purpose is to compare implementation B with the local-model baseline. The approved comparison model must differ from the Establish model; its hosting and harness configuration must be declared. It may handle the same tasks differently, but higher quality, lower cost, and less review are hypotheses to test. A model that generates more code may also introduce unnecessary complexity, plausible-looking defects, greater cost, or more demanding review work.

This is a new implementation, not a refactor of implementation A.

Complete and submit implementation B before lecture 7. The public discussion during lecture 7 will compare the Challenge results before the course introduces the fleet workflow for implementation C.

---

2. Starting Conditions

Begin in a new repository or clean implementation directory.

Use:

- the same common specification and acceptance scenarios;
- the transfer policy in the common specification;
- the approved comparison model;
- one coding-agent process at a time;
- normally the same primary stack chosen for Establish.

Keeping the primary stack stable makes the comparison easier to interpret. A group may change it with instructor approval when the change supports a clear learning question. The final comparison must then treat the stack change as a confounding factor rather than attributing every difference to the model.

---

3. Development-System Constraint

You may:

- use the approved comparison model for planning, implementation, testing, explanation, and review;
- begin new agent sessions when needed;
- use ordinary deterministic development tools;
- use the permitted shared artifacts described in the transfer policy.

You may not:

- copy application code or implementation-specific tests from implementation A;
- use an agent fleet, subagents, or parallel coding agents;
- describe implementation B as a comparison-model result without recording substantial manual correction.

The group remains responsible for every submitted decision and line of code, regardless of who or what generated it.

---

4. Required Work

4.1 Implement the common core again

Complete a fresh implementation of the required domain model, behavior, harness, and acceptance scenarios.

4.2 Repeat the evidence collection

Use the same evidence categories and measurement units as Establish. Preserve failed attempts and human corrections that help explain the result.

4.3 Compare the two single-agent workflows

Compare implementations A and B in terms of:

- acceptance-scenario completion;
- architecture and codebase shape;
- test quality and known defects;
- setup and audit friction;
- elapsed and active human time;
- model latency, usage, cost, or local resource use where measurable;
- amount and type of human intervention;
- optional scope completed.

Do not present correlation as proof. This is a comparison of two engineering cases, not an estimate of the effect of changing the model alone. The group has more domain knowledge during Challenge, and this practice effect is part of the comparison's limitations.

4.4 Apply runtime pressure

Repeat the primary pressure selected in Establish against implementation B using the same workload and procedure where practical. Then select one secondary pressure from the shared catalog that is exposed by an architectural difference or by evidence from A or B. Analyze how both implementations would respond to that secondary pressure; run it when practical and label projections clearly.

Compare the cumulative pressure evidence for A and B:

- Did the new implementation make different architectural assumptions?
- Does it fail or slow down in a different place?
- Did the comparison model introduce infrastructure or abstractions that the evidence does not justify?
- Which implementation responds better to the pressures studied, and according to what evidence?

For one queue, pool, request path, or other conserved-work boundary in A or B, apply Little's Law over one consistent observation window. Record average work in progress (`L`), average completion rate (`λ`), and average time in the system (`W`); check whether `L = λW` is approximately coherent; and state whether arrivals and completions were sufficiently balanced for the relationship to be useful. Use the result to interpret the boundary, not to claim a cause of delay or stability outside the observed conditions.

Connect one failed assumption or approaching limit to a justified response pattern such as an index, derived view, idempotency boundary, bounded queue, partition, quota, backpressure, rate limit, or graceful degradation. At this phase the response may remain a design hypothesis, but name the new correctness, failure, operational, or recovery obligation it would introduce.

Complete the chapter's documented-case transfer and pressure map as part of this cumulative pressure record. Link the selected source, separate reported facts from interpretation and project transfer, trace the failure across the relevant web boundaries, and identify the assumption, amplification path, user-visible effect, first useful signal, recovery mode, and evidence that recovery is complete. For an optional recovery investigation, choose a Challenge exploration starting point.

---

5. Deliverables

Submit the common submission bundle for implementation B plus:

- the architecture hypothesis for implementation B;
- an updated capability ownership card for the main event workflow;
- the documented-case transfer, pressure map, and Little's Law check;
- a concise A–B comparison.

The A–B comparison should distinguish:

- observed differences;
- plausible explanations;
- alternative explanations and confounding factors;
- what the available evidence cannot establish.

---

6. Evaluation Emphasis

This phase emphasizes:

- a genuinely fresh and complete implementation;
- consistent evidence across both phases;
- careful comparison rather than model enthusiasm;
- architectural and scalability reasoning;
- awareness of practice effects, scope changes, and human review.

A larger implementation B is not automatically a better implementation.

Compare Project

Concept connections
Explore definitions and connections in the web edition.
Explore this reading's concepts

Implementation C: Agent Fleet and Final Analysis

The common project specification applies to this phase.

---

1. Purpose

Compare creates a third fresh implementation of the Work Activity Tracker through an agent fleet and then analyzes all three implementations and development workflows.

A fleet is an orchestration strategy, not a model category. The central question is whether decomposition, specialization, and parallelism improve the result enough to justify coordination, duplicated work, integration, review, and resource cost.

Complete and submit implementation C before lecture 9. The public discussion and brief recap during lecture 9 use the fleet debugging views and final comparisons as evidence.

---

2. Starting Conditions

Begin in a new repository or clean implementation directory.

Use:

- the same common specification and acceptance scenarios;
- the transfer policy in the common specification;
- a declared base model;
- normally the same comparison model and primary stack used for Challenge.

Keeping the model and stack stable makes B–C primarily a comparison of orchestration. If the available fleet tooling requires another model or stack, document the difference and treat it as a limitation.

---

3. What Counts as a Fleet

For this project, a fleet must include:

- at least two independently operating agent instances or delegated agent executions;

- explicit responsibility or task boundaries;
- a defined output from each delegated task;
- at least one handoff, integration point, or parallel branch;
- human or agent review before the integrated result is accepted.

Asking one agent to write from several fictional perspectives does not count as a fleet.

More agents are not automatically better. Use the smallest fleet that allows the group to study a real coordination question.

---

4. Fleet Plan

Before implementation, record:

- the task graph or workflow;
- each role or delegated responsibility;
- allowed tools, files, or modules where relevant;
- shared state that can survive between delegated runs, including who can write and later read it, how writes are attributed, how long it persists, and how it is cleared;
- when session orchestration and resource access are separate, their policy ownership and precedence, bypass paths, and the authenticated identity, scoped delegated authority, and task, budget, policy, and trace identifiers passed across the boundary;
- expected output and validation for each task;
- dependencies and possible parallel work;
- who or what owns integration;
- where human approval is required;
- the conditions for reducing or stopping the fleet.

Role names such as scout, planner, implementer, and reviewer are examples, not requirements.

The plan may change. Preserve important changes and explain why they were made.

---

5. Required Work

5.1 Implement the common core again

Complete implementation C through the documented fleet workflow.

5.2 Evaluate coordination

In addition to the common evidence, record where possible:

- number and purpose of delegated tasks;

- work performed in parallel;
- duplicated exploration or implementation;
- handoffs and integration failures;
- conflicts or incompatible assumptions;
- idle time or blocked tasks;
- total model usage across the fleet;
- active human integration and review time;
- defects found after individual agents considered their work complete.

5.3 Turn the fleet trace into debugging data

Export or transform evidence from the actual fleet run into a documented machine-readable trace. JSON is recommended, but another portable format is acceptable. Use the downloadable portable fleet-trace example to study the minimum schema, application mapping, redaction record, and repeat-import contract. It is a transformation target rather than a required internal storage schema or a universal export format.

Map the fleet workflow to the application concepts:

- the project or fleet run becomes a workspace;
- delegated tasks become work items;
- agents and human participants become actors;
- task status maps to the documented lifecycle stages;
- delegation, start, completion, handoff, review, failure, retry, validation, and human intervention become activity events where the source trace supports them.

The trace format may contain additional relationships and metadata, such as parent tasks, dependencies, model information, duration, token use, cost, tool calls, validation results, or error details.

Provide a repeatable way to load the transformed trace into implementation C. Use the application to inspect the run and answer at least two debugging questions, such as:

- Where did work stall, fail, or require a retry?
- Which handoff or dependency delayed progress?
- Where did agents duplicate work or make incompatible assumptions?
- Where did validation or review reject apparently completed work?
- Where and why did a human intervene?
- What became the bottleneck as concurrent work increased?
- Can every physical model or tool attempt, including denied, failed, retried, and cancelled attempts, be linked to its delegated run, actor, policy result, and usage or cost provenance, and can that run be assigned an explicit outcome?

Present the findings through the application's existing feed, filters, summaries, and any small additional view needed to make the evidence understandable. A real-time integration, generic adapter for every agent tool, polished graph editor, and interactive step debugger are not required.

Treat imported traces as potentially sensitive. Remove secrets and unnecessary prompt or source-code content before submission.

5.4 Compare one agent with the fleet

Treat implementations A, B, and C as comparative engineering cases, not estimates of the causal effects of model capability or orchestration. Compare the recorded B and C workflows:

- Which observations suggest a benefit from delegation, and what else could explain them?
- Which tasks were too coupled, ambiguous, or small to delegate?
- Where did accepted throughput differ, under what conditions?
- Where did sequential work or review capacity become the bottleneck?
- Would one careful agent or a human have been preferable for any task?

Apply at least one relevant scalability law or formal model to the workflow. Amdahl's Law, Little's Law, queueing, backpressure, or the Universal Scalability Law may be appropriate.

For one narrower contrast, replace part of the existing fleet rehearsal with a **bounded paired task**:

1. Freeze one accepted repository revision and a small change brief with independent acceptance checks before either attempt. Use two fresh copies of that revision and fresh agent sessions; this probe may reuse B and does not become the starting code for C.
2. Run one single-agent attempt and one bounded fleet attempt using the same base-model version, harness, available tools and permissions, environment, brief, and acceptance checks. Declare the fleet size and decomposition as the treatment being examined.
3. Give both attempts the same predeclared wall-time limit, no more than 20 minutes each, and the same aggregate model-resource limit using the tool's supported token, cost, or request accounting. Count every fleet worker and retry against that limit; record which limit stops each attempt and any accounting gaps.
4. Hold the human-intervention policy constant, preserve failed and partial outcomes, and include integration and independent review in the recorded effort. Rotate condition order across groups where practical; do not transfer solutions or feedback from the first attempt to the second.
5. Compare accepted behavior, elapsed time, human effort, resource use, and correction work. Report one pair as one exploratory contrast; agents within the fleet are not independent trials. Human familiarity and stochastic variation can still affect the result.

This is a small task, not a fourth application or a requirement to repeat a whole build. If matched access or accounting is unavailable, use instructor-supplied paired traces with the same metadata and checks; identify their provenance and missing evidence. An unmatched pair may still be analyzed descriptively, but cannot establish an orchestration effect. Link the pair and its limits in the fleet evidence bundle rather than adding a separate report. To investigate unfinished work or routing decisions further, choose an optional Compare exploration starting point.

5.5 Analyze the three applications

Repeat the primary pressure against implementation C using the same workload and procedure where practical. Compare A, B, and C under that shared pressure. Then use the broader pressure catalog to select only the additional scenarios needed to explain consequential architectural differences. You do not need to apply every catalog scenario exhaustively to every implementation.

Use the final evidence matrix to compare accepted behavior, architecture, pressure, measurement and recovery, development friction, human–agent workflow, and resource use. Judge scalability, reliability, maintainability, simplicity, and resource efficiency from that evidence. Include optional-scope differences and the influence of practice, hardware, stack, tools, and other confounding factors. Link existing phase evidence and mark missing measurements explicitly.

The final result should support a conditional engineering decision, not declare a universal winner.

Apply at least one scalability law or formal model to an application-architecture decision raised by the primary or selected pressures. Name the variables and assumptions, show the calculation or reasoning, connect the result to runtime evidence or a stated pressure, and explain what the model does not capture. This application analysis is separate from the workflow model required in section 5.4.

5.6 Implement and evaluate one scaling response

Select implementation A, B, or C and one pressure supported by runtime evidence or a credibly projected limit that the available environment can exercise. Preserve the accepted revision and its baseline evidence, then implement the smallest justified response to that limit. The response may be an index, bounded cache, idempotency boundary, admission control, queue with an explicit pending state, tenant isolation mechanism, or another technique justified by the evidence. Distributed infrastructure is not required.

Keep the relevant fixture, workload, correctness checks, and measurement procedure unchanged. Rerun the common acceptance scenarios and the selected pressure, report every repetition, and compare latency, throughput, errors, correctness, and the chosen resource-efficiency proxy before and after. Exercise one failure or recovery obligation introduced by the response. State whether the result supported, challenged, or left the original hypothesis unresolved, and preserve an unsuccessful result instead of silently replacing it with a more elaborate design.

5.7 Validate one realistic deployment

Select implementation A, B, or C based on the evidence collected so far. Deploy the revision selected after the scaling-response experiment in an instructor-approved realistic environment outside the author's development process and rerun the shared baseline workload. Record the environment and configuration, operational steps, result, difference from the local run, first observed constraint, and what the run does not prove about larger scale.

---

6. Deliverables

Organize the final submission into four bundles. These are sections in one evidence index, not four new documents or repositories. Link each artifact once and refer to it from the relevant analysis; the common evidence and phase-specific requirements may share the same artifact.

6.1 Implementation

Provide the accepted implementation C revision and its common submission bundle, including setup, configuration, time and resource use, validation, architecture, pressure, and friction evidence. Include the cross-team audit and response, and link the already-submitted A and B revisions and evidence for comparison.

6.2 Fleet evidence

Connect the fleet plan and task graph to a compact selected execution trace, its documented transformation, a repeatable loader into C, and application views answering at least two debugging questions. Include coordination measurements, the B–C case analysis, the bounded paired task or supplied traces, and one workflow-level formal-model analysis. The trace should expose consequential delegation, validation, integration, failure, or human intervention; select evidence that explains the findings rather than submitting an uncurated transcript dump.

6.3 Scaling and deployment experiment

Include one application-level formal-model analysis, the before-and-after evidence for an implemented scaling response, and its new failure or recovery obligation. Add the realistic deployed-workload evidence and identify the selected implementation, revisions, environment, and measurement limits. Sections 5.5–5.7 define the experiment; the application model is separate from the workflow model in the fleet bundle.

6.4 Final decision

Combine the A–B–C evidence matrix with one decision narrative following section 7. Include the target architecture and scaling trade-off table, distinguishing implemented results from remaining proposals. Reference the other bundles and earlier phase evidence instead of repeating their measurements or writing another report for each comparison question.

---

7. Final Comparison

Use the final decision bundle to make one conditional engineering argument:

1. **Establish the comparison.** State what the common specification kept stable and what differed across A, B, and C, including optional scope and measurement gaps.

2. **Explain the consequential results.** Compare application architecture under the shared and selected pressures and development effort through acceptance; use the fleet debugging findings to show what final code or a raw transcript concealed.
3. **Bound the explanation.** Separate A–B–C observations from hypotheses about model capability, practice, architecture, and orchestration. Use the bounded paired task for the narrower workflow contrast, stating what was controlled and what remains unresolved; neither one pair nor the three builds establishes a general effect.
4. **Choose and set a revisit condition.** For stated project constraints, choose an architecture and a local single agent, comparison-model single agent, or fleet; explain rejected alternatives, responsibilities that remained human across the phases, ownership of operations and correction, and the next saturation signal or measurement that would change the decision.

Label important claims as observed, inferred, or proposed. The recap chapter shows how to connect the matrix to this argument.

---

8. Evaluation Emphasis

This phase emphasizes:

- a bounded and inspectable fleet;
- successful integration and validation;
- a useful self-observation loop in which implementation C exposes the workflow that produced it;
- evidence about coordination cost as well as throughput;
- application of scalability concepts to both software and workflow;
- causal humility in the three-way comparison;
- a defensible conditional decision rather than a tool ranking.

Fleet size, transcript length, generated code volume, and architectural complexity do not earn credit by themselves.

Course Development Environments

This file defines the operational choices that must be published for each course run. Model and agent products change faster than the book's architectural and measurement concepts, so the current assignments belong here rather than in permanent chapter prose.

Publication rule

The instructor must complete and publish the table below before the affected project begins. A phase does not begin until every student has a supported access path, a declared fallback for service failure, and a cost policy that does not require unannounced personal spending or hardware.

Phase	Required assignment before launch
Establish	Approved local model artifact or departmental endpoint; resolved revision where exposed; runtime and quantization where applicable; supported hardware or hosted allocation; single-agent harness; context and tool limits; network and data-boundary policy; fallback and support contact
Challenge	Approved second model and endpoint; comparison hypothesis; single-agent harness and enforcement rule; account or credit provision; data-boundary and retention policy; tool limits; fallback and support contact
Compare	Approved fleet harness; declared base model, preferably the Challenge model; maximum concurrent agents and total budget; isolation and external-action policy; trace export or transformation path; fallback and support contact

The instructor should publish one filled table per course run with the following fields:

Field	Establish	Challenge	Compare
Model and resolved revision or declared route	To be announced	To be announced	To be announced
Agent harness and version	To be announced	To be announced	To be announced
Orchestration rule	Single agent	Single agent	To be announced
Supported execution environment	To be announced	To be announced	To be announced
Student cost limit and provided credit	To be announced	To be announced	To be announced
Project-data egress and retention	To be announced	To be announced	To be announced
Tool, network, and external-action limits	To be announced	To be announced	To be announced
Fallback when the assigned service is unavailable	To be announced	To be announced	To be announced
Support and exception path	To be announced	To be announced	To be announced

Publish the course workflow

Before each phase begins, publish the operational details for the group and submission workflow. Use one group record per phase and one private individual portfolio per student and phase.

Detail	Required course-run decision
Groups	Registration location, group identifiers and membership, formation deadline, and route for membership changes.
Submissions	Locations and deadlines for the frozen implementation, audit report, completed evidence bundle, and private individual portfolio; late-work and correction rules.

Detail	Required course-run decision
Class browsing	Assigned-auditor access after the implementation deadline, course-only submission index opening after the audit window, source and evidence access, and handling of approved exceptions.
Audits	Assigned auditing groups, setup and probe time boxes, common tool policy, report destination, and fallback for missing reports or access.
Assessment	Published rubric version, feedback timing, group and individual score breakdowns, completed lecture activities, phase assessment and contribution minima, total and passing-status calculation, and correction history.

Keep the class browsing index focused on technical submissions. Give teaching staff access to the private records needed for assessment and each student access to their own results. Publish the live and remote participation paths and the completion and correction arrangements for mandatory individual work. Record approved changes to an individual requirement alongside any points adjustment. Before launch, test the access paths as an author, a classmate, and an instructor, including access to a frozen source revision.

Verify the assigned workflow

Before launch, run one small task with the published configuration and retain its trace. For Establish and Challenge, confirm that automatic delegation and unapproved model routes are disabled or fail closed; one visible chat window is not evidence of one agent. For Compare, verify child-task attribution, cancellation, usage accounting, workspace isolation, and trace export with the selected harness version. Repeat this check after a material tool or routing change and record the change date.

Comparison rules

- Choose a Challenge model different from Establish and publish the hypothesis the comparison will test; either model may be a foundation model, and hosting or size alone does not establish a capability ranking.
- Keep the Challenge model as the Compare fleet's base model when practical so B-C primarily changes orchestration.
- If a model, route, harness, or hardware allocation changes, record the actual configuration and narrow the comparison claim.
- Do not rank local inference latency across unequal student hardware as though the environment were controlled.
- Manual implementation remains legitimate and must be reported; access failure should not pressure students to conceal work or buy an unapproved service.
- When the assigned tool does not expose a requested configuration field, record it as unavailable rather than estimating it.

This operational sheet may change between course runs. The behavioral specification, evidence categories, and assessment rubric should remain stable unless a revision is announced before students begin the affected phase.

Course Assessment Rubric

This rubric turns the course learning outcomes and project evidence into student-facing scoring rules. It applies to every course run unless the published course environment or learning-platform instructions announce a revision before the affected phase begins.

Group and individual scoring records

Assess one project bundle per group and phase. Every registered member receives that group's project score for the phase; differences in individual contribution are assessed through the twelve-point self- and peer-assessment criteria. Participation and learning-diary points are also individual. The project introduction explains the submission stages and access to classmates' technical work.

For each phase, record the group identifier, membership, assessed implementation revision, criterion scores, brief reasons, and any missing evidence. Link that group result to each member's individual scoring record rather than assessing duplicate project submissions. Keep private individual portfolios together by phase, while scoring their diary entries by lecture and their self- and peer-assessment by criterion.

Give each group its project breakdown and feedback, and each student their individual breakdown, accumulated total, and status against every passing requirement. Show completed lecture activities and each phase's assessment and contribution scores so students can see which individual minimum is still unmet. Confidential peer reports remain available to the teaching staff; the class submission index contains technical work and excludes scores and private student records. Keep the original score and reason when a correction changes a result, and explain the adjustment to the affected students.

Project scoring: 30 points per phase

Each project receives 20 points from common criteria and 10 points from its phase-specific criteria. The same evidence may support several criteria, but the same accomplishment is not counted twice within one criterion.

Common criteria: 20 points

Criterion	Points	Full-credit evidence
Required web behavior and correctness	0-6	The common acceptance core works through the running application; persistence and invalid transitions behave correctly; known defects and missing evidence are reported.
Reproducible setup and operation	0-4	An unfamiliar group can follow the supported setup, run the application and checks, obtain demonstration data, and understand the declared environment and operational limits.
Scalability evidence and reasoning	0-5	Workloads, boundaries, conditions, distributions, limits, signals, formal relationships where required, trade-offs, and unsupported claims are handled explicitly.
Architecture and change capacity	0-3	Authority, request and data paths, contracts, ownership, consequential decisions, and change or recovery obligations are coherent and reviewable.

Criterion	Points	Full-credit evidence
Responsible development-workflow evidence	0-2	Material AI assistance, failed attempts, human correction, acceptance, resource use where measurable, and known limits are reported compactly and honestly.

Use the following general interpretation within each point range:

- **Full credit:** complete, internally coherent, bound to the submitted revision, and supported by relevant evidence.
- **Most credit:** the central outcome is present, but one important condition, check, explanation, or limitation is weak.
- **Some credit:** partial evidence exists, but the result is difficult to reproduce, compare, or defend.
- **No credit:** absent, unusable, contradicted by the artifact, or presented without the evidence needed to assess it.

Score required behavior and correctness independently of visual polish or extra features. A modest implementation can earn full credit when it satisfies the contract and supports its decisions with strong evidence.

Establish-specific criteria: 10 points

Criterion	Points	Full-credit evidence
Local single-agent constraint and project harness	0-4	The declared local-model workflow is followed; instructions, feature and architecture material, commands, and checks make accepted work reviewable; manual work is disclosed.
Baseline and bounded scalability claims	0-4	The two initial claims, shared workload, primary pressure, observation baseline, and evidence limits provide a repeatable starting point for later phases.
Ownership and local-workflow reflection	0-2	The ownership card and reflection distinguish model, harness, hardware or environment, project, and human contributions without turning impressions into causal claims.

Challenge-specific criteria: 10 points

Criterion	Points	Full-credit evidence
Fresh and comparable implementation B	0-3	Implementation B starts cleanly, respects the single-agent and transfer constraints, completes the common core, and records scope or configuration differences.
Architecture, friction, and pressure analysis	0-5	The architecture hypothesis, ownership card, standardized audit, primary and secondary pressures, documented-case transfer, recovery reasoning, and Little's Law check form one coherent analysis.
A-B interpretation	0-2	Observations, plausible explanations, practice effects, stack or hardware differences, and unresolved alternatives are separated clearly.

Compare-specific criteria: 10 points

Criterion	Points	Full-credit evidence
Fleet design, integration, correction, and trace	0-3	Independent agent work, task and authority boundaries, integration, one correction loop, transformed trace, and debugging findings are inspectable and bounded.
Applied scaling and deployment	0-5	Application- and workflow-level models, one implemented before-and-after response, its new failure or recovery obligation, realistic deployment, rerun workload, and target design are supported by evidence.
Final engineering judgment	0-2	The evidence matrix and decision narrative compare A, B, and C, label observation, inference, and proposal, reject alternatives, and state the next boundary and measurement.

Participation and learning diary: 3 points per lecture

Evidence	Points
Participates in the live lecture or completes the announced equivalent remote activity	1
Records one concrete idea, model, failure, or trade-off from the lecture in the learning diary	1
Connects that idea to project evidence, a prior assumption, or a decision the student would revisit	1

The diary entry may be written, audio, or video. Production polish is not graded; specificity and reflection are.

Self- and peer-assessment: 12 points per phase

Criterion	Points	Full-credit evidence
Evidence-based self-assessment	0-3	Identifies concrete contributions, decisions, missed commitments, learning, and next improvement without equating activity with value.
Useful peer feedback	0-3	Gives each teammate specific, respectful feedback connected to observable collaboration, review, integration, or project evidence.
Response to feedback and group reflection	0-3	Summarizes agreements and differences, records one adjustment, and identifies a groupworking condition that affected the result.
Reliable contribution and shared ownership	0-3	Instructor judgment based on corroborated project history, submitted evidence, self-assessment, and peer reports shows that the student contributed and supported shared acceptance of the work.

In each phase's existing self-assessment, answer: “Identify one consequential decision you challenged, tested, or reviewed; link the evidence and explain the outcome.” Use this evidence when assessing contribution and shared ownership; prompt counts, commit counts, and transcript length do not establish learning or contribution.

Peer ratings are evidence, not automatic votes on another student's grade. The instructor resolves unsupported, retaliatory, mutually inflated, or contradictory reports against the project record and may ask for clarification. Private circumstances should be handled through the course's ordinary support process rather than disclosed to teammates in detail.

Mandatory individual work

Individual participation and assessment are required in every phase. Check these requirements using the existing private portfolios and rubric scores.

Requirement	Minimum in each phase	Evidence used
Lecture participation and learning diary	Two of the three lecture activities, for six across the course	Each counted activity earns all three existing points: live participation or the announced equivalent remote activity, one concrete idea, and its connection to evidence, an assumption, or a decision.
Self- and peer-assessment	A submitted assessment earning at least 6 of 12 points, including at least 1 of 3 points for reliable contribution and shared ownership	The existing self-assessment, peer feedback, group reflection, and corroborated contribution evidence.

Diary points alone do not satisfy the participation requirement. Record live or remote participation separately from the diary content. The contribution minimum is an instructor judgment against the existing evidence and criterion, with peer reports used as evidence.

Use the published completion and correction arrangements for missing required work. An approved absence or support arrangement must specify the replacement activity or adjusted individual requirement as well as any change to available points. Record and explain the arrangement in the student's private scoring record.

Passing and final grade

The maximum is 153 points:

- projects: `3 × 30 = 90`;
- participation and learning diaries: `9 × 3 = 27`;
- self- and peer-assessment: `3 × 12 = 36`.

To pass, a student must:

1. receive at least 10 points from each of the three projects;
2. meet the mandatory lecture participation and diary requirement in every phase;
3. meet the mandatory self- and peer-assessment requirement in every phase; and
4. receive at least 50% of the available total points.

A student whose total reaches a grade threshold but who has an unmet mandatory requirement has not passed the course.

Let `p = 100 × earned_points / available_points` after any formally approved absence adjustment. Once all passing requirements are met, the final grade is:

Percentage	Grade
Below 50%	Fail
50% to below 60%	1
60% to below 70%	2
70% to below 80%	3
80% to below 90%	4
90% or above	5

Keep the original evidence and scoring record when a correction or approved adjustment changes a result.